

SERVIÇO PÚBLICO FEDERAL
UNIVERSIDADE FEDERAL DE GOIÁS
INSTITUTO DE INFORMÁTICA

PROJETO PEDAGÓGICO DE CURSO

Engenharia de Software

Diretor Eliomar Araújo de Lima

Vice-Diretor Iwens Gervásio Sene Junior

Coordenação de curso Nádia Félix Felipe da Silva

Vice-coordenação de curso Leonardo Andrade Ribeiro

Coordenação de estágio não-obrigatório Taciana Novo Kudo

Vice-coordenação de estágio não-obrigatório Luciana de Oliveira Berretta

Núcleo Docente Estruturante: Leonardo Antônio Alves (vice-presidente)
Leonardo Andrade Ribeiro
Luciana de Oliveira Berretta
Nádia Félix Felipe da Silva
Renato de Freitas Bulcão Neto (presidente)
Sofia Larissa da Costa Paiva
Taciana Novo Kudo

Goiânia – Goiás – Brasil
1º de dezembro de 2023

Sumário

1. Apresentação **3**
2. Exposição de motivos **4**
2.1. Contexto educacional e características locais 6
2.2. Demandas socioeconômicas 8
3. Objetivos do curso **9**
3.1. Objetivo geral 9
3.2. Objetivos específicos 9
4. Perfil do curso **10**
4.1. Formação ética e postura profissional 11
4.2. Desenvolvimento de conteúdos 12
4.3. Articulação entre teoria e prática 13
4.4. Interdisciplinaridade 16
5. Perfil do egresso **18**
5.1. Competências cognitivas do egresso 19
5.2. Competências comportamentais do egresso 20
5.3. Competências técnicas do egresso 22
6. Estrutura curricular **26**
6.1. Matriz curricular e fluxo sugerido 28
6.2. Componentes curriculares optativos 31
6.3. Pré-requisitos das disciplinas obrigatórias e optativas 31
6.4. Ementas das disciplinas com bibliografias básica e complementar 33
6.5. Equivalência de disciplinas e transição entre currículos 98
7. Políticas e procedimentos de gestão do curso **104**
7.1. Política e gestão de estágio curricular 104
7.2. Política de inserção de ações curriculares de extensão - ACEx 105
7.3. Políticas de ensino, pesquisa e extensão 108
7.4. Trabalho de Conclusão de Curso 109
7.5. NDE, monitoração e avaliação de ensino-aprendizagem e apoio a discentes 110
7.6. Gestão do curso e avaliação interna e externa 113
7.7. Política de qualificação do corpo docente e técnico-administrativo 113
7.8. Requisitos legais e normativos do curso 115
Referências **121**

Lista de abreviaturas e siglas

ACEx	Atividades Curriculares de Extensão
ACM	Association for Computing Machinery
BES	Bacharelado em Engenharia de Software
CAPES	Coordenação de Aperfeiçoamento de Pessoal de Nível Superior
CEPEC	Conselho de Ensino, Pesquisa, Extensão e Cultura da UFG
CH	Carga Horária
CH ACEX	Carga Horária de Atividades Curriculares de Extensão
CH Pra	Carga Horária Prática
CH Teo	Carga Horária Teórica
CH ToT	Carga Horária Total
CNPq	Conselho Nacional de Desenvolvimento Científico e Tecnológico
CONSUNI	Conselho Universitário
CPA	Comissão Própria de Avaliação
DCN	Diretrizes Curriculares Nacionais
ENADE	Exame Nacional de Desempenho de Estudantes
FL	Faculdade de Letras
IEEE	Institute of Electrical and Electronics Engineers
IME	Instituto de Matemática e Estatística
INEP	Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira
INF	Instituto de Informática
MEC	Ministério da Educação
NC	Núcleo Comum (disciplina de núcleo comum)
NDE	Núcleo Docente Estruturante
NE	Núcleo Específico (disciplina de núcleo específico)
NL	Núcleo Livre (disciplina de núcleo livre)
OBR	Disciplina de natureza obrigatória
OPT	Disciplina de natureza optativa
P&D	Pesquisa e Desenvolvimento
PPC	Projeto Pedagógico de Curso
Pré-req	Pré-requisito
RGCG	Regulamento Geral dos Cursos de Graduação
TCC	Trabalho de Conclusão de Curso
TIC	Tecnologia da Informação e Comunicação
UA	Unidade Acadêmica
UFG	Universidade Federal de Goiás

1. Apresentação

Nome do curso	Engenharia de Software
Grau acadêmico	Bacharelado
Classificação Internacional Normalizada da Educação, adaptada para os cursos de graduação e sequenciais do Brasil (Cine Brasil 2018)	Área geral: 06 Computação e Tecnologias da Informação e Comunicação (TIC) Área específica: 061 Computação e Tecnologias da Informação e Comunicação (TIC) Área detalhada: 0613 Produção de software Rótulo Cine Brasil: 0613E01 Engenharia de software
Área de conhecimento (CNPq/CAPES)	Ciências Exatas e da Terra (1.00.00.00-3) Ciência da Computação (1.03.00.00-7) Metodologia e Técnicas da Computação (1.03.03.00-6) Engenharia de Software (1.03.03.02-2)
Modalidade	Presencial
Local de oferta	Instituto de Informática da Universidade Federal de Goiás Alameda Palmeiras, Quadra D, Câmpus Samambaia Goiânia - Goiás, CEP 74690-900
Reconhecimento	Portaria N° 819 de 29 de outubro de 2015. Número de registro no e-MEC: 201307356.
Unidade Acadêmica Responsável	Instituto de Informática (INF)
Unidades executoras	Instituto de Informática (INF) Instituto de Matemática e Estatística (IME) Faculdade de Letras (FL)
Número de vagas	40 vagas anuais
Carga horária total	3200 h
Duração do curso em semestres	Mínima: 9 semestres Máxima: 14 semestres
Turno de funcionamento	Predominantemente noturno
Forma de ingresso	Sistema de Seleção Unificada (SiSU). Em caso de existência de vagas é possível o ingresso por: (a) transferência de outra instituição de ensino superior; (b) portar diploma de curso superior; ou (c) reingresso em curso superior. Essas opções dependem de processo seletivo específico conduzido pela UFG.

2. Exposição de motivos

O presente documento define o Projeto Pedagógico do Curso (PPC) do curso de Bacharelado em Engenharia de Software (BES) do Instituto de Informática (INF) da Universidade Federal de Goiás (UFG), elaborado segundo diretrizes da Instrução Normativa n. 01/2022 [UFG 2022]. O propósito do PPC é orientar as ações educativas do curso e definir seus fundamentos políticos, filosóficos e teórico-metodológicos, objetivos, o tipo de organização e as formas de implementação e avaliação do curso.

O BES foi criado em 2008 [CONSUNI 2008], tendo sido o primeiro curso de graduação nesta área no País [LOV 2008]. As revisões realizadas sobre o PPC original, a primeira feita em 2017 [CEPEC 2017] e a segunda descrita no presente documento, refletem a busca contínua pela qualidade e pelo aprimoramento das teorias e bases pedagógicas que norteiam o curso.

A motivação para esta segunda revisão do PPC de BES precisa ser compreendida a partir de uma perspectiva histórica do curso e do cenário no qual ele está inserido na atualidade, pois como observa Paulo Freire, “toda leitura de texto pressupõe uma rigorosa leitura do contexto”.

Não houve, desde a criação do BES em 2008, qualquer alteração relevante de objetivos do curso ou do perfil esperado de seus egressos. Contudo, muitos detalhes precisaram ser refinados para atender aos novos requisitos legais e normativos, com destaque para aqueles estabelecidos nas Diretrizes para a Extensão na Educação Superior Brasileira [CNE/CES 2018].

Para se adequar aos novos requisitos e à evolução do contexto acadêmico e social em que atua, o PPC do BES foi avaliado e reestruturado com a realização das seguintes alterações, que justificam a criação desta nova versão do PPC:

- Mudança no fluxo sugerido para os quatro primeiros períodos, que deixa de conter apenas disciplinas de núcleo comum, visando a formação básica em Computação e Matemática, para contemplar disciplinas do núcleo específico de formação em Engenharia de Software.

- Ampliação da carga horária de formação específica em Engenharia de Software, com mudança de conteúdos do núcleo comum obrigatório para disciplinas optativas e, no sentido inverso, transformando em disciplinas obrigatórias alguns conteúdos de disciplinas optativas.
- Redução de três para duas disciplinas optativas no curso, acompanhada de ampliação significativa no número de disciplinas optativas que podem ser escolhidas pelos discentes e na variedade de áreas de conhecimento contempladas por essas disciplinas.
- Inclusão das atividades curriculares de extensão (ACEx), que são integralmente contempladas na disciplina "Prática em Engenharia de Software", com carga horária de 320 horas.
- Ênfase em competências, contemplando conhecimentos, habilidades e atitudes, na formação do perfil do egresso e não apenas em conteúdos e teorias dos componentes curriculares. Em particular, foram explicitadas as atitudes éticas, a postura profissional e as capacidades cognitivas e comportamentais (*soft skills*) que compõem o perfil do egresso.

Essas modificações, que aprimoram e adequam o PPC às normas e leis vigentes, não representam mudanças isoladas e superficiais sobre o currículo anterior, mas um processo de ampla reflexão, liderado pelo Núcleo Docente Estruturante (NDE) do curso e fundamentado nas experiências e lições aprendidas por dezenas de docentes do curso ao longo de mais de uma década de atividades acadêmicas.

Insumos empregados na elaboração das mudanças incluem as Diretrizes Curriculares Nacionais (DCN) para os cursos da área de Computação [MEC 2016], os referenciais de formação para cursos de graduação em Computação da Sociedade Brasileira de Computação (SBC) [ZNM 2017] e os paradigmas para educação global em Computação [ACM/IEEE 2020]. Projetos pedagógicos recentes de cursos semelhantes, inclusive de outros países, também foram investigados, compondo um substancial corpo de informações que embasou a análise do currículo anterior e, à luz da experiência dos docentes do BES e do contexto local, a proposição das mudanças que formam este novo PPC do BES.

Embora as mudanças realizadas constituam uma evolução significativa para o PPC, elas não alteram o foco original do curso, que é a formação de egressos

aptos a aplicar, de forma efetiva, ética e com postura profissional, o conhecimento da Engenharia de Software.

As atividades do BES são desenvolvidas em um contexto em que se destaca, por um lado, a aptidão do Instituto de Informática (INF) para oferta de cursos de excelência na área da Computação e, por outro lado, as demandas de natureza econômica e social por cursos nesta área, onde é notável a carência de mão de obra capacitada para desenvolver e sustentar softwares de qualidade. Estes dois aspectos reforçam a importância do BES e são detalhados nas seções seguintes.

2.1. Contexto educacional e características locais

O INF possui sólida experiência no ensino superior na área da Computação, iniciada em 1983, com a criação do curso de Bacharelado em Ciência da Computação. Em 2009 foram criados mais dois cursos de graduação nesta área: o BES e o Bacharelado em Sistemas de Informação. Finalmente, em 2019, o INF criou o curso de Bacharelado em Inteligência Artificial que, assim como o BES, também foi o primeiro curso de sua natureza no País.

Além de ser o responsável por estes quatro cursos de graduação, o INF também ministra dezenas de disciplinas da área de Computação para diversos cursos de graduação de várias unidades acadêmicas da UFG. Na perspectiva de ensino de pós-graduação, a história do INF é similar, com ofertas concluídas de dezenas de edições dos mais variados cursos de especialização na área de Computação. Na pós-graduação *stricto sensu* o INF oferece, desde 2001, um curso de Mestrado em Ciência da Computação e, a partir de 2010, passou a oferecer o curso de Doutorado em Ciência da Computação. Assim, ao longo de mais de três décadas de atividades de ensino superior em Computação, tanto na graduação quanto na pós-graduação, o INF formou milhares de egressos, muitos deles ocupando cargos de destaque em universidades e organizações no Brasil e no exterior.

Nas perspectivas de pesquisa e extensão, o INF construiu, ao longo de sua história, um consistente envolvimento com empresas e organizações de Tecnologia da Informação e Comunicação (TIC). Um número significativo de projetos de Pesquisa e Desenvolvimento (P&D) já foi realizado pelo INF em

parceria com organizações locais, e alguns com empresas multinacionais (como Dell e HP, por exemplo). O INF também já participou de vários projetos de inovação tecnológica financiados pela Fundação de Apoio à Pesquisa do Estado de Goiás (FAPEG). O volume e intensidade das interações envolvendo pesquisa e extensão com organizações locais e nacionais levou o INF a criar órgãos complementares específicos para cooperação e interação com organizações externas, com destaque para:

- Instituto APOEMA: órgão complementar de P&D vinculado ao INF que desenvolve projetos para incorporação de novas tecnologias, processos e serviços que contribuem para a consolidação do pólo de TIC de Goiás. Atua em parcerias firmadas pelo INF com instituições, públicas ou privadas, nacionais ou estrangeiras, visando à colaboração, contratos e convênios. Busca parceiros para investimentos em projetos e o fornecimento de soluções inovadoras na área de TIC. Dessa forma, o órgão promove a integração entre ensino, pesquisa e extensão no INF, proporcionando experiência prática aos discentes e capacitando-os para a atuação profissional [APOEMA 2022].

- Centro de Excelência em Inteligência Artificial (CEIA): criado em parceria com a FAPEG, é uma unidade EMBRAPII – Empresa Brasileira de Pesquisa e Inovação Industrial - Organização Social Supervisionada pelo Ministério da Ciência, Tecnologia e Inovação (MCTI) – desde 2020. Sua missão é disponibilizar e transferir tecnologias baseadas em Inteligência Artificial, promovendo a competitividade de organizações públicas e/ou privadas por meio do desenvolvimento de soluções tecnológicas inovadoras com foco nos setores de saúde, logística, segurança, energia e cidades inteligentes [CEIA 2022].

- Fábrica de Software: órgão complementar vinculado ao INF que provê um ambiente profissional de produção de software, alojado em um ambiente educacional, com atribuições técnico-científicas relacionadas a software. Suas atribuições abrangem planejamento, administração, operação, manutenção e evolução de um ambiente tecnológico adequado para produção e sustentação de software de alta qualidade. A Fábrica disponibiliza e mantém, no ambiente acadêmico do INF, uma infraestrutura para produção e sustentação de software que integra as

melhores práticas aplicadas pela indústria de software às teorias e inovações geradas pela academia. Suas atividades são desenvolvidas por equipes técnicas multidisciplinares orientadas por processos definidos e institucionalizados, em alinhamento com os planejamentos institucionais do INF e da UFG [FÁBRICA 2022].

2.2. Demandas socioeconômicas

A demanda por profissionais especializados na produção de software vem crescendo de forma contínua, não apenas no Estado de Goiás ou no Brasil, mas em todo o mundo. Apesar dessa crescente demanda, na ocasião da criação do BES, não havia outro curso com a mesma denominação ou propósito no Brasil. A tradição do INF na socialização da Computação, juntamente com a forte atuação do seu corpo docente na área de Engenharia de Software, resultou em proposta pioneira no País de um curso especificamente voltado para a formação de Engenheiros de Software. Esta iniciativa pioneira do INF inspirou a criação de cursos similares, que hoje existem em todas as regiões do País.

A região da Grande Goiânia, em particular, reúne centenas de empresas de TIC, mas tem participação pouco expressiva no mercado de software nacional, e ainda menos no contexto internacional. Espera-se que a disponibilidade de profissionais qualificados possa ampliar essa participação, tornando-a compatível com a dimensão do Estado de Goiás, que está entre as dez maiores economias dos estados brasileiros. Portanto, o BES tem potencial para promover a indústria de software em Goiás, possibilitando o aumento da participação do estado em uma atividade econômica que está em contínua expansão, que são as tecnologias baseadas em software.

Na perspectiva social, o BES cria a possibilidade de acesso ao ensino superior para muitos que não podem usufruir do ensino privado. A política de quotas para ingresso no curso, estabelecida pela UFG, garante que metade das vagas do BES sejam destinadas a pessoas com algum tipo de vulnerabilidade social, o que representa uma real possibilidade de melhoria da situação socioeconômica dessas pessoas, a partir da formação em uma profissão que é cada vez mais valorizada pela sociedade como um todo.

Além disso, o curso funciona em turno predominantemente noturno, de forma que há possibilidade de os discentes realizarem atividades diurnas que lhes proporcionem estabilidade financeira para se manterem como estudantes do ensino superior. Nesse sentido, vale ressaltar que o mundo do trabalho relacionado a software não deve ser considerado apenas sob visão local ou regional, uma vez que a abertura de postos de trabalho na modalidade a distância tem crescido significativamente em um cenário de trabalho global para Engenharia de Software, sem restrições ou barreiras geográficas. Com isso, o discente do BES pode realizar estágio ou mesmo trabalhar formalmente em organizações localizadas em qualquer lugar do planeta, usando recursos tecnológicos para trabalho colaborativo e remoto.

Uma das mudanças propostas no presente PPC é justamente orientada à aplicação desses recursos tecnológicos em atividades de ensino, notadamente naquelas relacionadas com a prática da Engenharia de Software, preparando o discente para atuar em um mundo de trabalho globalizado e mediado por tecnologias de apoio ao trabalho remoto.

3. Objetivos do curso

3.1. Objetivo geral

O BES tem como objetivo fundamental:

> **Formar profissionais aptos a contribuir efetivamente para a produção e sustentação de software de qualidade, seguindo os princípios éticos da Engenharia de Software.**

3.2. Objetivos específicos

A formação de egressos segundo o objetivo geral do curso deve contribuir para realizar os seguintes objetivos específicos:

- Reconhecer e valorizar a diversidade, conforme Art. 3.º, inciso IV, da Constituição Federal: "*promover o bem de todos, sem preconceitos de origem, raça, sexo, cor, idade e quaisquer outras formas de discriminação*".
- Estimular relações étnico-raciais respeitosas com base na socialização da história da formação dos brasileiros.

- Ampliar a compreensão sobre questões ambientais e valorizar projetos sustentáveis.
- Aplicar postura profissional e ética em trabalhos envolvendo operação, produção, modificação, aquisição ou fornecimento de software.
- Contribuir para atendimento da demanda da sociedade por softwares de qualidade.
- Conceber, desenvolver e sustentar produtos inovadores com base em software.
- Promover o uso das melhores práticas de engenharia de software na indústria como um todo e, particularmente, na indústria regional.
- Fomentar a produção de software como atividade econômica de alto valor agregado.
- Criar alternativas econômicas e talentos humanos para fortalecer a indústria goiana de software.
- Aumentar a capacidade tecnológica e o capital intelectual do País na área de software.
- Disponibilizar para a sociedade cidadãos capacitados para lidar com software e tecnologias correlatas.

4. Perfil do curso

O perfil do BES baseia-se em princípios estabelecidos nas Diretrizes Curriculares Nacionais (DCN) para os cursos da área de Computação [MEC 2016], nos referenciais de formação para cursos de graduação em Computação da Sociedade Brasileira de Computação (SBC) [ZNM 2017] e nos paradigmas para educação global em Computação [ACM/IEEE 2020].

O propósito do perfil definido para o curso é promover o desenvolvimento de conteúdos didáticos focados em estratégias de aprendizagem adequadas, com acompanhamento permanente das atividades, garantindo a acessibilidade metodológica e a autonomia dos discentes, alinhadas com práticas pedagógicas inovadoras e embasadas em recursos que proporcionam aprendizagens diferenciadas dentro da área e que buscam estimular a ação dos discentes em uma relação teoria-prática equilibrada.

4.1. Formação ética e postura profissional

A vivência de princípios éticos e o conhecimento sobre a postura profissional do(a) bacharel(a) em Engenharia de Software são elementos essenciais para a formação do perfil esperado do egresso do BES. Ética é um assunto transdisciplinar que perpassa todos os componentes curriculares do curso. Em particular, este assunto é explicitamente abordado no primeiro período do curso, na disciplina "Software, Empreendedorismo e Sociedade", que introduz o Código de Ética e Prática Profissional da Engenharia de Software, cujos princípios são [ACM/IEEE 1999]:

- Em relação ao **público** - agir de forma consistente com o interesse público é o princípio primordial da ética da Engenharia de Software.
- Em relação ao **cliente** e ao **empregador**- agir e decidir visando ao melhor interesse de seus clientes e empregadores.
- Em relação ao **produto de software** - garantir que seus produtos e modificações atendam aos mais altos padrões profissionais possíveis.
- Em relação ao **julgamento** e **avaliação** - manter a integridade e a independência em seu julgamento profissional.
- Em relação à **gestão** - quando atua como gerente ou líder, seguir e promover uma abordagem ética para a gestão de processos de Engenharia de Software.
- Em relação à **profissão** - promover a integridade e a reputação da profissão.
- Em relação aos **colegas** - ser justo e apoiar seus colegas de profissão.
- Em relação a **si mesmo** - aprender continuamente sobre a prática de sua profissão e promover uma abordagem ética para esta prática.

O assunto ética volta a ser explicitamente tratado, com foco em aspectos relacionados à postura profissional, na disciplina do último período do curso, "Prática em Engenharia de Software". Vale ressaltar que o comportamento ético e profissional é trabalhado nas diversas atividades do curso, e não apenas em disciplinas que abordam explicitamente este conteúdo. Em qualquer disciplina do BES, a postura ética é critério a ser observado para aprovação do discente na disciplina. Também é esperada e monitorada pelo NDE do BES a postura ética dos docentes que atuam no curso, que devem

atuar junto aos discentes com base em conduta ética, servindo de exemplo constante da postura profissional esperada do egresso.

A exigência de uma conduta apropriada, dentro e fora da sala de aula, tanto dos docentes quanto dos estudantes, contribui para a formação ética e para o desenvolvimento das atitudes e comportamentos esperados do egresso do BES. Isso significa, por exemplo, promover a qualidade de vida e o respeito à diversidade e ao meio ambiente. Nesse sentido, não apenas as atividades nas disciplinas, mas todas as ações acadêmicas no curso, devem ser pautadas pela reflexão do seu impacto no contexto no qual se inserem.

4.2. Desenvolvimento de conteúdos

A formação proposta para o BES está alinhada com as diretrizes nacionais e internacionais para cursos da área da Computação [MEC 2016, ZNM 2017, ACM/IEEE 2020]. Os conteúdos específicos da Engenharia de Software se fundamentam em referências internacionalmente reconhecidas e consolidadas, com destaque para o guia para o corpo de conhecimento da Engenharia de Software [SWEBOK 2023], o modelo de competência da Engenharia de Software [SWECOM 2014] e a norma internacional de processos do ciclo de vida do software [ISO/IEC/IEEE 2017]. A organização e a aplicação dos conteúdos na estrutura curricular do BES seguem as orientações dos referenciais nacionais e internacionais citados, valorizando a formação em conteúdos gerais de Matemática e Computação, e enfatizando os conteúdos específicos da Engenharia de Software.

As estratégias de aprendizagem do curso se baseiam na construção concomitante do conhecimento de conteúdos gerais e específicos, e não na separação desses conteúdos em duas fases (formação comum e formação específica), como ocorria no PPC anterior. Esta mudança reflete experiências relatadas por docentes, mostrando dificuldade em separar conteúdos gerais e específicos em etapas distintas do curso e, principalmente, a desmotivação relatada por estudantes, por não compreenderem a verdadeira essência do curso até o final da etapa de conteúdos gerais. Portanto, este PPC propõe uma visão holística para o aprendizado da Engenharia de Software, mesclando conteúdos de base (disciplinas de núcleo comum), que podem estar

associados a outras áreas de conhecimento, com conteúdos específicos da Engenharia de Software, ao longo de todos os períodos do curso.

As disciplinas do BES que são específicas da Engenharia de Software são divididas em dois grupos. O primeiro grupo representa as subáreas do conhecimento da Engenharia de Software, notadamente aquelas definidas em [SWEBOK 2023], como "Requisitos de Software" e "Arquitetura de Software". Cada subárea é contemplada por pelo menos uma disciplina específica, que tem como foco principal de interesse o conteúdo desta subárea. Todavia, este conteúdo não é isolado somente nesta disciplina do curso, mas é revisitado em outras disciplinas, refletindo as interdependências entre conhecimentos na Engenharia de Software. A disciplina "Teste de Software", por exemplo, enfatiza o conteúdo de testes, mas inclui aspectos de *design* de software (associado a testes de integração) e de requisitos de software (associado a testes de aceitação). Em síntese, as disciplinas não se limitam a uma projeção cartesiana das subáreas de conhecimento da Engenharia de Software propostas em [SWECOM 2014]. Em vez disso, cobrem reiteradamente, com conteúdos complementares, essas subáreas.

O segundo grupo do núcleo específico do curso contempla disciplinas que abordam as principais aplicações práticas do corpo de conhecimento da Engenharia de Software. Neste grupo aparecem disciplinas como "Software para Sistemas Ubíquos" e "Software para Persistência de Dados" e se destaca a disciplina "Prática em Engenharia de Software", cujo propósito é aplicar as competências desenvolvidas durante o curso para a produção e/ou evolução de software, em ambiente de produção realista e visando ao atendimento de demandas da sociedade.

4.3. Articulação entre teoria e prática

O equilíbrio entre teoria e prática é uma característica essencial da estratégia de aprendizagem adotada no curso, pois o egresso do BES precisa ter envolvimento com o fazer, ou seja, com a aplicação do conhecimento da Engenharia de Software. Para isso, as ações pedagógicas previstas nos componentes curriculares exercitam tanto a aquisição de conhecimentos teóricos quanto a capacidade de aplicar, na prática, estes conhecimentos para realizar atividades típicas do profissional de Engenharia de Software. Essa

articulação entre teoria e prática é explicitada na estrutura curricular do curso pela definição de "*competências mínimas*" em cada disciplina específica da Engenharia de Software, estabelecendo critérios que o egresso precisa alcançar para ter aproveitamento na disciplina, que contemplam o domínio de conhecimentos teóricos e a capacidade de aplicá-los para resolver problemas práticos.

Vale destacar que o termo "*competência*" utilizado neste PPC engloba três aspectos: *conhecimentos* (o que se sabe), *habilidades* (o que se consegue fazer com o saber) e *atitudes* (os comportamentos e disposições para fazer algo). O egresso do BES deve demonstrar não apenas conhecimentos em Engenharia de Software, mas também capacidade para aplicar esses conhecimentos com atitudes compatíveis com o código de ética e a postura profissional da Engenharia de Software.

Assim, o PPC determina que, aquilo que convencionalmente é chamado de conteúdo (ou conhecimento teórico), e que consta na ementa de cada disciplina, deve ser exercitado pelo discente, com um nível bem definido de proficiência, e deve ser acompanhado e avaliado pelo docente, de forma objetiva e explicitada no plano da disciplina. A ementa de cada disciplina descreve conhecimentos (conceitos, definições, leis e teorias) que o discente deve saber e relatar e as competências mínimas especificam capacidades que o discente deve exercitar e demonstrar, ou seja, enquanto a ementa define o conteúdo do estudo, as competências mínimas definem a capacidade esperada do discente em relação a tal conteúdo. Para ser aprovado em uma disciplina, o discente deve demonstrar sua competência em cada disciplina, em termos de conhecimentos, habilidades e atitudes. Logo, há três aspectos de competência que devem ser desenvolvidos e avaliados em todas as disciplinas:

- **Conhecimentos teóricos**: aquilo que se sabe ou se conhece.

  Envolve capacidades cognitivas para desenvolver raciocínio, resolver problemas, abstrair, aprender, explicar, avaliar, discutir, analisar, caracterizar, modelar, sintetizar, descrever e correlacionar os conhecimentos propostos na ementa.

  Toda disciplina tem como competência teórica mínima:

    - Explicar os conceitos e o conteúdo da ementa da disciplina e a sua relação com as demais áreas da Engenharia de Software e das ciências correlatas, notadamente Matemática e Computação.

- **Habilidades práticas**: aquilo que se consegue fazer com o conhecimento.

  Envolve capacidades para realizar, planejar, executar, aplicar, verificar, realizar, criar, desenvolver, programar, modificar, projetar, construir, testar, depurar, modelar, derivar, refatorar e validar produtos e processos de Engenharia de Software relacionados à ementa.

  Toda disciplina tem como competência prática mínima:

    - Aplicar os conceitos e o conteúdo da ementa da disciplina para realizar processos, atividades, técnicas, procedimentos e métodos de Engenharia de Software, gerando produtos ou serviços de software de qualidade.

- **Atitudes comportamentais**: a forma como se aplica o conhecimento.

  Envolve capacidades para agir com ética e postura profissional, trabalhar em equipe, liderar, comunicar, inovar, aprimorar aptidão, iniciativa, entusiasmo, boa vontade, confiabilidade e sensibilidade cultural ao exercer as competências definidas pela ementa.

  Toda disciplina tem como competência comportamental mínima:

    - Realizar atividades teóricas e práticas com base nos conceitos e no conteúdo da ementa da disciplina, demonstrando atitudes consistentes com o código de ética e postura profissional da Engenharia de Software.

Para ter aproveitamento em qualquer disciplina, o discente deve ser avaliado nestes três aspectos de competência, no nível de profundidade e abrangência definido para a disciplina pelo PPC.

4.4. Interdisciplinaridade

Produzir software exige o emprego de pelo menos dois domínios de conhecimento: o domínio da solução, que envolve o corpo de conhecimento da Engenharia de Software; e o domínio do problema, que apresenta as demandas e necessidades de qualquer área de conhecimento (por exemplo, saúde, educação, segurança, governo, finanças, agricultura, comércio, entretenimento) que justificam uma solução baseada em software. A interdisciplinaridade do BES é inerente às interações entre esses dois domínios de conhecimento, já que o software existe para resolver problemas em uma área de aplicação. Produzir, sustentar e evoluir software envolve, necessariamente, interações entre diferentes áreas do conhecimento (domínio do problema) que constituem a área de aplicação do software e conhecimentos de Engenharia de Software (domínio da solução) que constroem e mantêm o software operacional e útil para aquela aplicação.

O BES valoriza a autonomia do discente no acesso a conhecimentos de outras áreas, possibilitando a livre escolha de disciplinas de Núcleo Livre (NL). Para a integralização curricular o discente deve concluir pelo menos 128 horas em disciplinas de NL, livremente escolhidas pelo discente dentre as dezenas oferecidas semestralmente, em todas as áreas de conhecimento, pela UFG. Convém destacar que esta carga horária definida pelo PPC constitui a quantidade mínima exigida e que o discente pode cursar uma carga horária maior, se assim o desejar. O extenso e diversificado conjunto de opções de disciplinas de outras áreas do conhecimento contribui para uma formação ampla do discente e para a compreensão das possibilidades de interação entre a Engenharia de Software e as demais áreas de conhecimento.

Outros elementos do BES que promovem a interdisciplinaridade e a autonomia do discente em relação à sua formação, estimulando a ação discente em uma relação teoria-prática, são o estágio curricular, não obrigatório, as disciplinas optativas e as atividades complementares necessárias para a integralização curricular. O discente que opta por realizar o estágio curricular não obrigatório tem a oportunidade de vivenciar atividades profissionais da Engenharia de Software ainda como estudante, aprendendo, na prática, as relações entre domínios de conhecimento que são inerentes a essas atividades.

As disciplinas optativas do BES são vinculadas a assuntos variados, com temáticas contextuais de áreas específicas, relevantes para a formação do discente. Dentro do conjunto de disciplinas optativas definido neste PPC há aquelas que compõem o currículo de outros cursos da área de computação, com destaque para Sistemas de Informação e Ciência da Computação e que reforçam a interdisciplinaridade do BES com estes cursos. O discente pode escolher livremente as optativas que deseja cursar dentro deste conjunto, exercendo sua autonomia para direcionar sua formação para áreas mais específicas, de acordo com os seus interesses individuais.

Em relação às atividades complementares, o discente deve cumprir um mínimo de 192 horas em atividades escolhidas pelo próprio discente, dentro de um rol de tipos de atividade que integram ensino (monitoria, por exemplo), P&D (incluindo iniciação científica, estudando domínios de problemas que podem ser resolvidos com software) e extensão (como participação em projetos de extensão na UFG, que abordam as mais diversas áreas de conhecimento).

Vale ressaltar que, além de estimular a interdisciplinaridade entre áreas de conhecimento distintas, o BES também promove relações interdisciplinares entre os diferentes componentes curriculares do curso. As competências desenvolvidas em cada disciplina envolvem a aplicação de competências correlatas, adquiridas em outras disciplinas do curso, culminando com o componente curricular “Prática em Engenharia de Software”, cuja natureza envolve a aplicação conjunta das competências adquiridas ao longo do curso para criação de software e transferência tecnológica visando ao atendimento de demandas da comunidade externa à UFG.

Este componente curricular tem 320 horas e garante a participação do discente em um ou mais projetos integradores de Engenharia de Software que envolvem problemas reais da comunidade, cuja solução envolve, total ou parcialmente, construção, modificação ou aplicação de software. Esta carga horária é integralmente computada como Atividade Curricular de Extensão (ACEx), já que as atividades desenvolvidas estão alinhadas com a regulamentação das ACEx nos PPCs da UFG [CEPEC 2021], ou seja, têm como público principal a comunidade externa, constituem um processo formativo, articulado ao ensino e à pesquisa, estimulam e potencializam as

relações entre a universidade e outros setores da sociedade e promovem a participação estudantil por meio de sua inclusão como membro da equipe executora de ações de extensão.

Adicionalmente, e sempre respeitando a liberdade metodológica de cada docente, o NDE deve orientar a definição de programas de disciplinas que cultivem a interdisciplinaridade e que sejam inovadores e embasados em recursos que proporcionem aprendizagens diferenciadas dentro da área de conhecimento de cada disciplina.

5. Perfil do egresso

Como define a Seção 3 deste PPC, o objetivo do BES é formar pessoas com *competência* em Engenharia de Software. Segundo [SWECOM 2014], uma pessoa competente tem a capacidade necessária para realizar, em um determinado nível de proficiência, as atividades de trabalho a ela atribuídas.

O perfil profissional do egresso do BES é definido pelo conjunto de competências desenvolvidas durante o curso, articuladas com as necessidades e demandas da sociedade por tecnologias baseadas em software e com o mundo do trabalho em Engenharia de Software, nos níveis local, regional, nacional e global. A competência adquirida no BES deve propiciar ao egresso uma formação sólida em Engenharia de Software, baseada em fundamentos de Computação e de Matemática, formando cidadãos capazes de trabalhar em um contexto de atuação abrangente, que decorre da crescente necessidade do uso de tecnologias baseadas em software. Esse trabalho deve ser pautado em postura profissional e conduta ética que integram as competências esperadas do egresso do BES.

O foco em competências tem o propósito de evitar combinações de três perfis indesejados para um profissional da Engenharia de Software:

> (a) perfil teórico, que conhece conteúdos e teorias, mas não é capaz de aplicá-los, como por exemplo, a pessoa que conhece linguagens e regras de programação, mas não consegue programar;

(b) perfil prático, que consegue realizar tarefas, sem saber explicar conteúdos e teorias envolvidas, como a pessoa capaz de programar, mas que desconhece regras ou normas de programação; e

(c) perfil antiético, que conhece conteúdos e teorias, consegue colocá-las em prática, porém age em contradição com o código de ética e postura profissional da Engenharia de Software.

O NDE do BES deve realizar o acompanhamento dos egressos, avaliando a sua inserção no mundo do trabalho e a adequação do perfil do egresso definido neste PPC às necessidades e demandas para o profissional de Engenharia de Software. Esse acompanhamento deve ser feito por meio de pesquisa sistematizada, conduzida anualmente e contemplando uma amostragem significativa do conjunto de egressos do curso.

O perfil esperado do egresso do BES combina domínio de teorias e conteúdos (conhecimentos) com capacidades de aplicação prática (habilidades) e postura ética e profissional (atitudes) no exercício de atividades de Engenharia de Software. Os conhecimentos que devem ser adquiridos e as atitudes que devem ser demonstradas pelo egresso são definidos, respectivamente, pelas ementas das disciplinas que compõem o currículo do BES (descrito na Seção 6) e pelos princípios do código de ética e postura profissional da Engenharia de Software (descritos na Seção 4.1). As competências esperadas do egresso são classificadas em três categorias [SWECOM 2014]: cognitivas, comportamentais e técnicas.

5.1. Competências cognitivas do egresso

Capacidades cognitivas não são exclusivas da Engenharia de Software, mas se aplicam à aquisição e à aplicação do conhecimento necessário em todas as atividades esperadas de um profissional desta área. O perfil do egresso do BES contempla as seguintes competências cognitivas [SWECOM 2014]:

- Raciocínio: base para tomada de decisão usando critérios lógicos. Há diversos tipos de raciocínio (tais como indutivo, dedutivo, heurístico, abstrato, hierárquico e associativo) e todos são aplicáveis em atividades de Engenharia de Software.

- Análise: técnicas para coletar, organizar, agrupar e avaliar dados para embasar conclusões ou para tomada de decisão. Essas técnicas incluem medições estatísticas e análise causal, de riscos e de impacto.

- Resolução de problemas: métodos que empregam raciocínio, análise e priorização de dados para solucionar problemas. Exemplos aplicados à Engenharia de Software incluem dividir e conquistar, refinamento progressivo, abordagens top-down e bottom-up, reúso e reconhecimento de padrões, e abordagens iterativas e incrementais.

- Inovação: técnicas para criação de modelos e abstrações para facilitar a análise e a resolução de problemas. Exemplos típicos incluem *brainstorming*, prototipagem, modelagem e simulação.

Essas competências, que não são independentes, mas sobrepostas e mutuamente complementares, tornam-se cada vez mais importantes à medida que se ampliam o escopo e a complexidade dos trabalhos pertinentes à Engenharia de Software, alavancados pelas demandas cada vez maiores da sociedade por tecnologias baseadas em software.

5.2. Competências comportamentais do egresso

Capacidades comportamentais, assim como as cognitivas, também não são exclusivas da Engenharia de Software e também se aplicam em todas as atividades esperadas de um profissional desta área. No entanto, enquanto competências cognitivas focam em aspectos racionais visando aquisição e uso de conhecimento, competências comportamentais dizem respeito a fatores emocionais e subjetivos do profissional. Nessa perspectiva, o perfil do egresso do BES contempla as seguintes competências comportamentais [SWECOM 2014]:

- Aptidão: executar tarefas da área de Engenharia de Software, aplicando efetivamente o conhecimento; agir para melhorar continuamente sua própria competência nesta área.

- Iniciativa: agir espontaneamente, com ânimo e disposição natural, para iniciar e continuar tarefas de Engenharia de Software.

- Entusiasmo: enfrentar, sem abatimento, dificuldades e desafios de Engenharia de Software, atuando com interesse e dedicação nas tarefas e transmitindo confiança à equipe.

- Ética no trabalho: ser confiável, adquirir e melhorar competências, ter disposição para cumprir as tarefas e agir de acordo com o código de ética e postura profissional da Engenharia de Software.

- Boa vontade: assumir tarefas sempre que solicitado e desempenhá-las com efetividade, mesmo que não seja um trabalho de sua preferência ou em sua área de especialização.

- Confiabilidade: demonstrar, ao longo do tempo, comportamento ético, honestidade e integridade em ações e decisões sobre o trabalho.

- Sensibilidade cultural: perceber e aceitar diferenças étnicas, sociais e culturais, respeitando orientações e valores próprios de cada pessoa.

- Comunicação: escutar ou ler as ideias dos demais membros da equipe, compreendendo e valorizando essas ideias, e expressar suas próprias ideias com clareza e concisão, na forma oral ou por escrito.

- Espírito de equipe: trabalhar de forma harmoniosa e colaborativa com outros, compartilhando tarefas com boa vontade e entusiasmo.

- Liderança: estimular, motivar, influenciar, inspirar e orientar pessoas em uma equipe a fim de atingir objetivos comuns.

As competências comportamentais refletem características individuais e estão fortemente correlacionadas. Elas são denominadas de diferentes formas, como *soft skills* ou habilidades interpessoais, e são difíceis de avaliar porque envolvem questões subjetivas. Todavia, é cada vez maior a importância deste tipo de competência, principalmente para lidar com os desafios do trabalho em equipe, já que atividades relevantes de Engenharia de Software envolvem, necessariamente, a colaboração entre pessoas com diferentes personalidades e capacidades complementares. O profissional de Engenharia de Software precisa colaborar em equipe, de forma profissional e ética, para executar processos e elaborar produtos de software de qualidade.

5.3. Competências técnicas do egresso

Competências técnicas, ao contrário das comportamentais e cognitivas, são específicas do profissional de Engenharia de Software e se relacionam diretamente com a capacidade de aplicar conhecimentos técnicos para realizar processos de Engenharia de Software. Na perspectiva técnica, o egresso do BES deve demonstrar competências em [SWEBOK 2023]:

- Requisitos de software:
  - Planejar e executar a engenharia de requisitos para um sistema (conceitos operacionais; partes interessadas; ambientes e cenários operacionais de hardware, software e manuais; dependências e restrições tecnológicas; requisitos e sua alocação a elementos do sistema; rastreabilidade entre requisitos do sistema e do software);
  - Planejar e executar a engenharia de requisitos para o software de interesse (partes interessadas, infraestrutura, comunicação, padrões, métodos, responsabilidades e autoridades);
  - Eliciar, analisar, especificar, verificar, validar e gerenciar requisitos do produto de software (gerência de configuração de requisitos, rastreabilidade e dependências entre requisitos, priorização de requisitos e negociação de conflitos);
  - Identificar e tratar os tipos de requisitos de software (funcionais, informacionais, desempenho, manutenibilidade, segurança, usabilidade, compatibilidade, confiabilidade e portabilidade).
- Arquitetura e design de software:
  - Aplicar fundamentos de design (abstração, coesão, acoplamento e ocultação de informação) e métodos e estratégias de design (*top-down, bottom-up*, iterativo e incremental, refinamentos sucessivos, padrões de design, paradigmas orientados a objetos e a eventos) para definição de componentes;
  - Projetar tratamento de eventos e de exceções, persistência de dados, tolerância a falhas, concorrência e distribuição de processamento em componentes do software.
  - Criar e modificar arquitetura de software (estilos, visões e padrões arquiteturais); decisões de projeto (construir, comprar, adaptar);

    - Analisar e avaliar a qualidade do design de produto de software (revisões, análise estática, simulações e protótipos).
- Implementação e integração de software:
    - Planejar e gerenciar a construção de software (selecionar padrões de codificação, linguagens de programação, ferramentas e frameworks, gerência de configuração, medições e avaliações de qualidade);
    - Codificar software (design detalhado, programação, codificação defensiva, geração de código, refatoração, documentação);
    - Depurar código fonte (testes de unidade, cobertura de testes, técnicas e ferramentas de depuração);
    - Integrar software (gerência de dependências, estratégias e testes para integração, implantação e entrega de código; automação e integração contínua).
- Teste de software:
    - Planejar e executar testes para software e para sistema (objetivos de teste, partes interessadas, critérios de sucesso e falha, critérios de término, casos de teste, demonstração de resultados de testes);
    - Identificar e tratar os tipos de teste (funcional, estrutural, integração, aceitação, desempenho, estresse, regressão, entre outros) necessários em um software;
    - Definir, disponibilizar e manter infraestrutura para teste de software (ambiente de teste, dados, ferramentas, geração e arquivamento de documentos e resultados de testes);
    - Medir testes e rastrear defeitos de software (coleta e análise de dados de testes, relato de resultados de testes, análise de cobertura e efetividade dos testes, identificação e monitoramento de ações corretivas relacionadas aos testes).
- Sustentação e descontinuação de software:
    - Planejar e efetuar a transição do software para ambiente operacional (restrições operacionais, instalação, treinamento de usuários e mantenedores, procedimentos de ativação e desativação, aceitação do sistema e do software);
    - Planejar e prover suporte técnico à operação do software (detecção e troca de componentes obsoletos, diagnóstico e tratamento de

incidentes e anomalias, coleta e monitoramento de dados operacionais, preparação para descontinuação do software);
    - Planejar e realizar manutenção de software (criação de *baselines*, tratamento de pedido de mudança, identificação e classificação de problemas de software, análise de impacto de mudanças, realização de mudanças em software).
- Ciclo de vida de software e processos de Engenharia de Software:
    - Planejar e realizar o ciclo de vida de desenvolvimento de software (escolha e instanciação do modelo de ciclo de vida);
    - Definir e adaptar processos de Engenharia de Software (considerando contexto, autoridades e responsabilidades);
    - Executar e gerenciar processos de Engenharia de Software (monitoramento e controle, com orientação para a equipe);
    - Avaliar e melhorar processos de Engenharia de Software (coleta e avaliação de dados de desempenho dos processos, estratégias e modelos para melhoria de processos).
- Verificação e validação da qualidade de software:
    - Gerenciar a qualidade de software (cultura da qualidade, políticas, objetivos e atributos da qualidade, partes interessadas, modelos, padrões e procedimentos para qualidade, ferramentas, métricas critérios e métodos de avaliação da qualidade, planejamento e monitoramento da qualidade, recursos e infraestrutura para avaliação da qualidade, melhoria contínua e controle estatístico de qualidade de processos e produtos de software);
    - Garantir a qualidade de software (análise estática de software, verificação, validação, inspeção e revisão por pares, auditoria independente, coleta, análise e comunicação de dados da qualidade, identificação, priorização e tratamento de incidentes relacionados à qualidade do software).
- Segurança e proteção de software:
    - Proteger contra uso malicioso (identificação de riscos e de requisitos de proteção contra uso indevido, análise para detecção de vulnerabilidades em software, modelagem de ameaças e de superfície de ataque ao software, padrões de ataque, coleta e monitoramento de métricas de proteção, padrões de projeto e de código para proteção de software; segurança da informação);

    - Garantir segurança no uso legítimo do software (tipos de riscos de segurança – riscos ao usuário, ao ambiente ou à propriedade – relacionados ao software e ao sistema, definição de requisitos de segurança para o uso do software, priorização e tratamento de riscos de segurança, análise estática para detecção de riscos de segurança em software, testes de segurança operacional de software e de sistema, padrões de segurança de software, coleta e monitoramento de métricas de segurança, padrões de projeto e de código para segurança operacional de software).
- Gerência de configuração de software:
    - Planejar gerência de configuração (seleção de componentes para controle, organização de repositórios de dados e código, definição de estratégias e procedimentos ramificação e integração de versões, definição de processo para controle de mudanças, infraestrutura e ferramentas para gestão de configuração);
    - Realizar gerência de configuração (identificação de itens de configuração, controle de versionamento, gestão de incidentes e problemas, relato de situação de itens de configuração, gestão de *baselines*, auditoria física e funcional de configuração);
    - Gerenciar liberação e entrega (planejamento de entregas, uso de ferramentas de entrega, geração de versão para entrega, *patches* para correção de versão liberada).
- Medição de software:
    - Planejar medição de produtos e processos (identificação de necessidades e objetivos de medição e de produtos e processos a medir, definição de indicadores, escalas e métricas, identificação de partes interessadas, definição de métodos para coleta e análise de dados, de metas e limites para indicadores e de formatos para relato de indicadores, seleção de ferramentas para medição, planejamento de armazenamento e acesso a dados de medição);
    - Realizar medição de produtos e processos (coleta e validação de dados, armazenamento e análise de dados, geração e distribuição de relatórios de medição).
- Interação Humano-Computador (IHC) via software:
    - Definir requisitos de interação (identificação de partes interessadas e usuários, definição de processo e ferramentas para desenvolver

IHC, especificação e prototipagem de requisitos de usabilidade, identificação de elementos do sistema relacionados à IHC e de restrições e padrões aplicáveis ao projeto de IHC);

- Projetar estilo de interação e estilo visual (definição de modelos e metáforas de IHC, identificação de cenários e modos de interação, definição de diálogos e fluxo de interação, tratamento de erros na IHC, rastreabilidade entre requisitos e testes de usabilidade, projeto de interface entre IHC e outros elementos do sistema, definição de leiaute de página/tela, seleção e projeto de ícones, menus, temas de cores e fontes de caracteres);
- Avaliar usabilidade (inspeção, teste, avaliação heurística e por especialistas, identificação de sujeitos representativos dos grupos de usuários, projeto de testes de usabilidade e relato de resultados);
- Ter acessibilidade como foco (identificação de necessidades especiais de IHC – daltonismo, controle motor fino, limitação de audição ou visão – e de usuários com tais necessidades, questões de gênero, idade e culturais, uso de ferramentas e técnicas para prover acessibilidade, teste de acessibilidade).

6. Estrutura curricular

A estrutura curricular do BES é organizada em nove períodos letivos, cada um com duração de um semestre letivo. O Quadro 1 resume a distribuição da carga horária entre os tipos de componente da estrutura curricular do BES.

Quadro 1. Resumo da distribuição da carga horária do curso.

Tipo de componente curricular (disciplina)	**Quantidade**	**Carga Horária (CH)**	**Percentual**
Núcleo Comum (NC) Obrigatório	17	1152	36%
Núcleo Específico (NE) Obrigatório	21	1600	50%
Núcleo Específico (NE) Optativo	2	128	4%
Núcleo Livre (NL)	livre	128	4%
Atividades Complementares (AC)	livre	192	6%
CARGA HORÁRIA TOTAL (CHT)		**3200**	**100%**
Atividades Curriculares da Extensão (**ACEx**)	1	320*****	10%
***** CH de **ACEx** está totalmente incluída na CH da disciplina Prática em Engenharia de Software do NE			

As disciplinas de NC são de natureza obrigatória e constituem o conjunto de conteúdos básicos para a formação profissional do discente. Elas fornecem uma base sólida em Ciência da Computação e em Matemática.

As disciplinas de NE são próprias da área de Engenharia de Software e dão especificidade à formação profissional do egresso do BES. Algumas delas são de natureza obrigatória, outras são optativas. O discente deve cursar pelo menos duas das disciplinas definidas como optativas, escolhidas pelo próprio discente do conjunto de disciplinas optativas definido na Seção 6.2.

As disciplinas de NL ampliam e diversificam a formação discente, promovem interdisciplinaridade e transdisciplinaridade, possibilitam o aprofundamento em áreas de interesse do discente e viabilizam o intercâmbio entre discentes de diferentes cursos da UFG. O discente deve cursar pelo menos 128 horas de NL, escolhidas pelo próprio discente do conjunto de disciplinas oferecidas como NL a cada semestre letivo pela UFG. Apenas a carga horária mínima é definida para componentes de NL, podendo ser cumprida, por exemplo, com duas disciplinas de NL com 64 horas ou quatro disciplinas de 32 horas.

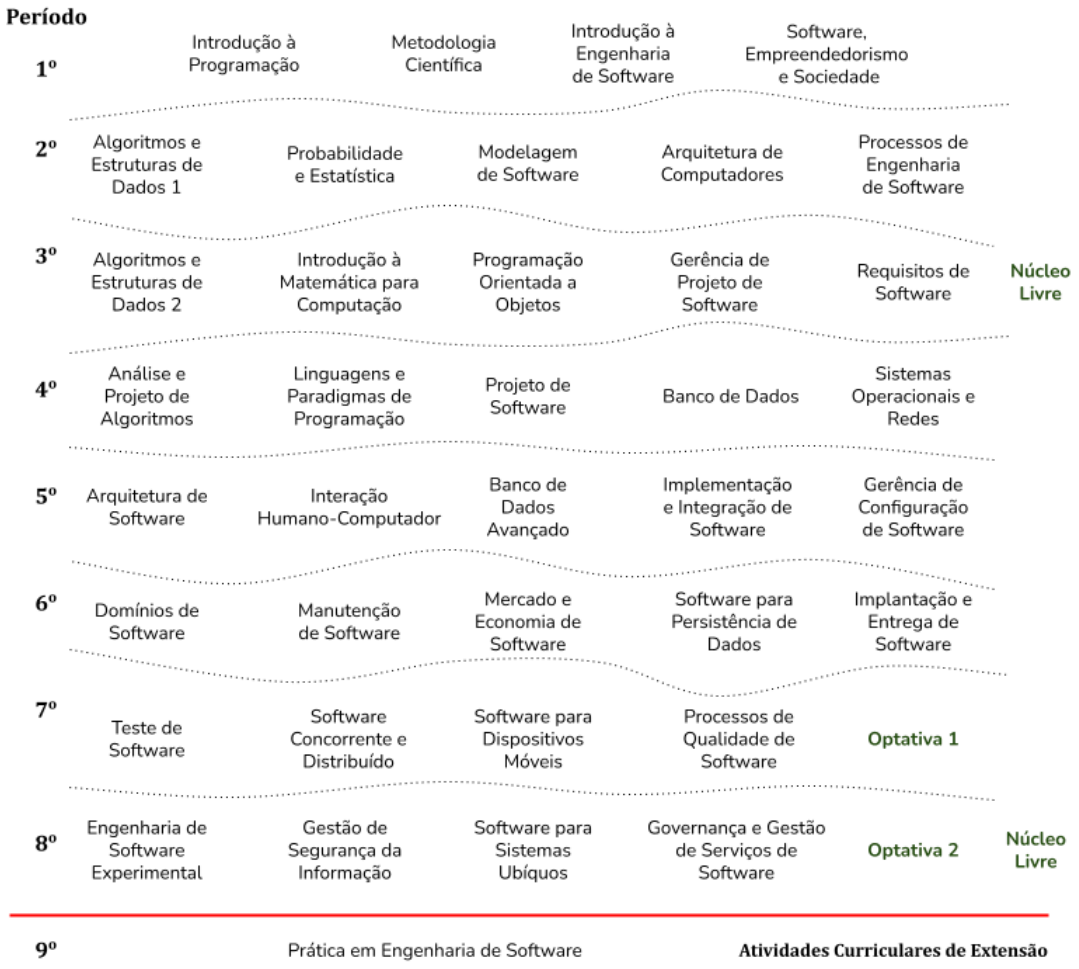
O componente de ACEx, de caráter obrigatório, insere o discente com papel ativo na execução de ações de extensão realizadas em projetos que visam atender demandas da sociedade relacionadas a software.

A estrutura curricular do curso inclui, no mínimo, 192 horas de Atividades Complementares (AC), que são atividades acadêmicas livremente escolhidas pelo discente ao longo do curso, excetuando-se componentes curriculares. Essas atividades abrangem participação em monitoria, tutoria, projeto (P&D, ensino, extensão e cultura), estágio curricular não obrigatório, evento (conferência, seminário, palestra, congresso), dentre outras atividades. Critérios para aproveitamento de atividades complementares são definidos em regulamento específico vigente.

6.1. Matriz curricular e fluxo sugerido

A Figura 1 sintetiza o fluxo sugerido para o curso, incluindo a indicação dos períodos em que os componentes curriculares optativos podem ser cursados.

Figura 1. Fluxo curricular sugerido para o BES.



A matriz curricular do curso é apresentada no Quadro 2, na ordem do fluxo curricular sugerido pelo curso, e descreve, para cada disciplina:

- A Unidade Acadêmica (UA) responsável pela oferta da disciplina.
- A carga horária da disciplina, contemplando a carga total (CH ToT, teórica (CH Teo), prática (CH Pra) e curricular de extensão (CH ACEx).
- Os pré-requisitos (Pré-req), se houver, a natureza da disciplina, obrigatória (OBR) ou optativa (OPT), e o tipo de Núcleo da disciplina, núcleo comum (NC), núcleo específico (NE) ou núcleo livre (NL).

Quadro 2. Matriz curricular na forma de fluxo sugerido.

1º PERÍODO									
ID	**Disciplina**	**UA**	**CH Teo**	**CH Pra**	**CH ACEx**	**CH ToT**	**Pré-req**	**Natureza**	**Núcleo**
1	Introdução à Engenharia de Software	INF	64	0	0	64		OBR	NC
2	Metodologia Científica	INF	32	32	0	64		OBR	NC
3	Introdução à Programação	INF	48	80	0	128		OBR	NC
4	Software, Empreendedorismo e Sociedade	INF	64	0	0	64		OBR	NE
	Carga horária acumuladada	**320**	**Carga horária no período**			**320**			
2º PERÍODO									
ID	**Disciplina**	**UA**	**CH Teo**	**CH Pra**	**CH ACEx**	**CH ToT**	**Pré-req**	**Natureza**	**Núcleo**
5	Algoritmos e Estruturas de Dados 1	INF	32	32	0	64	3	OBR	NC
6	Arquitetura de Computadores	INF	48	16	0	64		OBR	NC
7	Probabilidade e Estatística	IME	64	0	0	64		OBR	NC
8	Processos de Engenharia de Software	INF	32	32	0	64		OBR	NE
9	Modelagem de Software	INF	16	48	0	64		OBR	NE
	Carga horária acumuladada	**640**	**Carga horária no período**			**320**			
3º PERÍODO									
ID	**Disciplina**	**UA**	**CH Teo**	**CH Pra**	**CH ACEx**	**CH ToT**	**Pré-req**	**Natureza**	**Núcleo**
10	Algoritmos e Estruturas de Dados 2	INF	64	0	0	64	5	OBR	NC
11	Gerência de Projeto de Software	INF	32	32	0	64		OBR	NE
12	Programação Orientada a Objetos	INF	32	32	0	64	3	OBR	NC
13	Introdução à Matemática para Computação	INF	64	0	0	64		OBR	NC
14	Requisitos de Software	INF	32	32	0	64		OBR	NE
	Núcleo Livre					64			NL
	Carga horária acumuladada	**1024**	**Carga horária no período**			**384**			
4º PERÍODO									
ID	**Disciplina**	**UA**	**CH Teo**	**CH Pra**	**CH ACEx**	**CH ToT**	**Pré-req**	**Natureza**	**Núcleo**
15	Banco de Dados	INF	48	16	0	64		OBR	NC
16	Sistemas Operacionais e Redes	INF	48	16	0	64		OBR	NC
17	Linguagens e Paradigmas de Programação	INF	32	32	0	64	3	OBR	NC
18	Projeto de Software	INF	64	0	0	64	12	OBR	NC
19	Análise e Projeto de Algoritmos	INF	64	0	0	64	10,13	OBR	NC
	Carga horária acumuladada	**1344**	**Carga horária no período**			**320**			
5º PERÍODO									
ID	**Disciplina**	**UA**	**CH Teo**	**CH Pra**	**CH ACEx**	**CH ToT**	**Pré-req**	**Natureza**	**Núcleo**
20	Arquitetura de Software	INF	32	32	0	64		OBR	NE
21	Banco de Dados Avançado	INF	48	16	0	64	15	OBR	NC
22	Gerência de Configuração de Software	INF	32	32	0	64		OBR	NE
23	Implementação e Integração de Software	INF	32	32	0	64	18	OBR	NE
24	Interação Humano-Computador	INF	32	32	0	64		OBR	NC
	Carga horária acumuladada	**1664**	**Carga horária no período**			**320**			

6º PERÍODO									
ID	**Disciplina**	**UA**	**CH Teo**	**CH Pra**	**CH ACEx**	**CH ToT**	**Pré-req**	**Natureza**	**Núcleo**
25	Domínios de Software	INF	16	48	0	64		OBR	NE
26	Manutenção de Software	INF	32	32	0	64		OBR	NE
27	Mercado e Economia de Software	INF	64	0	0	64		OBR	NE
28	Software para Persistência de Dados	INF	32	32	0	64	15	OBR	NE
29	Implantação e Entrega de Software	INF	32	32	0	64	23	OBR	NE
Carga horária acumuladada		**1984**	**Carga horária no período**			**320**			
7º PERÍODO									
ID	**Disciplina**	**UA**	**CH Teo**	**CH Pra**	**CH ACEx**	**CH ToT**	**Pré-req**	**Natureza**	**Núcleo**
30	Teste de Software	INF	32	32	0	64		OBR	NE
31	Processos de Qualidade de Software	INF	48	16	0	64		OBR	NE
32	Software Concorrente e Distribuído	INF	32	32	0	64	23	OBR	NE
33	Software para Dispositivos Móveis	INF	32	32	0	64	18	OBR	NE
	OPTATIVA 1					64		OPT	NE
Carga horária acumuladada		**2304**	**Carga horária no período**			**320**			
8º PERÍODO									
ID	**Disciplina**	**UA**	**CH Teo**	**CH Pra**	**CH ACEx**	**CH ToT**	**Pré-req**	**Natureza**	**Núcleo**
34	Engenharia de Software Experimental	INF	16	48	0	64		OBR	NE
35	Gestão de Segurança da Informação	INF	32	32	0	64		OBR	NC
36	Governança e Gestão de Serviços de Software	INF	48	16	0	64	8	OBR	NE
37	Software para Sistemas Ubíquos	INF	32	32	0	64	23	OBR	NE
	OPTATIVA 2					64		OPT	NE
	Núcleo Livre					64			NL
Carga horária acumuladada		**2688**	**Carga horária no período**			**384**			
9º PERÍODO									
ID	**Disciplina**	**UA**	**CH Teo**	**CH Pra**	**CH ACEx**	**CH ToT**	**Pré-req**	**Natureza**	**Núcleo**
38	Prática em Engenharia de Software	INF	0	320	320	320		OBR	NE
Carga horária acumuladada		**3008**	**Carga horária no período**			**320**			
Carga Horária Acumulada (3008h) + Atividades Complementares (192h) = Carga Horária Total do Curso (3200h)									

6.2. Componentes curriculares optativos

O Quadro 3 especifica as disciplinas que compõem o conjunto de disciplinas optativas do BES.

Quadro 3. Disciplinas optativas do BES.

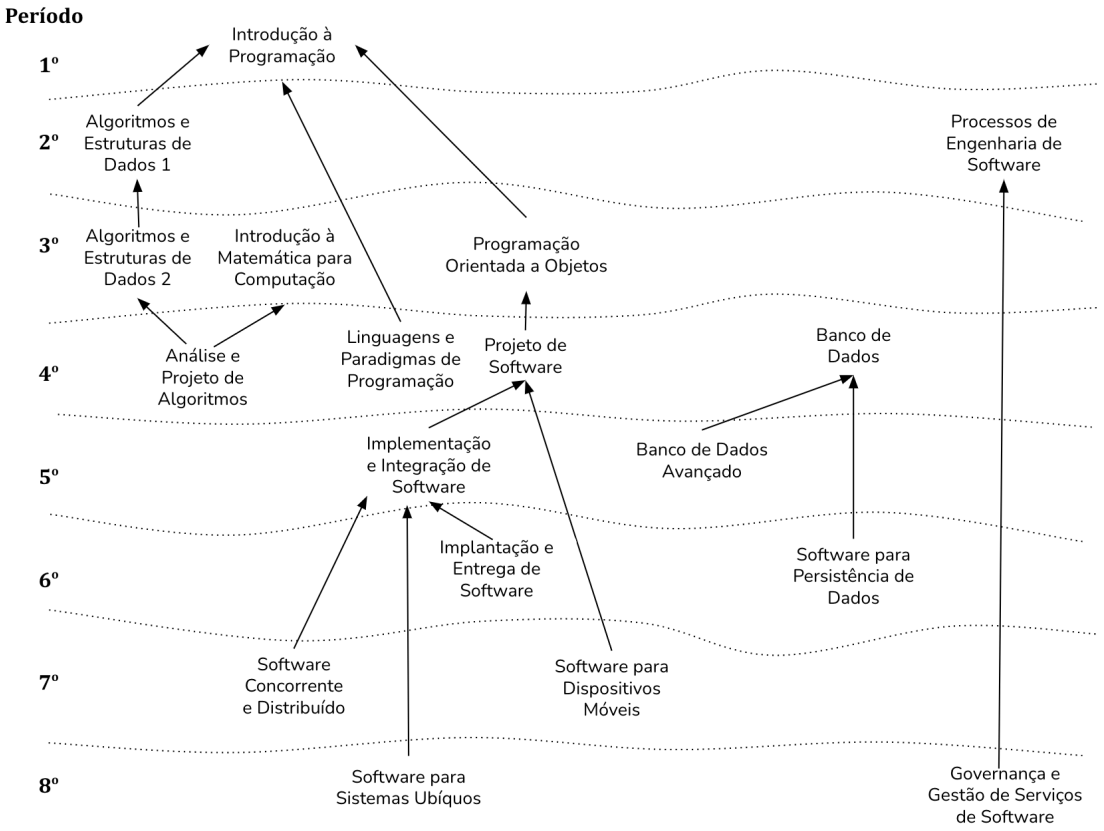
DISCIPLINAS OPTATIVAS								
ID	**Disciplina**	**UA**	**CH Teo**	**CH Pra**	**CH ACEx**	**CH ToT**	**Pré- req**	**Natureza**
39	Compiladores	INF	48	16	0	64	47	OPT
40	Desenvolvimento Full Stack	INF	32	32	0	64	49	OPT
41	Empreendedorismo Digital	INF	64	0	0	64		OPT
42	Engenharia de Sistemas de Informação 1	INF	48	16	0	64	45	OPT
43	Gestão de Sistemas e de TI	INF	64	0	0	64		OPT
44	Introdução à Língua Brasileira de Sinais[1]	FL	64	0	0	64		OPT
45	Introdução aos Sistemas de Informação	INF	64	0	0	64		OPT
46	Jogos Digitais	INF	16	48	0	64	12	OPT
47	Linguagens Formais e Autômatos	INF	64	0	0	64	13	OPT
48	Matemática Discreta	INF	64	0	0	64	13	OPT
49	Sistemas Distribuídos	INF	32	32	0	64	12,16	OPT
50	Sistemas Inteligentes de Apoio à Decisão	INF	32	32	0	64	45	OPT
51	Software Básico	INF	0	64	0	64	6	OPT
52	Tópicos em Engenharia de Software[2]	INF	32	32	0	64		OPT

6.3. Pré-requisitos das disciplinas obrigatórias e optativas

Os pré-requisitos das disciplinas obrigatórias são definidos na Seção 6.1 e estão sintetizados na Figura 2, onde as disciplinas aparecem alinhadas na horizontal, de acordo com o período em que devem ser cursadas no fluxo sugerido para o curso.

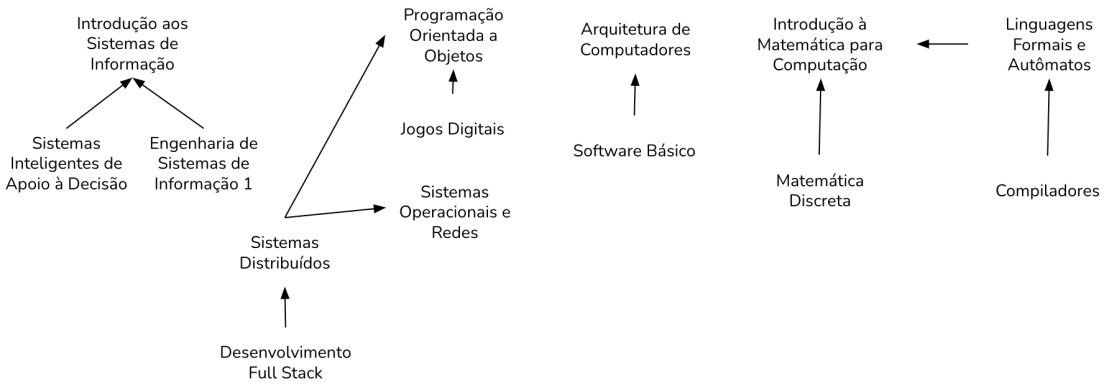
[1] CCU - Componente Curricular Unificado
[2] Disciplina de tema variado.

Figura 2. Pré-requisitos das disciplinas obrigatórias.



Os pré-requisitos das disciplinas optativas são definidos na Seção 6.2 e estão sintetizados na Figura 3. Algumas disciplinas optativas dependem de disciplinas obrigatórias. Por exemplo, “Jogos Digitais” tem como pré-requisito a disciplina obrigatória “Programação Orientada a Objetos”.

Figura 3. Pré-requisitos das disciplinas optativas.



6.4. Ementas das disciplinas com bibliografias básica e complementar

6.4.1. Disciplinas de formação básica (obrigatórias do núcleo comum)

Algoritmos e Estruturas de Dados 1

Ementa

Noções de complexidade de algoritmos (notações de complexidade). Recursividade. Algoritmos de pesquisa: pesquisa sequencial e binária. Algoritmos de ordenação. Tipos abstratos de dados. Estruturas de dados utilizando vetores: pilhas, filas, listas (simples e circulares). Estruturas de dados com alocação dinâmica de memória: pilhas, filas, listas (simplesmente encadeadas, duplamente encadeadas e circulares).

Bibliografia Básica

1. FEOFILOFF, P. Algoritmos em Linguagem C. Editora Campus/Elsevier, 2009.
2. SZWARCFITER, J. L.; MARKENZON, L. Estruturas de Dados e seus Algoritmos. 4ª edição, LTC,2010.
3. TENENBAUM, A. M.; LANGSAM, Y.; AUGENSTEIN, M. Estruturas de Dados Usando C, São Paulo, Makron Books, 2007.

Bibliografia Complementar

1. CORMEN, T. H. et al., Algoritmos: Teoria e Prática. 2.ª edição, Rio de Janeiro: Editora Campus, 2012.
2. SALVETTI, D. D.; BARBOSA, L. M. Algoritmos. Makron Books, São Paulo, 1998.
3. SEDGEWICK, R. Algorithms in C++ (Parts 1-4), Addison-Wesley, 3rd Edition, 1998.
4. ZIVIANI, N. Projeto de Algoritmos com implementação em Pascal e C. São Paulo: Editora Thomson, 3.ª edição, 2010.
5. ZIVIANI, N. Projeto de Algoritmos com implementação em Java e C++. São Paulo: Editora Thomson, 2006.

Algoritmos e Estruturas de Dados 2

Ementa

Árvores: formas de representação, recursão em árvores, árvores binárias, árvores binárias de busca, árvores balanceadas (AVL e rubro-negras). Filas de prioridades. *Heaps*, *Heapsort*. *Hashing*: tipos de funções de *hashing*; tratamento de colisões. Definições de Grafos. Estruturas de Dados para representação de grafos. Algoritmos básicos em grafos.

Bibliografia Básica

1. FEOFILOFF, P. Algoritmos em Linguagem C. Editora Campus/Elsevier, 2009.
2. SZWARCFITER, J. L.; Markenzon, L. Estruturas de Dados e seus Algoritmos. 4ª edição, LTC, 2010.
3. TENENBAUM, A. M.; LANGSAM, Y.; AUGENSTEIN, M. Estruturas de Dados Usando C, São Paulo, Makron Books, 2007.

Bibliografia Complementar

1. CORMEN, T. H. et al., Algoritmos: Teoria e Prática. 5ª edição, Rio de Janeiro: Editora Campus, 2012.
2. SALVETTI, D. D. e BARBOSA, L. M., Algoritmos, Makron Books, São Paulo, 1998.
3. SEDGEWICK, R. Algorithms in C++ (Parts 1-4), 3rd edition, Addison-Wesley, 1998.
4. ZIVIANI, N. Projeto de Algoritmos com implementação em Pascal e C. 3ª edição, São Paulo: Editora Thomson, 2010.
5. ZIVIANI, N. Projeto de Algoritmos com implementação em Java e C++. São Paulo: Editora Thomson, 2006.

Análise e Projeto de Algoritmos

Ementa

Medidas de complexidade. Análise assintótica de limites de complexidade para algoritmos iterativos e recursivos, técnicas de prova de cotas inferiores. Corretude de Algoritmos. Exemplos de análise de algoritmos. Técnicas de projeto de algoritmos: dividir para conquistar, programação dinâmica, algoritmos gulosos. Introdução à NP-Completude.

Bibliografia Básica

1. BRASSARD, G.; BRATLEY, P. Fundamentals of Algorithmics. Prentice-Hall, Inc., Upper Saddle River, NJ, 1996. ISBN: 0-13-335068-1.
2. CORMEN, T. H.; LEISERSON, C. E.;RIVEST, R. L.; STEIN, C. Algoritmos: Teoria e Prática, 3.a edição, Campus, 2012.
3. PAPADIMITRIOU, C. H.; VAZIRANI, U. V.; DASGUPTA, S. Algoritmos. 2009. McGraw-Hill Brasil. ISBN 9788577260324.

Bibliografia Complementar

1. AHO, A. V.; HOPCROFT, J. E.; ULLMAN, J.D. The Design and Analysis of Computer Algorithms, Addison Wesley Publishing Company, 1974. ISBN 0-201-00029-6.
2. BAASE, S.; GELDER, A. V. Computer Algorithms: Introduction to Design and Analysis, 3rd Edition, Pearson, 1999.
3. MAMBER, U. Introduction to Algorithms. Addison Wesley Publishing Company. 1989.
4. SEDGEWICK, R.; WAYNE, K. Algorithms. 4th edition, Addison-Wesley Professional, 2011. ISBN: 978- 0321573513.
5. SZWARCFITER, J. L.; MARKENZON, L. Estrutura de Dados e seus Algoritmos. 3.a edição, LTC Editora, 2010. ISBN 978852161750.

Arquitetura de Computadores

Ementa

Fundamentos de arquitetura de computadores: visão geral de computadores modernos; evolução da arquitetura dos computadores. Memória e representação de dados e instruções. Processador, ciclo de instrução, formato e endereçamento. Noções básicas de programação e processamento de linguagem de montagem. Dispositivos de entrada e saída. Sistemas de interconexão (barramentos), interfaceamento e técnicas de entrada e saída. Hierarquia de memória. Fundamentos de arquiteturas paralelas e métricas de desempenho.

Bibliografia Básica

1. BRYANT, R.; O'RALLARON, D. Computer Systems: A Programmer 's Perspective, 2nd Edition, Addison Wesley, 2010.

2. STALLINGS, W. Arquitetura e Organização de Computadores, 9a. Edição, Prentice-Hall, 2010.
3. TANENBAUM, A. Organização Estruturada de Computadores, Editora LTC, 2007.

Bibliografia Complementar

1. HENNESSY, J. L.; PATTERSON, D. A. Computer Architecture: A Quantitative Approach, 4th Edition, Morgan Kaufmann, 2007.
2. MONTEIRO, M. A. Introdução à Organização de Computadores. 4ª. Edição. Editora LTC, Rio de Janeiro, 2001.
3. PATTERSON, D. A.; HENNESSY, J. L. Computer Organization and Design: The Hardware/Software Interface, 3rd edition, Morgan Kaufmann, 2007.
4. STALLINGS, W. Computer Organization and Architecture: Designing for Performance. 10th Edition. Prentice-Hall, Inc., Upper Saddle River, NJ, USA. 2010.
5. WEBER, R. F. Fundamentos de Arquiteturas de Computadores, 2.ª Edição, Editora Sagra-Luzzatto, 2001.

Banco de Dados

Ementa

Fundamentos de bancos de dados: componentes de sistemas de bancos de dados; dados e metadados, esquemas e bases de dados. Modelo de dados conceitual: modelo de dados de Entidade-Relacionamento (ER) e suas extensões; prática de modelagem de dados. Modelo de dados operacional: modelo de dados relacional e restrições de integridade de dados; dependências funcionais e formas normais; prática de normalização de esquema relacional. Mapeamento de esquema conceitual ER para esquema operacional relacional. Consulta a bancos de dados relacionais: noções de álgebra e cálculo relacional. Linguagem SQL: definição (SQL-DDL) e manipulação (SQL-DML) de dados; prática de definição e manipulação de esquemas em SQL.

Bibliografia Básica

1. ELMASRI, R.; NAVATHE, S. B. Sistemas de Banco de Dados, 6.ª ed., Pearson - Addison Wesley, 2011.

2. HEUSER, C. A. Projeto de Banco de Dados, 6.ª edição, Bookman, Porto Alegre, 2009.
3. SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. Sistema de Banco de Dados, 5.ª edição, Editora Campus, Rio de Janeiro, 2006.

Bibliografia Complementar

1. BEAULIEU, A. Learning SQL. 2. ed. Beijing: O'Reilly, 2010.
2. GARCIA-MOLINA, H.; ULLMAN, J. D.; WIDOM, J. D. Database Systems: The Complete Book, 2.ª edição, Prentice Hall, 2009.
3. RAMAKRISHNAN, R.; GEHRKE, J. Sistemas de Gerenciamento de Banco de Dados, tradução da 3.ª edição, São Paulo, McGraw-Hill, 2008.
4. TEOREY, T.; LIGHTSTONE, S.; NADEAU, T. Projeto e Modelagem de Bancos de Dados, Editora Campus, Rio de Janeiro, 2007.

Banco de Dados Avançado

Ementa

Mapeamento objeto-relacional (ORM): uso de bancos de dados relacionais em linguagens de programação orientadas a objetos. Funções, gatilhos, procedimentos armazenados, SQL embutida e consultas parametrizadas. Arquitetura de referência para Sistemas Gerenciadores de Bancos de Dados (SGBDs): processamento e otimização de consulta; indexação e ajuste (*tunning*) de bancos de dados relacionais. Processamento de transações em SGBDs: propriedades ACID, controle de concorrência e recuperação de falhas. SGBDs não-relacionais (No-SQL): arquitetura e desempenho de sistemas orientados a documentos, orientados a colunas, baseados em pares chave-valor e baseados em grafos; definição, carga, consulta e atualização de dados. Segurança de sistemas de bancos de dados: autorizações e papéis, segurança de aplicações (SQL injection), auditoria. Mineração de dados, *data warehouse* e processamento de dados massivos.

Bibliografia Básica

1. ELMASRI, R.; NAVATHE, S. B. Sistemas de Banco de Dados, 6.ª ed., Pearson - Addison Wesley, 2011.
2. GARCIA-MOLINA, H.; ULLMAN, J. D.; WIDOM, J. D. Database Systems: The Complete Book, 2.ª edição, Prentice Hall, 2009.

3. KEITH, M.; SCHINCARIOL, M. Pro JPA 2: mastering the Java Persistence API. Apress: Springer-Verlag, 2009.

Bibliografia Complementar

1. CONNOLLY, T. M.; BEGG, C. E.; STRACHAN, A. D. Database systems: a practical approach to design, implementation and management, 3rd. Edition, Addison Wesley, 2010.
2. MANNINO, M. V. Projeto, Desenvolvimento de Aplicações e Administração de Banco de Dados, 3ª ed, SP, McGraw-Hill, 2008.
3. RAMAKRISHNAN, R.; GEHRKE, J. Sistemas de Gerenciamento de Banco de Dados, 3.ª edição, São Paulo, McGraw-Hill, 2008.

Gestão de Segurança da Informação

Ementa

Fundamentos de Sistema de Gestão de Segurança da Informação (SGSI): políticas de segurança; autenticação, autorização, auditoria. Princípios de segurança da informação: confidencialidade, integridade e disponibilidade; classificação de informações visando a segurança. Tipificação de ameaças e ataques em sistemas de informação; avaliação e monitoramento de riscos à segurança. Segurança operacional no contexto de segurança da informação: riscos associados a pessoas, ética e engenharia social; segurança física. Vulnerabilidades em tecnologias: softwares, serviços e protocolos; prevenção e detecção de intrusão e de ataque. Tópicos emergentes e tendências em segurança da informação.

Bibliografia básica

1. IMONIANA, JOSHUA O. Auditoria de Sistemas de Informação. 2. ed., SP: Atlas, 2008.
2. SCHMIDT, P. Fundamentos de Auditoria de Sistemas. São Paulo: Atlas, 2006.
3. STALLINGS, W. Criptografia e segurança de redes: princípios e práticas. 4. ed., Prentice Hal, 2008.

Bibliografia complementar

1. KIZZA, J. Computer Network Security. New York: Springer, 2005.
2. KUROSE, J. F.; ROSS, K. Redes de Computadores e a Internet. 6. ed. Pearson Education, 2013.
3. LYRA, M. R. Segurança e Auditoria em Sistemas de Informação. 1. ed., Ciência Moderna, 2009.
4. TCU. Manual de Auditoria Operacional do TCU. Disponível em https://portal.tcu.gov.br/data/files/F2/73/02/68/7335671023455957E18818A8/Manual_auditoria_operacional_4_edicao.pdf, 2020.
5. TIPTO, H. F. Information Security Management Handbook, Auebach Publications, 2004.

Interação Humano-Computador

Ementa

Conceitos de interação humano-computador: IHC, interface, interação, *affordance*; critérios de qualidade em IHC (usabilidade, experiência do usuário, acessibilidade, comunicabilidade); experiência de usuário de software (qualidade em uso); semiótica. Design centrado no usuário: PACT (Pessoas, Atividades, Contexto, Tecnologias); princípios e diretrizes gerais do design de interação; identificação das necessidades dos usuários (levantamento dos requisitos); organização do espaço problema (personas, cenários e tarefas). Projeto da interação com o usuário: criação, modelagem e especificação de interface de usuário; ações perceptíveis em objetos de interação (*affordance*); ferramentas para prototipação e construção de interfaces. Design de software interativo: padrões arquiteturais e padrões de projeto para a camada de interação do software (MVC, MVP, MVVM e outros); integração entre componentes da camada de interação e demais componentes de código do software. Construção de software interativo: implementação e integração de componentes de código para a camada de interação de um software. Avaliação de interfaces: métodos de avaliação por inspeção (avaliação heurística, percurso cognitivo, inspeção semiótica); métodos de avaliação por observação (teste de usabilidade, avaliação de comunicabilidade, prototipação em papel); métodos e técnicas para validação da experiência do usuário (qualidade em uso).

Bibliografia básica

1. BARBOSA, S.D.J.; SILVA, B.S.; SILVEIRA, M.S.; GASPARINI, I.; DARIN, T.; BARBOSA, G.D.J. (2021) Interação Humano-Computador e Experiência do Usuário. Disponível em http://leanpub.com/ihc-ux.
2. GOTHELF, J.; SEIDEN, J. Lean UX: Applying Lean Principles to Improve User Experience, O'Reilly, 2013.
3. KRUG, S., Don´t make me think: revisited. New Riders, 2014.
4. ROGERS, Y.; SHARP, H.; PREECE, J. Design de Interação - Além da Interação Homem-computador. 3a Ed., Porto Alegre: Bookman, 2013.

Bibliografia complementar

1. BORBA, F. E. Ajax: guia de programação. São Paulo: Erica, 2006.
2. COUTAZ, J.; BASS, L. Developing software for the user interface. Addison-Wesley, 1991.
3. CYBIS, W. O. Ergonomia e usabilidade: conhecimentos, métodos e aplicações. 2a. edição. São Paulo: Novatec, 2010.
4. HARTSON, H. R.; HIX, D. Developing user interfaces: ensuring usability through product & process. Wiley, 1993.
5. OLSEN, D. R. Developing user interfaces. Morgan Kaufmann, 1998.
6. PREECE, J.; SHARP, H.; ROGERS, Y. Interaction Design: Beyond Human-Computer Interaction, Wiley, 2015.

Introdução à Engenharia de Software

Ementa

Fundamentos de Engenharia de Software: conceito de software; ciclo de vida de software e ciclo de vida de desenvolvimento de software. Especificação de software: Requisitos de software; Arquitetura de software; Projeto (design) de software; Modelos e métodos de engenharia de software. Implementação e validação de software: Construção de software; Teste de software. Sustentação de software: Operações de engenharia de software; Manutenção de software; Gerência de configuração de software. Gestão de software: Gerência de engenharia de software; Processo de engenharia de software; Qualidade de software; Segurança de software; Prática profissional de engenharia de software; Economia da engenharia de software. Disciplinas correlatas à Engenharia de Software: Fundamentos de Computação, Matemática e Engenharia.

Bibliografia Básica

1. PRESSMAN, R.; MAXIM, B. Software Engineering: A Practitioner's Approach, 8th edition, McGraw-Hill, 2014.
2. SOMMERVILLE, I. Software Engineering, 10th Edition, Pearson, 2016.
3. SWEBOK - Guide to the Software Engineering Body of Knowledge, V.4, IEEE Computer Society, 2023. Disponível em https://www.computer.org/web/swebok.

Bibliografia Complementar

1. ISO/IEC/IEEE 24765. Systems and software engineering - Vocabulary, 2010. Portal de periódicos da CAPES (http://goo.gl/FrGm7c).
2. MPS.BR. Melhoria de Processos do Software Brasileiro. Disponível em http://www.softex.br/mpsbr/, último acesso julho/2016.
3. Software Engineering Competency Model (SWECOM), IEEE. Disponível em https://www.computer.org/web/peb/swecom.
4. WAZLAWICK, R. S. Engenharia de software: teoria e prática. Campus, 2013.

Introdução à Matemática para Computação

Ementa

Noções básicas de lógica, lógica proposicional, lógica de primeira ordem. Linguagem experimental. Introdução a técnicas básicas de demonstrações. Conjuntos e relações.Relações de equivalência. Relações de ordens parciais. Séries e sequências. Indução matemática. Ordem de grandeza e notação assintótica.

Bibliografia Básica

1. GERSTING, J. L. Fundamentos Matemáticos para a Ciência da Computação, 5a. edição, Editora LTC, 2004.
2. HUTH, MICHAEL; RYAN, MARK. Lógica em Ciência da Computação: modelagem e argumentação sobre sistemas. Segunda edição. Editora LTC, 2008.
3. SCHEINERMAN, EDWARD R., Matemática Discreta, Uma introdução, Thomson Pioneira, 2003.
4. SOUZA, JOÃO NUNES DE. Lógica para Ciência da Computação. Editora Campus, 3a. Edição, 2015.

Bibliografia Complementar

1. ALBERTSON, M. O., HUTCHINSON, J. P., Discrete Mathematics with Algorithms, John Wiley & Sons, 1988.
2. GONÇALVES, A., Introdução à Álgebra, Coleção Projeto Euclides, 5ª ed., IMPA, Rio de Janeiro, 2013.
3. GRIMALDI, R. P., Discrete and Combinatorial Mathematics - An Applied Introduction, 5th Edition, Pearson, 2003.
4. MATTSON, H. F., Discrete Mathematics with applications, John Wiley & Sons, 1993.
5. SANTOS, J. P.. Introdução à Teoria dos Números, Coleção Matemática Universitária, IMPA, Rio de Janeiro, 1998.
6. SILVA, F. C.; FINGER, M.; DE MELO, A. C. V. Lógica para Computação. Thomson Learning, 2006.
7. ROSS, K. A., WRIGHT C. R. B., Discrete mathematics, Prentice-Hall, 1998.
8. ROSEN, K., Matemática Discreta e suas Aplicações. 6ª ed., McGraw-Hill Science, 2009.

Introdução à Programação

Ementa

Fundamentos de algoritmos: conceito e formas de representação; identificação de problemas com solução algorítmica. Conceitos de programas: constantes; tipos de dados primitivos; variáveis; atribuição; entrada e saída de dados; expressões; estruturas de decisão; estruturas de repetição; ponteiros. Estruturas de dados homogêneas e heterogêneas: vetores, matrizes, cadeias de caracteres, registros. Subprograma: função; passagem de parâmetros por valor e referência. Manipulação de arquivos: abertura, fechamento, leitura e gravação; arquivos texto e binário e tipos de acesso (sequencial e indexado). Transcrição de algoritmos para linguagem de programação. Domínio de uma linguagem de programação: sintaxe e semântica; interpretação e compilação; técnicas de depuração; uso de bibliotecas.

Bibliografia básica

1. ASCENCIO, A. F. G.; CAMPOS, E. A. V. Fundamentos da Programação de Computadores. 3.ª edição. Editora Pearson, 2012.
2. FOBERLONE, A. L. V.; EBERSPACHER, H. F. Lógica de Programação: A construção de algoritmos e estruturas de dados. 3.ª edição, São Paulo, Prentice Hall, 2005.
3. SCHILDT, H. C Completo e Total. 3.ª Ed, São Paulo, Makron, 1996.

Bibliografia complementar

1. CORMEN, T. H et al., Algoritmos: Teoria e Prática. 2.ª edição. Rio de Janeiro: Editora Campus, 2002.
2. FARRER, H. at al. Programação Estruturada de Computadores: Algoritmos Estruturados. 3.ª edição. Rio de Janeiro: LTC, 1989.
3. FEOFILOFF, P. Algoritmos em Linguagem C. Editora Campus/Elsevier, 2009.
4. SALVETTI, D.; BARBOSA, L. M. Algoritmos, São Paulo: Makron, 1998.
5. SEDGEWICK, R. Algorithms in C. 3rd ed., Reading, Mss, Addison-Wesley, 1998.

Linguagens e Paradigmas de Programação

Ementa

Conceitos de linguagens de programação: estrutura léxica e sintática; contexto e semântica; interpretação e compilação; ambientes de programação e de execução. Paradigmas de programação imperativo, declarativo, procedural, funcional, lógico, orientado a objeto. Características de linguagens de programação e critérios de seleção de linguagens de acordo com os domínios de aplicação. Descrição de sintaxe e semântica de linguagens de programação. Tipos de dados: primitivos, compostos, ponteiros e referências; verificação de tipos e escopos. Expressões e atribuições: tipos de expressões, operadores, conversões de tipos. Estruturas de controle: seleção, iteração e desvio; variáveis e expressões de controle; subprogramas; passagem de parâmetros; tratamento de exceções.

Bibliografia Básica

1. GHEZZI, C.; JAZYERI, M. Programming Language Concepts, 3rd Edition, Wiley; 3rd Edition; 1998.
2. SCOTT, M. L. S. Programming Language Pragmatics, 3rd Edition, Morgan Kaufmann, 2009.
3. SEBESTA, R. W. Concepts of Programming Languages; 10th Edition, Pearson, 2012.

Bibliografia Complementar

1. FRIEDMAN, D. P.; WAND, M. Essentials of Programming Languages, 2rd Edition, The MIT Press, 2001.
2. PRATT, T. W.; ZELKOWITZ, M. V. Programming Languages: Design and Implementation, 4th Edition, Prentice Hall, 2000.
3. ROY, P. V.; HARIFI, S. Concepts, Techniques, and Models of Computer Programming, 1st Edition, The MIT Press, 2004.
4. TURBAK, F.; GIFFORD, D. Design Concepts in Programming Languages, 1st Edition, The MIT Press, 2008.
5. VAREJÃO, F. Linguagens de Programação, 1ª Edição, Campus, 2004.

Metodologia Científica

Ementa

Fundamentos de metodologia científica: ciência, verdade e pesquisa científica; senso comum e conhecimento científico. Métodos de pesquisa: o método científico; pesquisa qualitativa, quantitativa e bibliográfica; estilos de pesquisa em Computação. Elaboração de pesquisa: questões de pesquisa, objetivo, metodologia e planejamento de pesquisa; coleta, organização e análise de dados; descrição de resultados e conclusões. Leitura de texto científico: postura crítica do leitor; técnicas de leitura; vocabulário padrão da Engenharia de Software. Revisão da literatura: estudos primários, secundários e terciários; literatura *cinza*; mapeamento e revisão sistemática. Organização e escrita de texto científico: normas para formatação e padronização de textos; apresentação oral de trabalhos acadêmicos.Ética na ciência: pesquisa envolvendo humanos ou animais; validação de resultados; vieses e limitações de resultados; plágio.

Bibliografia Básica

1. ANDRADE, M. M. Introdução à metodologia do trabalho científico: elaboração de trabalhos na graduação. 9. ed., SP, Atlas, 2009.
2. GIL, A. C. Como Elaborar Projetos de Pesquisa. 5. ed., SP, Atlas, 2010.
3. MARCONI, M. A.; LAKATOS, E. M. Fundamentos de Metodologia Científica. 7. ed., SP, Atlas, 2010.
4. ISO/IEC/IEEE 24765-2017. International Standard - Systems and software engineering - Vocabulary.

Bibliografia Complementar

1. BARROS, A. J. S.; LEHFELD, N. A. S. Fundamentos da Metodologia Científica. 3 ed., São Paulo: Pearson Prentice Hall, 2007.
2. CERVO, A. L. Metodologia Científica: Para uso dos estudantes universitários. 3. ed., São Paulo: Mc Graw Hill, 1983.
3. PARBERRY, I. How to Present a Paper in Theoretical Computer Science: Speaker's Guide for Students. ACM SIGACT News. 4(2), 1993.
4. WAZLAWICK, R. S. Metodologia de Pesquisa em Ciência da Computação. Rio de Janeiro: Elsevier, 2009. 6a reimpressão.

Probabilidade e Estatística

Ementa

Estatística descritiva. Noções sobre amostragem. Noções de probabilidade: introdução à teoria de conjuntos, espaço amostral, eventos, frequência relativa, fundamentos de probabilidade, probabilidade condicional e eventos independentes. Conceitos gerais de variáveis aleatórias. Distribuições discretas de probabilidade: Uniforme e Binomial. Distribuições contínuas de probabilidade: Uniforme, Normal e t-Student. Estimação pontual e intervalar para uma população: média e proporção. Teste de hipóteses para uma população: média e proporção. Correlação linear e regressão linear simples.

Bibliografia Básica

1. BUSSAB, WILTON O. & MORETTIN, PEDRO A. Estatística Básica. 6ª Ed. São Paulo: Saraiva, 2010.
2. CRESPO, A. A.: Estatística Fácil. Saraiva, São Paulo, Brasil, 1998.
3. TRIOLA, M. F. Introdução à Estatística. 10. ed. Rio de Janeiro: LTC, 2008.

Bibliografia Complementar

1. HINES, W. W.: Probabilidade e Estatística para Engenharia. 4ª Ed. Rio de Janeiro: LTC, 2006.
2. LARSON, R.; FARBER, B.: Estatística Aplicada. 2a ed., Pearson Prentice Hall, São Paulo, Brasil, 2004.
3. LEVIN, JACK: Estatística Aplicada A Ciências Humanas. . Harbra, São Paulo, 2000.
4. MONTGOMERY: Estatística Aplicada à Engenharia. 2ª Ed. São Paulo: LTC, 2004.
5. MOORE, D. S.: A Estatística Básica e sua Prática. LTC, Rio de Janeiro, Brasil, 2005.
6. WALPOLE, R. E.; MYERS, R. H. M. S. L. Y. K. Probabilidade e Estatística para engenharia e ciências. 8. ed. São Paulo: Pearson, 2009.

Programação Orientada a Objetos

Ementa

Paradigma de orientação a objetos: classes, objetos, métodos, interfaces, mensagens, encapsulamento, herança, polimorfismo, associações e composição; abstração e tipos abstratos de dados. Conceitos da linguagem Java: identificadores, palavras chave, literais, operadores, separadores e comentários; tipos de dados primitivos e classes; conversão de tipo e classes *wrapper*. Instanciação de objetos: inicialização de objetos; método construtor e operador new; destruição de objetos; uso de pacotes; declaração, importação e visibilidade de classes. Operações básicas: vetores, expressões, operadores e estruturas de controle e de seleção; declaração de atributos e métodos; sobrecarga de métodos; modificadores de acesso; tratamento de exceções. Herança em Java: derivação, sobrescrita, adição de método e atributo, classes finais; polimorfismo; classes, interfaces e métodos abstratos; operador *instanceof*. Noções de modelagem orientada a objetos com Unified Modeling Language (UML) e de padrões de projeto orientado a objetos.

Bibliografia Básica

1. WU, C. T. An introduction to object-oriented programming with Java. 5th ed. McGraw Hill Higher Education, 2000.

2. DEITEL, P. J.; DEITEL, H. M. Java: como programar. Tradução: Furmankiewicz, E. 8ª edição. Prentice Hall, 2005.
3. BORATTI, I. C. Programação orientada a objetos em Java. 1ª edição. Visual Books, 2007.

Bibliografia Complementar

1. SEDGEWICK, R; WAYNE, K. Introduction to programming in Java: an interdisciplinary approach. Pearson Addison-Wesley, 2008.
2. HORSTMANN, C. S. Core Java – Volume II – Advanced Features, Prentice Hall, 6ª Edição, 2004.
3. GOODRICH, M. T.; TAMASSIA, R. Estruturas de dados e algoritmos em Java. Tradução: Copstein, B; Oliveira, J. B. 4ª ed. Bookman, 2007.
4. GUEDES, G. UML 2 : uma abordagem prática. Novatec, 2009.

Projeto de Software

Ementa

Fundamentos de *design* de software: concorrência, controle e tratamento de eventos, persistência de dados, distribuição, tratamento de erro e exceção, tolerância a falhas, interação e apresentação, e segurança. Design detalhado e arquitetural de software: comportamento e estrutura do software. Projeto de interface de usuário. Notação para *design* de software. Métodos, estratégias e padrões de *design* de software. Análise de qualidade e avaliação de *design* de software. Ferramentas para *design* de software.

Bibliografia Básica

1. BASS, L. et al. Software Architecture in Practice, 1rd edition, AddisonWesley, 1998.
2. BUDGEN, B. Software Design, 2nd edition, Addison Wesley, 1993.
3. LARMAN, C. Utilizando UML e padrões: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento interativo. Bookman, 2007.

Bibliografia Complementar

1. BOOCH, G. UML guia do usuário: o mais avançado tutorial sobre Unified Modeling Language (UML). Elsevier, 2006.

2. FOWLER, M. Patterns of enterprise application architecture. AddisonWesley, 2003.
3. FREEMAN, E. et al. Use a cabeça! padrões de projetos, (design patterns), Alta Books, 2007.
4. GAMMA, E. et al. Design patterns elements of reusable object-oriented software. Reading: Addison Wesley, 1995.
5. NYGARD, M. Release It! Design and Deploy Production Ready Software. Pragmatic Bookshelf, 2007.

Sistemas Operacionais e Redes

Ementa

Conceitos básicos de sistemas operacionais. Processos e threads. Gerência de memória. Programação com threads e paralelismo. Plataformas de virtualização. Arquitetura de Redes TCP/IP (Internet). Camada de transporte. Camada de aplicação. Programação para redes.

Bibliografia básica

1. LAUDON, K. C.; LAUDON, J. P. Sistemas de Informação Gerenciais. 9.ª edição. São Paulo: Pearson Education, 2013.
2. KUROSE, J.F.; ROSS, K. Redes de Computadores e a Internet, 6.ª edição. Pearson Education, 2013.
3. TANENBAUM, A. S. Computer Networks, 4th ed., Prentice Hall, 2003.
4. OLIVEIRA, R. S.; CARISSIMI, A. S.; TOSCANI, S. S. Sistemas Operacionais. 3.ª edição, Porto Alegre, Editora Sagra Luzzatto, 2004.
5. SILBERSCHATZ, A. Fundamentos de Sistemas Operacionais, 9.ª edição, LTC, 2015.
6. TANEMBAUM, A. S. Sistemas Operacionais Modernos. 3.ª ed. Prentice Hall, 2009.

Bibliografia complementar

1. DANTAS, M. Redes de comunicação e computadores: abordagem quantitativa. Visual Books, 2009.
2. PETERSON, L. L.; DAVIE, B. S. Redes de Computadores: Uma Abordagem de Sistemas, 3.ª edição. Campus Elsevier, 2004.

3. STALLINGS, W. Data and Computer Communications, 8th edition, Pearson/Prentice Hall, 2007.
4. TORRES, G. Redes de computadores: curso completo. Axcel Books, 2001.
5. DEITEL, H. Sistemas Operacionais, Prentice Hall, 2005.
6. MACHADO, F. Arquitetura de Sistemas Operacionais. 2a ed. LTC, 1997.
7. NEMETH E.; SNYDER, G.; HEIN, T. R. Unix system administration handbook. Pearson Prentice Hall, 1997.
8. O'GORMAN, J. Operating Systems with Linux. Palgrave, 2001.

6.4.2. Disciplinas obrigatórias do núcleo específico

Além de ementa e bibliografia (básica e complementar), a descrição das disciplinas obrigatórias do núcleo específico de Engenharia de Software inclui dois novos elementos: a distribuição da carga horária nos tópicos da ementa e as “competências mínimas” para aprovação na disciplina.

A carga horária sugerida em cada tópico da ementa visa a garantir consistência e coerência do plano da disciplina com o PPC como um todo. Esta sugestão da carga horária não é arbitrária, sendo definida de forma criteriosa para atender o perfil definido para o egresso do curso. O esforço planejado para cada tópico da ementa busca reduzir variações na qualidade do processo de ensino-aprendizagem decorrentes de opiniões e preferências do docente que ministra a disciplina.

As competências mínimas, conforme discute a Seção 4.3, definem as capacidades que o discente deve demonstrar para ter aproveitamento na disciplina. As competências mínimas da disciplina são definidas por um conjunto de atividades, cada uma com um determinado nível de proficiência. As atividades definem o que o estudante deve estar apto a realizar com o conteúdo da disciplina e o nível de proficiência define a maturidade com que deve realizar a atividade. São considerados dois níveis de proficiência:

- **Faz com orientação**. O discente é capaz de realizar a atividade, com assistência do docente, mas sem necessidade de instruções detalhadas. Neste nível, o discente conhece e compreende a atividade, mas depende de orientação para realizá-la com qualidade profissional.
- **Faz sem orientação**. O discente é capaz de realizar a atividade sem assistência do docente e sem instruções detalhadas sobre a atividade. Neste nível, o discente conhece e compreende a atividade e consegue realizá-la com qualidade profissional, sem orientação.

As competências mínimas não representam condições desejadas ou ideais de capacidade pois, de fato, espera-se que o discente possa realizar atividades além daquelas definidas nas competências mínimas e em maiores níveis de proficiência. Contudo, ao realizar todas as competências mínimas nos níveis especificados, o discente está apto à aprovação na disciplina em questão.

Arquitetura de Software

Ementa

Conceitos de arquitetura de software (4h): acoplamento, coesão, ocultação de informação, modularização, reutilização; representação de arquitetura (visões, pontos de vista, diagramas). Descrição de arquitetura de software (8h): normas, notações e modelos para documentação de arquitetura de software; mapeamento de requisitos (funcionais e não funcionais) para arquitetura de software. Estilos e padrões arquiteturais (12h): arquitetura de referência; camadas, *broker*, transações, troca de mensagens, *pipe-and-filter*, notificação de eventos (*publish-subscribe*), cliente-servidor, *blackboard*, MVC (e variantes); exemplos de arquiteturas para domínios específicos (aplicação web, aplicativo móvel, compilador, SGBD, SO, IDE, *browser*). Padrões arquiteturais para integração de aplicações (12h): transferências de arquivos, bases de dados compartilhadas, chamadas de procedimento remoto e trocas de mensagens. Avaliação de arquitetura de software (8h): critérios de qualidade de arquitetura de software; verificação e validação de arquitetura de software; engenharia reversa e reengenharia (refatoração) de arquitetura..
Prática em arquitetura de software (20h): desenvolvimento de arquitetura de software (componentes, interfaces (internas e externas), dados e fluxos de

dados; visões estáticas (estruturais) e dinâmicas (comportamentais) do software); integração de arquiteturas de softwares.

Competências mínimas:

- (Faz sem orientação). Compreender a documentação de arquitetura de software.
- (Faz com orientação). Documentar arquitetura de software.
- (Faz com orientação). Analisar arquitetura de software.
- (Faz com orientação). Criar arquitetura de software que se beneficia de padrões de arquitetura.
- (Faz com orientação). Integrar aplicações por meio de arquitetura de software.

Bibliografia básica

1. BASS, L.; CLEMENTS, P.; KAZMAN, R. Software Architecture in Practice, 3rd edition, Pearson Education, Inc., 2013.
2. CLEMENTS, P.; et al. Documenting Software Architectures: Views and Beyond. 2 nd edition, Pearson Education, 2011.
3. ISO/IEC/IEEE 42010:2011 - Systems and software engineering - Architecture description. International Organization for Standardization. 2011-11-24.
4. MENDES, A. Arquitetura de software: desenvolvimento orientado para arquitetura. Rio de Janeiro: Campus, 2022.

Bibliografia complementar

1. BOSCH, J. Design and use of software architectures: adopting and evolving a product-line approach. AddisonWesley, 2000.
2. CERVANTES, H.; KAZMAN, R. Designing Software Architectures: A Practical Approach, Addison-Wesley, 2016.
3. HEINEMAN, G. T.; COUNCILL, W. T. Component-based software engineering: putting the pieces together. Addison-Wesley, 2001.
4. JOHNSON, R. Expert one-on-one J2EE development without EJB. Wiley Publishing, 2004.
5. PUTMAN, J. Architecting with RM-ODP. Prentice Hall, 2001.

Domínios de Software

Ementa

Conceitos de domínios de software (4h): domínio (ou tipo) de aplicação (saúde, finanças, comércio eletrônico, entretenimento e jogos, controle de sistemas, redes sociais e gestão de empresas, entre outros); domínio (ou tipo) de software (por exemplo, aplicação web, aplicativo móvel, compilador, SGBD, SO, IDE e *browser*). Requisitos de software para um domínio específico (12h): características e particularidades de um domínio de software e de aplicação. Arquitetura e projeto de software para um domínio específico (12h): modelos e padrões de arquitetura e projeto de software aplicáveis a domínios específicos. Construção de software para um domínio específico (32h): modelos e padrões de implementação e integração de software aplicáveis a domínios específicos. Validação de software para um domínio específico (4h): avaliação de atendimento de requisitos e especificidades de um domínio de software.

Competências mínimas:

- (Faz com orientação). Explicar a construção de software, a relação dessa área de conhecimento com as demais da engenharia de software e a relação com o conceito de sistema.
- (Faz com orientação). Usar e configurar ferramentas para edição, documentação, compilação, depuração (debugging), build, teste, controle de versão, coleta de medidas (desempenho, consumo de memória, análise estática e cobertura), integração contínua, e aplicativos fundamentais de linha de comandos.
- (Faz com orientação). Criar código orientado a objetos em conformidade com o projeto (design) detalhado seguindo estratégias recomendadas.
- (Faz com orientação). Usar processos para a construção de software (criação de código, controle de versão, inspeção e integração).
- (Faz com orientação). Detalhar projeto (design) em conformidade com requisitos de software.
- (Faz com orientação). Colaborar com a construção de código em equipe.

- (Faz com orientação). Explicar domínios de desenvolvimento (categorias) e as especificidades correspondentes.
- (Faz com orientação). Explicar um domínio de software e as estratégias de desenvolvimento correspondentes.

Bibliografia básica

1. BOOCH, G. Object-oriented analysis and design with applications. 2nd edition. Addison-Wesley, 1994.
2. MCLAUGHLIN, B. D.; POLLICE, G.; WEST, D. Head First Object-Oriented Analysis & Design, O'Reilly, 2006.
3. PILONE, D.; MILES, R.; Head-First Software Development, O'Reilly, 2008.

Bibliografia complementar

1. BEEDLE, M.; SCHWABER, K. Agile software development with Scrum. Prentice Hall, 2002. (Series in agile software development).
2. COCKBURN, A. Agile software development Boston: Addison-Wesley, 2002. (Agile software development series).
3. ODELL, J. J. Advanced object-oriented analysis and design using UML. Cambridge University Press, SIGS Books, 1998.
4. PENKER, M.; ERIKSSON, H. E. Business modeling with UML: business patterns at work. John Wiley & Sons, 2000.
5. WAZLAWICK, R. S. Análise e design orientados a objetos para sistemas de informação: Modelagem com UML, OCL e IFML. 3.ª edição. Campus, 2010.

Engenharia de Software Experimental

Ementa

Conceitos de Engenharia de Software Experimental (4h): metodologia científica aplicada à Engenharia de Software; pesquisa qualitativa e quantitativa; teorias, leis e conjecturas científicas; visão geral de métodos para aquisição de conhecimento e validação de hipóteses científicas; ameaças à validade da pesquisa. Revisão de literatura (4h): tipos de estudos (primários, secundários, terciários); revisão e mapeamento sistemático da literatura; literatura cinzenta e revisão sistemática *multivocal*; tipos de ameaças à

validade. Prática de revisão literária (20h): realização de pesquisa bibliográfica em Engenharia de Software. Estudo de caso e pesquisa-ação (4h): similaridades e diferenças entre estudo de caso e pesquisa-ação; planejamento da pesquisa; seleção de cenários e de participantes; condução da pesquisa; formas de coleta e análise de dados; observação de impactos e consequências; reflexão e conclusões;tipos de ameaças à validade. Pesquisa de campo (*survey*) e teoria fundamentada (*grounded theory*) (4h): similaridades e diferenças entre *survey* e *grounded theory*; definição de escopo; elaboração e refinamento de questões de pesquisa; seleção de audiência (público alvo); condução da pesquisa; formas de coleta de dados (busca na literatura, *survey*, grupo focal, etnografia); codificação e categorização de dados; busca por padrões e proposta de teorias; tipos de ameaças à validade. Experimento controlado e simulação (4h): grupo experimental e grupo de controle; variáveis dependentes e independentes; seleção de participantes; experimento randomizado e *quasi-experimento*; empacotamento de experimento; tipos de ameaças à validade. Prática de experimentação científica (24h): realização de experimentos em Engenharia de Software.

Competências mínimas:

- (Faz com orientação). Aplicar princípios científicos da Engenharia de Software Experimental na realização de experimentos com software.
- (Faz com orientação). Realizar estudo secundário sistemático em Engenharia de Software.

Bibliografia básica

1. GETTINBY, G; GARDINER, W. P. Experimental design techniques in statistical practice: a practical software-based approach. Horwood Pub., 1998.
2. JURISTO, N. Basics of software engineering experimentation. Kluwer Academic Publishers, 2001.
3. WOHLIN, C. et al. Experimentation in Software Engineering. 2nd edition. Springer, 2012.

Bibliografia complementar

1. HALL, E. M. Managing risk methods for software systems development. Addison-Wesley, 1998.

2. POLYAK, B. T. Introduction to optimization. Optimization Software, 1987.
3. QUALIDADE no setor de software brasileiro. Brasília, D.F.: MCT, 1997.
4. WAZLAWICK, R. S. Metodologia de Pesquisa para Ciência da Computação. Editora Campus, 2009.
5. WOHLIN, C. Experimentation in software engineering: an introduction. Kluwer Academic, 2000.

Gerência de Configuração de Software

Ementa

Conceitos de gerência de configuração (4h): sistema de gerência de configuração (GC); funções de GC e sua relação com processos de desenvolvimento e manutenção de software. Identificação e armazenamento de configuração (8h): identificação de item de configuração (IC); atributos de um IC; organização de repositório de ICs; controle de acesso e de concorrência a ICs em equipes centralizadas e distribuídas. Controle de mudanças em IC (8h): solicitação de mudança; relatórios de status; controle de versões de IC; linhas base (*baselines*) de configuração de software. Controle de construção (*build*) (8h): controle de dependências entre ICs; fluxo de ramificação e junção (*branch/merge*) de configurações; auditoria de configuração; transição de configuração para outro ambiente. GC no ciclo de vida do software (8h): normas e modelos para GC (IEEE, MPS.BR); GC em metodologias ágeis (*builds* frequentes, desenvolvimento iterativo e incremental); GC adequada ao tipo de software (sistema de sistemas, multiplataformas, família de produtos e variantes, sistemas críticos). Prática com ferramentas de GC (28h): identificação da configuração, controle da situação da configuração, avaliação e revisão da configuração, gerenciamento de liberação e entrega.

Competências mínimas:

- (Faz com orientação). Planejar um sistema de gerência de configuração para software.
- (Faz com orientação). Definir fluxo de ramificação/derivação e de junção (merge) de código.

- (Faz com orientação). Planejar liberação (release) e implantação (deploy) de software.
- (Faz sem orientação). Realizar operações essenciais de GC em ferramenta de GC.

Bibliografia básica

1. Configuration Management Principles and Practice, The Agile Software Development Series, Anne Mette, Jonassen Hass, Pearson Education, 2003.
2. IEEE 828-2012 - IEEE Standard for Configuration Management in Systems and Software Engineering.

Bibliografia complementar

1. SWEBOK - Guide to the Software Engineering Body of Knowledge V.4, Chapter 8 - Software Configuration Management. IEEE Computer Society, 2023. Disponível em http://swebok.org.
2. Software Configuration Management Patterns: Effective Teamwork, Practical Integration, Stephen P. Berczuk et al., Addison-Wesley, 2003.
3. The Build Master: Microsoft's Software Configuration Management Best Practices, Vincent Maraya, AddisonWesley, 2005.

Gerência de Projeto de Software

Ementa

Conceitos de gerência de projeto (4h): ciclo de vida de produto, serviço e projeto; partes interessadas (*stakeholders*); correlação entre projetos, operações e programas; gerência de portfólio de projetos; escritório de gestão de projetos (PMO). Tipos de projeto de engenharia de software (4h): aquisição, desenvolvimento, fornecimento, implantação, migração, sustentação, manutenção (refatoração, correção, incremento), descontinuação; características e especificidades de cada tipo de projeto. Tipos de ciclo de vida de projeto de software (4h): preditivo, iterativo, adaptativo; processos empíricos e processos definidos e sua relação com métodos ágeis e métodos planejados para gerência de projeto. Qualidade de processo de gerência de projeto de software (4h): modelos de referência para gerência de projetos.

Gerência de projetos de engenharia de software (24h): escopo, tempo, custo, qualidade, recursos humanos. Gerência de projetos aplicada a projetos de software (24h): comunicações, riscos, aquisições, integração e partes interessadas.

Competências mínimas:

- (Faz com orientação). Usar normas, modelos e ferramentas de gerência de projeto para planejamento e acompanhamento de projeto de software.
- (Faz com orientação). Selecionar e implementar o tipo de modelo de processo (dirigido por plano, incremental ou adaptativo) de acordo com as características do contexto do projeto de software.
- (Faz com orientação). Selecionar e implementar o tipo de ciclo de vida de software (ou paradigma de engenharia de software - cascata, espiral, iterativo, baseado em modelo de maturidade, etc.) de acordo com as características do contexto do projeto de software.

Bibliografia básica

1. CHEMUTURI, M. K.; CAGLEY, T. M. Jr. Mastering Software Project Management: Best Practices, Tools and Techniques. J. Ross Publishing, 2010..
2. COHN, M. Agile estimating and planning Upper Saddle River, NJ: Prentice Hall Professional Technical Reference, 2006.
3. PMI. Um Guia do Conhecimento em Gerenciamento de Projetos. 5.ª edição. São Paulo: Saraiva, 2013.

Bibliografia complementar

1. FAIRLEY, R.E. Managing and Leading Software Projects. John Wiley & Sons, 2009.
2. HIGHSMITH, J. A. Agile project management: creating innovative products 2nd. edition. Addison-Wesley, 2010.
3. MARTINS, J. C. C. Gerenciando projetos de desenvolvimento de software com PMI, RUP e UML. 4.ª edição. Brasport, 2007.
4. SCHWABER, K. Agile project management with Scrum. Microsoft Press, 2004.

Governança e Gestão de Serviços de Software

Ementa

Conceitos de governança e de gestão de serviços de software (4h): software como um serviço (SaaS); correlação entre governança e gestão; níveis de governança e gestão (corporativa, de TI e de software); princípios de governança e gestão (conformidade (*compliance*) e alinhamento ao objetivo da organização). Governança de serviços de software (16h): políticas e plano diretor de TI; padrões e modelos de governança de TI; planejamento e controle estratégico de serviços; objetivos de controle; alinhamento entre objetivos organizacionais e objetivos dos serviços de software; diagnóstico de maturidade de processos e serviços de software; indicadores e avaliação de desempenho de serviços; direcionamento, avaliação, monitoramento e controle de serviços de software. Governança de dados organizacionais (8h): segurança da informação; riscos organizacionais relacionados à segurança física e lógica de sistemas de software; lei de proteção de dados pessoais; padrões e modelos de segurança da informação. Gestão de serviços de software (24h): estratégia organizacional para serviços de software (portfólio de serviços); padrões e modelos relacionados a serviços de software; ciclo de vida de serviços de software (planejamento, desenvolvimento, implantação, sustentação, descontinuação); manutenção de serviços de software (correção, refatoração, adaptação e evolução); central de serviços (gestão de incidentes e problemas em serviços); controle de configuração, de mudança e de qualidade de serviços (acordo de nível de serviço). Modelagem de serviços de software (12h): notação BPMN (*Business Process Modeling Notation*); representação de serviços de software como processos de negócio; análise de processo atual ("*as is*"), validação e evolução de processos ("*to be*") de serviços de software.

Competências mínimas:

- (Faz com orientação). Planejar a estratégia de software para uma organização, considerando a pouca disponibilidade de informações de longo prazo, a escassez de recursos para implementação dessa estratégia e a necessidade de alinhamento entre a GS e a governança corporativa.

- (Faz com orientação). Controlar o uso atual e futuro de software, otimizando o seu valor agregado para o negócio da organização.
- (Faz com orientação). Planejar e implementar (projetar, desenvolver, implantar, monitorar, medir, controlar e sustentar) uma estratégia organizacional para GSS em conformidade com o plano estratégico da organização.
- (Faz com orientação). Modelar SS e processos de negócio, usando BPMN, em conformidade com os requisitos organizacionais dos processos e serviços.

Bibliografia básica

1. BROOKS, P. Metrics for Service Management: Designing for ITIL. 1st edition. Van Haren Publishing, 2012.
2. GREMBERGEN, W. V.; HAES, S. Enterprise Governance of Information Technology: Achieving Strategic Alignment and Value. Springer, 2010.
3. SHIAVON, M. Acordos de nível operacional para controle do processo de manutenção de software, Mestrado em Engenharia Elétrica, Universidade Federal de Goiás, 2009.
4. FERNANDES, A. A. Implantando a governança de TI: da estratégia à gestão dos processos e serviços. 3. ed., RJ: Brasport, 2012. 615p.

Bibliografia complementar

1. SOULA, J. M. F. ISO/IEC 20000: Gerenciamento de serviços de tecnologia da informação. Rio de Janeiro: Brasport, 2013.
2. GUTH, S. Contract Negotiation Handbook: Software as a Service. Guth Ventures LLC, 2013.
3. MOLINARO, L. F. R.; RAMOS, K. H. C. Gestão de Tecnologia da Informação: Governança de TI: Arquitetura e Alinhamento entre Sistemas de Informação e o Negócio. São Paulo: LTC, 2011.
4. FITZSIMMONS, J. A.; FITZSIMMONS, M. J.; BORDOLOI, S. K. Service management: operations, strategy, information technology. 8. ed., McGraw-Hill, 2014.
5. FREITAS, M. A. S. Fundamentos do gerenciamento de serviços de TI. 2. ed. São Paulo: Brasport, 2013.

Implantação e Entrega de Software

Ementa

Conceitos de DevOps (4h): conceitos básicos; histórico. Técnicas para integração de código (16h): integração baseada em design de software; configuração de software em tempo de execução, integração (*build*) e entrega (*delivery*) de software ou entrega por fluxo via aplicação (*deploy*); automação para integração, transição e entrega contínuas. Monitoramento de software. Detecção e remoção de defeitos de código (16h): leitura e inspeção de código; depuração (*debugging*) e refatoração; classificação e tratamento de defeitos e débitos técnicos. Técnicas avançadas de implementação (28h): polimorfismo, reflexão, expressões regulares, parametrização (*generics*), fecho (*closure*), registros (*logging*), globalização (internacionalização e localização) de código.

Competências mínimas:

- (Faz com orientação). Construir código de forma colaborativa em equipe.
- (Faz com orientação). Usar e configurar ferramentas para edição, documentação, compilação, testes de unidade e de integração, e depuração (debugging) de software.
- (Faz com orientação). Usar e configurar ferramentas para controle de versão, coleta de medidas (desempenho, consumo de memória, análise estática e cobertura), integração (build) e transição de software.
- (Faz com orientação). Usar processos e técnicas adequadas para implementação e integração de software (criação de código, controle de versão, inspeção, teste e depuração).
- (Faz com orientação). Avaliar medidas e atributos de qualidade de código.
- (Faz com orientação). Ler código, localizar defeitos, depurar e refatorar código.

Bibliografia básica

1. GOODLIFFE, P. Como ser um programador melhor. Novatec, 2015.
2. MCCONNELL, S. Code Complete: um guia prático para a construção de software 2.ª edição. Porto Alegre, RS: Bookman, 2005.
3. ORAM, A.; WILSON, G. Beautiful code. O'Reilly, 2007.

Bibliografia complementar

1. AGANS, D. J. Debugging the nine indispensable rules for finding even the most elusive software and hardware problems. AMACOM, 2002.
2. BECK, K. Implementation patterns Upper Saddle River, NJ: Addison-Wesley, 2008.
3. FOWLER, M. Refatoração: aperfeiçoando o projeto de código existente. Porto Alegre: Bookman, 2004.
4. HUMBLE, J.; FARLEY, D. Entrega Contínua: Como Entregar Software de Forma Rápida e Confiável. Bookman, 2014.
5. KIM, G.; HUMBLE, J.; WILLIS, J.; DEBOIS, P. Manual de DevOps. Como Obter Agilidade, Confiabilidade e Segurança em Organizações Tecnológicas. Alta Books, 2018.
6. MARTIN, R. C. Clean code: a handbook of agile software craftsmanship. Prentice Hall, 2009.
7. MATYAS, S.; GLOVER, A.; DUVALL, P. Continuous Integration: Improving Software Quality and Reducing Risk. Addison-Wesley, 2007.

Implementação e Integração de Software

Ementa

Conceitos e terminologia (4h): princípios de implementação e integração de software (usar padrões, minimizar complexidade, antecipar mudanças, garantir verificabilidade); relação entre requisitos, design, implementação, integração e teste de software. Planejamento de implementação e integração de código (12h): critérios para escolha de linguagens e ferramentas de programação; instalação e configuração do ambiente de construção (IDE - *Integrated Development Environment)* para o tipo de projeto; plano de garantia de qualidade (inspeção, e teste) de código; plano de gerência de configuração (*build* e instalação) de código; padrões de codificação (leiaute e estilo); leitura de arquitetura e de design de software. Técnicas para implementação de código (16h): estratégias para criação e documentação de código fonte (variáveis, classes, interfaces, subrotinas, recursão, comandos condicionais, laços, tratamento de exceção); programação defensiva; busca e reutilização de código em bibliotecas e *frameworks*; práticas ágeis (programação em pares, propriedade coletiva de código). Técnicas para integração de código (12h):

integração baseada em design de software; configuração de software em tempo de execução, integração (*build*) e entrega (*delivery*) de software; automação para integração, transição e entrega contínuas. Detecção e remoção de defeitos de código (12h): leitura e inspeção de código; teste de unidade e de integração; depuração (*debugging*) e refatoração; classificação e tratamento de defeitos e débitos técnicos. Técnicas avançadas de implementação (8h): polimorfismo, reflexão, expressões regulares, parametrização (*generics*), fecho (*closure*), registros (*logging*), globalização (internacionalização e localização) de código.

Competências mínimas:

- (Faz com orientação). Construir código de forma colaborativa em equipe.
- (Faz com orientação). Usar e configurar ferramentas para edição, documentação, compilação, testes de unidade e de integração, e depuração (debugging) de software.
- (Faz com orientação). Usar e configurar ferramentas para controle de versão, coleta de medidas (desempenho, consumo de memória, análise estática e cobertura), integração (build) e transição de software.
- (Faz sem orientação). Criar código orientado a objetos em conformidade com o projeto (design) detalhado seguindo estratégias recomendadas.
- (Faz com orientação). Usar processos e técnicas adequadas para implementação e integração de software (criação de código, controle de versão, inspeção, teste e depuração).
- (Faz com orientação). Avaliar medidas e atributos de qualidade de código.
- (Faz com orientação). Ler código, localizar defeitos, depurar e refatorar código.
- (Faz com orientação). Criar e executar testes de unidade e de integração básicos empregando estratégias adequadas.

Bibliografia básica

1. GOODLIFFE, P. Como ser um programador melhor. Novatec, 2015.

2. MCCONNELL, S. Code Complete: um guia prático para a construção de software 2.ª edição. Porto Alegre, RS: Bookman, 2005.
3. ORAM, A.; WILSON, G. Beautiful code. O'Reilly, 2007.

Bibliografia complementar

1. AGANS, D. J. Debugging the nine indispensable rules for finding even the most elusive software and hardware problems. AMACOM, 2002..
2. BECK, K. Implementation patterns Upper Saddle River, NJ: Addison-Wesley, 2008.
3. FOWLER, M. Refatoração: aperfeiçoando o projeto de código existente Porto Alegre: Bookman, 2004.
4. IRVINE, K. R. C++ and object-oriented programming. Upper Saddle River: Prentice-Hall, 1997. 526 p.
5. MARTIN, R. C. Clean code: a handbook of agile software craftsmanship. Prentice Hall, 2009.

Manutenção de Software

Ementa

Conceitos de manutenção de software (4h): manutenção no contexto de sustentação de software; categorias (tipos) de manutenção (correção, prevenção, adaptação, adição, aperfeiçoamento); leis de Lehman. Manutenibilidade de software (8h): modularidade, reusabilidade, testabilidade, analisabilidade e modificabilidade; indicadores e métricas de manutenção (manutenibilidade, densidade de falhas e defeitos). Processos e atividades de manutenção (12h): ciclo de vida com integração e entrega contínuas (devops) e com entrega discreta; fluxo de trabalho e resultados esperados para cada tipo de manutenção; interface com Gerência de Configuração e Garantia da Qualidade; especificidades da manutenção corretiva (relato e confirmação de falha; identificação, classificação e tratamento de anomalias de software). Gestão de mudanças em software (8h): solicitação de mudança; análise de impacto e viabilidade; deliberação e planejamento de mudança; implementação de mudança; monitoramento e validação de mudança; testes de regressão; implantação de mudança. Técnicas de manutenção (12h): rastreabilidade bidirecional entre componentes (interdependências); compreensão de programas; estimativas

de custo e prazo de mudança; reengenharia; engenharia reversa; depuração; refatoração; descarte de software; ameaças à segurança em atividades de manutenção de software. Prática de manutenção de software (20h): seleção e uso de ferramentas e técnicas para cada tipo de manutenção.

Competências mínimas:

- (Faz com orientação). Planejar a sustentação de um software.
- (Faz com orientação). Planejar, monitorar e executar processo de transição de software para ambiente de produção.
- (Faz com orientação). Planejar um sistema de suporte técnico e registro de pedidos de mudança em software.
- (Faz com orientação). Realizar manutenção corretiva, adaptativa, preventiva e aditiva em software.

Bibliografia básica

1. IEEE Std 14764-2022 - Software engineering — Software life cycle processes — Maintenance.
2. SWEBOK - Guide to the Software Engineering Body of Knowledge, V.4, Chapter 7: Software Maintenance, IEEE Computer Society, 2023. Disponível em https://swebok.org.

Bibliografia complementar

1. Software maintenance management: evaluation and continuous improvement, Alain April, IEEE Computer Society, 2008.
2. Software maintenance: concepts and practice, Penny Grubb, Armstrong A. Takang. World Scientific, 2003.
3. SWECOM - Software Engineering Competency Model, Chapter 15: Software Sustainment Skill Area, IEEE Computer Society, 2014. Disponível em: https://www.computer.org/volunteering/boards-and-committees/professional-educational-activities/software-engineering-competency-model.

Mercado e Economia de Software

Ementa

Conceitos de mercado e economia de software (4h): mercado interno e externo de software; oportunidades de empreendedorismo e inovação em software; software como produto e como serviço; estimativa de custo e preço de software. Modelos de negócio para software (4h): aquisição e contratação de software (produtos e serviços); modelos genéricos (venda, permuta, aluguel, serviço, B2B, B2C, assinatura, licenciamento); modelos digitais (código aberto *freemium, premium*, push/pull, uso de propaganda/publicidade, robin hood, layer player, white label). Legislação de software (16h): leis, normas e regulamentos brasileiros para o mercado local e para a exportação de software; formas de contratação de pessoas para trabalho em software; lei de software, direitos autorais e propriedade intelectual; legislação correlata (lei geral de proteção de dados; marco civil da internet; lei de licitações). Mercado interno e externo de software (8h): características e exigências do mercado interno e externo; programas de incentivo à exportação e à produção de software; fontes de recursos nacionais e internacionais; publicação e divulgação de software para o mercado alvo. Engenharia econômica de software (8h): tributação (impostos e taxas); precificação de software e serviços associados; ciclo de vida econômico do software (demanda, modelagem, produção, fornecimento, distribuição, sustentação, substituição ou retirada); métodos de análise econômica (custo-benefício, *breakeven*, retorno de investimento, *business case*, avaliação de alternativas e tomada de decisão). Planejamento de negócios baseados em software (24h): identificação de oportunidades de inovação em software; análise de oferta/demanda; criação de planos de negócio de software; planejamento para o mercado (local, nacional e global).

Competências mínimas:

- (Faz sem orientação). Explicar características essenciais do mercado de software nacional e global.
- (Faz sem orientação). Explicar as principais leis e normas pertinentes à indústria de software no Brasil.
- (Faz sem orientação). Explicar os principais modelos de negócio baseados em software aplicados no mercado global.

- (Faz com orientação). Conhecer o impacto da legislação sobre negócios baseados em software.
- (Faz com orientação). Elaborar plano de negócio para software inovador e para negócios digitais baseados em software, usando modelos de negócio orientados ao mercado.
- (Faz com orientação). Precificar produtos e serviços de software, observando a realidade do mercado e as características do projeto.
- (Faz com orientação). Alinhar decisões técnicas com os objetivos do negócio no qual o software está inserido.

Bibliografia básica

1. GRADY, S. O. The software paradox: the rise and fall of the commercial software market. O'Reilly, 2015.
2. SALIM, C. S. Construindo planos de negócios: todos os passos necessários para planejar e desenvolver negócios de sucesso. 3.ª edição. Rio de Janeiro: Elsevier, 2005.
3. SWEBOK - Guide to the Software Engineering Body of Knowledge, V.4, Chapter 15 - Software Engineering Economics. IEEE Computer Society, 2023. Disponível em http://swebok.org.

Bibliografia complementar

1. BROOKS J., F. P. O mítico homem-mês: ensaios sobre engenharia de software. Elsevier, 2009.
2. ENGHOLM J. H. Engenharia de software na prática. São Paulo: Novatec, 2010.
3. LINS, B. F. E. et al. O mercado de software no Brasil: problemas institucionais e fiscais Brasília (DF): Câmara dos Deputados, Coordenação de Publicações, 2007.
4. PIMENTEL, L. O. A proteção jurídica da propriedade intelectual de software: noções básicas e temas relacionados. IEL, 2008.

Modelagem de Software

Ementa

Conceitos de modelagem de software (4h): modelos como representação visual de software; distinção entre modelos de software (modelos de requisitos, arquitetura, design, código, testes) e modelos de engenharia de software (modelos de ciclo de vida, paradigma de desenvolvimento, gerência de projeto, processos de negócio). Paradigma de orientação a objetos (28h): objeto (oid), classe, encapsulação (interface e implementação de objeto); troca de mensagens entre objetos; abstrações (classificação, associação, composição, generalização e especialização); Notações para modelos orientados a objetos (4h): Linguagem de Modelagem Unificada (UML); modelos estruturais/estáticos e comportamentais/dinâmicos. Modelagem de requisitos de software (8h): modelo conceitual de dados; modelo de casos de uso; modelo de atividades e de estados. Modelagem de arquitetura de software (8h): modelo de pacotes; modelo de componentes; modelo de implantação. Modelagem de componentes de código de software (8h): modelo de classes e de objetos; modelo de interação. Avaliação da qualidade de modelos OO (4h): critérios de qualidade para modelos orientados a objetos; métodos de avaliação (análise estática) de modelos de software.

Competências mínimas:

- (Faz com orientação). Explicar o que é modelo de software e sua relação com qualidade de software e processos técnicos de engenharia de software.
- (Faz sem orientação). Empregar notações para registro de software orientado a objetos.
- (Faz com orientação). Criar modelos de software orientado a objetos.

Bibliografia básica

1. AMBLER, S. W. Agile modeling: effective practices for eXtreme programming and the unified process. Wiley, 2002.
2. EVANS, E. Domain-driven design: tackling complexity in the heart of software Boston, MA: Addison-Wesley, 2004,
3. MCLAUGHLIN, B. D.; POLLICE, G.; WEST, D. Head First Object-Oriented Analysis & Design, O'Reilly, 2006.

4. OMG. Object Management Group - About The Unified Modeling Language Specification Version 2.5.1. Disponível on-line em: https://www.omg.org/spec/UML/2.5.1/About-UML/#documents.

Bibliografia complementar

1. FOWLER, M. Analysis patterns: reusable object models. Addison-Wesley, 1997.
2. KOWAL, J. A Behavior models: specifying user's expectations. Prentice Hall, 1992.
3. MELLOR, S. J; SHLAER, S. Análise de sistemas orientada para objetos. São Paulo: McGraw-Hill, 1990.
4. MILLETT, S.; TUNE, N. Patterns, Principles and Practices of Domain-Driven Design. Wrox, 2015.
5. WAGNER, F. Modeling software with finite state machines: a practical approach. Auerbach, 2006.

Prática em Engenharia de Software

Ementa

Conceitos de prática profissional em Engenharia de Software: ambientação com o modo de operação da Fábrica de Software do INF; revisão do código de ética da Engenharia de Software; Aplicação do corpo de conhecimento da Engenharia de Software no contexto de projeto de extensão realizado na Fábrica de Software. Emprego de processos de Engenharia de Software em abrangência e profundidade; seleção e utilização de normas, métodos, técnicas e ferramentas de Engenharia de Software. Integração e consolidação de competências esperadas do profissional de Engenharia de Software, com exercício de práticas e atitudes embasadas no código de ética e na postura profissional. Fortalecimento entre ensino, pesquisa e extensão, por meio de projetos de especificação, desenvolvimento e manutenção de software para a comunidade externa à UFG.

Competências mínimas:

- (Faz sem orientação). Participar de equipes de trabalho de projetos na Fábrica de Software, aplicando o código de ética e a postura profissional da Engenharia de Software.

- (Faz sem orientação). Realizar pacotes de trabalho, desenvolvendo produtos ou serviços de software, dentro de um projeto na Fábrica de Software, aplicando conteúdos do corpo de conhecimento da Engenharia de Software.
- (Faz sem orientação). Aplicar competências cognitivas, comportamentais e técnicas para planejar, criar, modificar ou analisar produtos e serviços de software em projetos executados na Fábrica de Software.

Bibliografia básica

1. FÓRUM DE PRÓ-REITORES DE EXTENSÃO DAS INSTITUIÇÕES PÚBLICAS DE EDUCAÇÃO SUPERIOR BRASILEIRAS (FORPROEX). Política Nacional de Extensão Universitária. Porto Alegre: UFRGS/Pró-Reitoria de Extensão, 2012.
2. International Standard ISO/IEC/IEEE 12207:2017 – Systems and software engineering — Software life cycle processes.
3. SOMMERVILLE, I. Engenharia de Software. Pearson, 10.ª edição, 2015.
4. MCCONNELL, S. Code complete: a practical handbook of software construction. 2nd edition. Microsoft Press, 2009.

Bibliografia complementar

1. PAULA, W. P. Engenharia de software: fundamentos, métodos e padrões. 3.ª edição. LTC, 2009.
2. PRESSMAN, R. Engenharia de Software. 8.ª ed. McGraw-Hill, 2014.
3. SCHACH, S. R. Engenharia de software: os paradigmas clássicos orientados a objetos. 7.ª edição. McGraw-Hill, 2009.
4. SWEBOK - Guide to the Software Engineering Body of Knowledge, V.4, IEEE Computer Society, 2023. Disponível em https://swebok.org.

Processos de Engenharia de Software

Ementa

Conceitos de processos de engenharia de software (4h): definição de processo de engenharia de software (propósito, resultados, atividades); correlação entre processo e método, modelo, procedimento e metodologia; taxonomia de processos do ciclo de vida de software. Processos de contratação (*agreement*) de software (4h): aquisição e fornecimento de software. Processos organizacionais habilitadores de projetos de software (8h): gestão de modelo de ciclo de vida, de infraestrutura, de portfólio, de pessoas, de qualidade, e de conhecimento. Processos de gestão técnica de software (8h): gerência de projeto, de decisão, de risco, de configuração, e de informação; medição e garantia da qualidade. Processos técnicos de especificação de software (8h): definição de requisitos, arquitetura e design de software. Processos técnicos de construção de software (8h): implementação, integração, verificação, transição e validação de software. Processos técnicos de sustentação de software (4h): operação, manutenção e descontinuação de software. Modelos de qualidade de processos (20h): implementação, avaliação e melhoria da qualidade de processos de Engenharia de Software.

Competências mínimas:

- (Faz sem orientação). Explicar processos de Engenharia de Software e a relação dessa área de conhecimento com as demais da engenharia de software.
- (Faz sem orientação). Correlacionar os conceitos de processo de software, método de software, modelo de software, modelo de processo e modelo de qualidade de processo.
- (Faz com orientação). Explicar os processos do ciclo de vida de software: objetivo de cada processo e principais resultados.
- (Faz com orientação). Explicar o uso de modelos de qualidade de processo para implementação, avaliação e melhoria de processos de Engenharia de Software.

Bibliografia Básica

1. FAIRLEY, R. E. Managing and Leading Software Projects, Wiley-IEEE Computer Society Press, 2009.

2. ISO/IEC/IEEE 12207:2017. Standard for Systems and Software Engineering-Software Life Cycle Processes.
3. LARMAN, C. Agile and iterative development: a manager's guide. Addison-Wesley, 2004.

Bibliografia Complementar

1. BEEDLE, M.; SCHWABER, K. Agile software development with Scrum. Prentice Hall, 2002.
2. JOHNSON, K. A.; KULPA, M. K. Interpreting the CMMI: a process improvement approach. Auerbach, 2003.
3. PROWELL, S. J. Cleanroom software engineering technology and process Reading, Mass.: Addison-Wesley, 1999.

Processos de Qualidade de Software

Ementa

Conceitos de processos de qualidade de engenharia de software (4h): distinção entre garantia de qualidade e controle de qualidade; custos e impactos da qualidade de software; questões éticas e culturais da qualidade de software; taxonomia de processos de qualidade de software (processos do ciclo de vida do software). Processo de gestão da qualidade de software (4h): políticas, objetivos e procedimentos de qualidade de software organizacionais; critérios e métodos de avaliação da qualidade; monitoramento e avaliação da qualidade de software organizacional; competências relacionadas à qualidade de software. Processo de garantia da qualidade de software (12h): definição do conceito de conformidade; critérios e procedimentos para garantia de qualidade do produto e do processo de software; comunicação de resultados de avaliação de qualidade; priorização e resolução de incidentes e não conformidades. Processo de verificação de software (24h): estratégia para verificação de artefatos; priorização e seleção de artefatos de requisitos, design, implementação e integração para verificação; identificação e tratamento de anomalias detectadas na verificação; leis e técnicas aplicáveis à análise estática de qualidade (inspeção, revisão, auditoria) de software. Processo de validação de software (20h): estratégia para validação do atendimento de requisitos pelo software; priorização e seleção de tipos de teste e cenários/casos de teste para validação

do software; identificação e tratamento de anomalias detectadas na validação; leis e técnicas aplicáveis à análise dinâmica de qualidade (teste) de software.

Competências mínimas:

- (Faz sem orientação). Explicar processos de Engenharia de Software relacionados com a qualidade de software e a relação da área de conhecimento de qualidade de software com as demais da Engenharia de Software.
- (Faz com orientação). Explicar a diferença e a relação entre a garantia de qualidade do produto e do processo.
- (Faz com orientação). Explicar a diferença e a relação entre garantia de qualidade e controle de qualidade de software.
- (Faz com orientação). Usar um processo para garantia da qualidade de software.
- (Faz com orientação). Usar um processo para verificação de software.
- (Faz com orientação). Usar um processo para validação de software.
- (Faz com orientação). Colaborar em uma equipe na execução de um processo de qualidade de software.
- (Faz com orientação). Documentar resultados de processos de qualidade de software.
- (Faz com orientação). Identificar e utilizar ferramentas de apoio ao controle e garantia de qualidade de artefatos de software.

Bibliografia Básica

1. GALIN, D. Software Quality Assurance: From Theory to Implementation, Addison-Wesley, 2004
2. MURPHY, Mark L. C/C++ software quality tools Upper Saddle River: Prentice Hall PTR, 1996.
3. WAGNER, S. Software Product Quality Control. Springer-Verlag Berlin Heidelberg. 2013.

Bibliografia Complementar

1. BARTIÉ, A. Garantia da qualidade de software. Campus, 2002.
2. KAN, S. H. Metrics and models in software quality engineering. 2nd edition. Addison-Wesley, 2003.
3. KELLIHER, T. P.; KEEGAN, J. G.; OLIVER, D. W. Engineering complex systems with models and objects. McGraw-Hill, 1997.
4. OSCIANSK, A.; SOARES, M. S. Qualidade de Software. Novatec, 2007.

Requisitos de Software

Ementa

Conceitos de requisitos de software (4h): o construto "requisito"; taxonomia de requisitos de software; necessidades, expectativas e requisitos de *stakeholders*; requisitos de negócio, de sistema e de software; atributos de um requisito; características de um requisito individual e de um conjunto de requisitos. Processo de engenharia de requisitos (12h): eliciação de requisitos, análise de requisitos, especificação de requisitos, verificação e validação de requisitos. Modelos e métodos da engenharia de requisitos (32h): técnicas para eliciação (revisão de literatura, etnografia, entrevista, questionário, *workshop*, *benchmark*), gestão (rastreabilidade, aprovação e controle de mudanças), análise (modelagem conceitual de processos e dados, casos de uso e histórias de usuário, protótipo e *storyboard*), especificação (elaboração de documentos de requisitos) e verificação e validação (leitura de modelos e documentos) de requisitos de software. Gerência de projetos de engenharia de requisitos (8h): gestão de mudanças e evolução de requisitos; rastreabilidade de requisitos; priorização de requisitos; medição de requisitos. Mapeamento de requisitos para projeto arquitetural de software (8h): relação entre requisitos e arquitetura de software; alocação de requisitos ao software.

Competências mínimas:

- (Faz sem orientação). Explicar requisitos de software, a relação dessa área de conhecimento com as demais da engenharia de software e, particularmente, com a arquitetura de software.
- (Faz com orientação). Usar um processo de requisitos de software.
- (Faz com orientação). Eliciar requisitos de software.
- (Faz com orientação). Construir modelos conceituais de software.
- (Faz com orientação). Especificar requisitos de software.
- (Faz com orientação). Verificar e validar requisitos de software.
- (Faz com orientação). Identificar e aplicar ferramentas de requisitos de software.
- (Faz sem orientação). Colaborar em uma equipe na execução de um processo de requisitos.

Bibliografia Básica

1. ISO/IEC/IEEE 29148. ISO/IEC/IEEE International Standard - Systems and software engineering - Life cycle processes - Requirements engineering, 2018.
2. ROBERTSON, S. Mastering the Requirements Process: Getting Requirements Right. Addison-Wesley Professional, 3rd edition, 2012.
3. WIEGERS, K. E. Software Requirements. Microsoft Press, 3rd edition, 2013.
4. WINTERS, J. P.; SCHNEIDER, G. Applying use cases: a practical guide, 2nd edition. Addison-Wesley, 2001. (The Addison-Wesley object technology series).

Bibliografia Complementar

1. COCKBURN, A. Writing Effective Use Cases. Addison-Wesley, 2001.
2. MELLOR, S. J; SHLAER, S. Análise de sistemas orientada para objetos. São Paulo: McGraw-Hill, 1990.
3. MILLER, G.; ARMOUR, F. Advanced use case modeling: software systems. Addison-Wesley, 2001. (object technology series).
4. YOURDON, E.; COAD, P. Análise baseada em objetos. 2a. edição. Rio de Janeiro: Campus, 1996.
5. WIEGERS, K. E. More about software requirements: thorny issues and practical advice. Microsoft Press, 2006.

Software Concorrente e Distribuído

Ementa

Conceitos de concorrência e distribuição em software (4h): processos e *threads*; distribuição, concorrência e paralelismo; middleware, internet das coisas, contêineres, troca de mensagens, chamada de serviços web; qualidade de produto de software (escalabilidade, tolerância a falhas, disponibilidade, desempenho). Execução e comunicação de processos em software (4h): *sockets*; chamadas remotas; objetos distribuídos; *threads*; semáforos; bloqueios e *deadlocks*. Construção de software distribuído (28h): arquitetura, projeto e implementação de aplicação distribuída; uso de interface de programação; RESTful, serviços web e micro-serviços (microservices); troca de mensagens (*messaging*). Construção de software concorrente (24h): projeto e

implementação de concorrência e paralelismo em software; *thread safety*; aplicações *IO-intensive* e *CPU-intensive*; memória compartilhada; sincronização e controle de concorrência baseada em bloqueio e em *Software Transactional Memory*. Verificação e validação de software concorrente e distribuído (4h): métodos e critérios para análise estática (inspeção) e para análise dinâmica (teste) de software concorrente e de software distribuído.

Competências mínimas:

- (Faz com orientação). Explicar a construção de software, concorrência/distribuição e a relação desses com a engenharia de software e de sistema.
- (Faz com orientação). Usar e configurar ferramentas para edição, documentação, compilação, depuração (debugging), build, teste, controle de versão, coleta de medidas (desempenho, consumo de memória, análise estática e cobertura), integração contínua, e aplicativos fundamentais de linha de comandos.
- (Faz com orientação). Criar código orientado a objetos em conformidade com o projeto (design) detalhado seguindo estratégias recomendadas.
- (Faz com orientação). Usar processos para a construção de software (criação de código, controle de versão, inspeção e integração).
- (Faz com orientação). Detalhar projeto (design) em conformidade com requisitos de software.
- (Faz com orientação). Localizar e reutilizar código (bibliotecas e frameworks).
- (Faz com orientação). Colaborar com a construção de código em equipe.
- (Faz com orientação). Avaliar a qualidade interna de código e, quando apropriado, aplicar estratégias recomendadas de refatoração.
- (Faz com orientação). Explicar conceitos de concorrência e distribuição.
- (Faz com orientação). Criar código que faz uso de concorrência, distribuição e ferramentas correlatas.

Bibliografia básica

1. BUTCHER, P. Seven Concurrent Models in Seven Weeks. Pragmatic Bookshelf, 2014.
2. DAIGNEAU, R. Service Design Pattern. Addison-Wesley, 2011.
3. FOSTER, I. Designing and building parallel programs concepts and tools for parallel software engineering Reading, Mass.: Addison-Wesley, 1995.

Bibliografia complementar

1. BARNES, L. L. Client/server & beyond strategies for 21st century. Prentice-Hall, 1997.
2. NEWMAN, S. Building Microservices. O'Reilly, 2015.
3. PAGE-JONES, M. Fundamentals of object-oriented design in UML. Addison-Wesley, 2003.
4. RODRIGUES, L.; GUERRAOUI, R. Introduction to reliable distributed programming. Springer, 2006.
5. SHATZ, S. M. Development of distributed software concepts and tools. Macmillan, 1993.

Software, Empreendedorismo e Sociedade

Ementa

Contexto histórico da Engenharia de Software e sua importância para a Sociedade (4h). Engenharia de Software como subárea da Computação e interdisciplinaridade (2h). Curso superior em Engenharia de Software e perfil profissional (4h). Demanda de mercado para o profissional de Engenharia de Software (4h). Regulamentação da profissão (4h). Ética e o profissional de Engenharia de Software (4h). Direitos autorais, licenças e patentes (4h). Crimes digitais (4h). Software livre (2h). Questões ambientais, étnico-raciais, de gênero, de saúde e de inclusão digital (16h). Empreendedorismo digital (16h): conceitos básicos, inovação tecnológica e análise de casos.

Competências mínimas:

- (Faz com orientação). Compreender a evolução da Engenharia de Software.
- (Faz com orientação). Apresentar casos de aplicação da Engenharia de Software no cotidiano que evidenciam a sua importância crescente para a Sociedade.
- (Faz com orientação). Apresentar a Engenharia de Software como subárea da Computação e a sua intersecção com outras subáreas.
- (Faz com orientação). Apresentar o perfil profissional de cursos superiores em Engenharia de Software.
- (Faz com orientação). Compreender as oportunidades de empreendedorismo, o mercado de trabalho e a regulamentação da profissão de Engenheiro de Software.
- (Faz com orientação). Discutir casos que envolvam a prática da ética profissional do Engenheiro de Software.
- (Faz com orientação). Apresentar as principais leis e normas brasileiras aplicadas a produtos de software.
- (Faz com orientação). Discutir casos do cotidiano do profissional de Engenharia de Software que envolvam questões ambientais, étnico-raciais, de gênero, de saúde e de inclusão digital.

Bibliografia Básica

1. MACIEL, C.; VITERBO, J. (Org.). Computação e Sociedade: a Profissão – volume 1. 1. ed. Cuiabá: EdUFMT – Editora da Universidade Federal de Mato Grosso, 2020. 272p. Disponível em: https://bit.ly/2IQC6Hv
2. MACIEL, C.; VITERBO, J. (Org.). Computação e Sociedade: a Sociedade – volume 2. 1. ed. Cuiabá: EdUFMT – Editora da Universidade Federal de Mato Grosso, 2020. 271p. Disponível em: https://bit.ly/38UTTIp
3. MACIEL, C.; VITERBO, J. (Org.). Computação e Sociedade: a Tecnologia – volume 3. 1. ed. Cuiabá: EdUFMT – Editora da Universidade Federal de Mato Grosso, 2020. 269p. Disponível em: https://bit.ly/2Hdx5YT

Bibliografia Complementar

1. CHALITA, G. Os Dez Mandamentos da Ética. Editora Nova Fronteira, 2003.
2. DRUMMOND, V. Internet, Privacidade e Dados Pessoais. Editora Lumen Juris, 2003.
3. LUCCA, N.; FILHO, A. S. Direito & Internet. Editora Edipro, 2001.
4. MASIERO, P. Ética em Computação. Editora da USP, 2000.
5. PAESANI, L. M. Direito de Informática, 3a edição, Editora Atlas 2012.

Software para Dispositivos Móveis

Ementa

Conceitos de computação móvel (4h): *smartphones* e outros tipos de dispositivos; sensibilidade ao contexto; tecnologias de comunicação e persistência de dados; infraestrutura remota e computação em nuvem (*cloud computing*). Arquiteturas de software para dispositivos móveis (12h): componentes (visuais, de negócio e de dados); segurança e privacidade da aplicação; integração com infraestrutura remota, considerando banda e latência. Ambientes para desenvolvimento de aplicativos móveis (12h): plataformas, frameworks, ferramentas e interfaces de programação de aplicações (APIs) de software para dispositivos móveis. Projeto da interação com o usuário em aplicativos móveis (12h): qualidade da experiência do usuário em dispositivos móveis; componentes visuais e controle da interação. Prática de desenvolvimento de aplicativos móveis (24h): apresentação, processamento e transferência de dados; sincronização e serviços web.

Competências mínimas:

- (Faz sem orientação). Participar da construção de software em equipe e colaborar com outros membros da equipe.
- (Faz com orientação). Elaborar o projeto (design) de software móvel.
- (Faz sem orientação). Implementar a persistência de dados.
- (Faz com orientação). Auxiliar a escolha de frameworks, plataformas e ambientes de construção de construção de software.
- (Faz com orientação). Executar revisões, testes e inspeções do aplicativo móvel.

Bibliografia básica

1. LECHETA, Ricardo R., Google Android. Aprenda a Criar Aplicações Para Dispositivos Móveis com o Android SDK, 4ª edição, 2012, São Paulo: Novatec.
2. FOROUZAN, B. A., COLLEGE, D. A.. Cryptography and Network Security. McGraw-Hill, 2008
3. SCHACH, Stephen R. Engenharia de Software. 7. ed. McGraw-Hill - Artmed, 2009.

Bibliografia complementar

1. DORNELAS, J. C. A. Empreendedorismo: Transformando Idéias em Negócios. São Paulo: Ímpetus, 2005.
2. FOWLER, Martin. Refactoring. Addison-Wesley, 1999. 431 p p.
3. LEWIS, Shaum. Native Mobile Development: A Cross-Reference for iOS and Android. O'Reilly Media; 1st edition (November 26, 2019).
4. OROSZ., Gergely. Building Mobile Apps at Scale: 39 Engineering Challenges. Primedia E-launch LLC (April 7, 2021)
5. PREECE, Jennifer; ROGERS, Yvonne; SHARP, Helen. Design de Interação: Além da Interação Homem-Computador. Porto Alegre: Bookman, 2005.

Software para Persistência de Dados

Ementa

Conceitos de persistência de dados (4h): dados, metadados e modelos de dados; transferência e armazenamento de dados em software; sistemas de arquivos, sistemas de bancos de dados e *data warehouse*. Tecnologias para persistência de dados (8h): arquivos em formato texto e binário; serialização de objetos; formato JSON; documentos XML; bancos de dados SQL (relacionais) e NoSql (não relacionais); frameworks ETL (*extract, transform, load*). Arquitetura de software para persistência de dados (8h): padrões e estilos arquiteturais de persistência de dados; arquitetura do componente de persistência de dados. Integração de aplicações baseada em dados (8h): integração por troca de arquivos; integração por dados compartilhados. Evolução de bancos de dados (8h): manutenção evolutiva, adaptativa e corretiva de esquema de dados; extração, modificação e migração de dados;

refatoração de bancos de dados. Prática em persistência de dados (28h): modelagem e implementação de aplicação com persistência em sistema de arquivos e em banco de dados.

Competências mínimas:

- (Faz com orientação). Explicar a construção de software, persistência e a relação desses com a engenharia de software e de sistema.
- (Faz com orientação). Identificar e usar estratégia adequada de persistência de dados para um dado problema.
- (Faz com orientação). Usar ferramentas para definição e manipulação de bancos de dados.
- (Faz com orientação). Criar código que cria, busca, atualiza e remove dados em bancos de dados.
- (Faz com orientação) Refatorar (*refactoring*) bases de dados.

Bibliografia básica

1. AMBLER, S. W. Agile database techniques: effective strategies for the agile software developer. Wiley, 2003.
2. MCMURTRY, D. et al. Data Access for Highly-Scalable Solutions: Using SQL, NoSQL, and Polyglot Persistence. Microsoft Press, 2013.
3. REDMOND, E.; WILSON, J. Seven databases in Seven Weeks. Pragmatic Bookshelf, 2012.

Bibliografia complementar

1. AMBLER, S. J.; SADALAGE, P. J. Refactoring Databases. Addison-Wesley, 2006.
2. FALINO, J.; HERION, D.; MARTINER, W. Building distributed applications with ADO. John Wiley & Sons, 1999.
3. GOLENDZINER, L. G.; PRICE, R. T. Bancos de dados para aplicações não convencionais. EBAI, 1989.
4. KING, G.; BAUER, C. Java Persistence com Hibernate. Ciência Moderna, 2007.
5. MANNINO, M. V. Projeto, desenvolvimento de aplicações e administração de banco de dados. McGraw-Hill, 2008.

Software para Sistemas Ubíquos

Ementa

Conceitos de computação ubíqua (8h): internet das coisas (IoT); virtualização e computação em nuvem (*cloud computing*); sensores e atuadores; dispositivos inteligentes e computação autônoma; sistemas ciberfísicos (software embarcado e automação); sistemas sensíveis ao contexto. Arquitetura de sistemas ubíquos (12h): componentes de hardware (sensores e atuadores) e sua integração com componentes de software; requisitos de desempenho em tempo-real; requisitos de segurança (vulnerabilidades, criptografia, certificados digitais). Ambiente para desenvolvimento de aplicações ubíquas (12h): plataformas, frameworks, ferramentas e interfaces de programação de aplicações (APIs) de software para sistemas ubíquos. Prática de desenvolvimento de software para IoT (32h): projeto e desenvolvimento de código para sensor e atuador; intercâmbio de dados entre dispositivos de hardware (ou virtualizados) e software.

Competências mínimas:

- (Faz com orientação). Explicar a construção de software, dispositivos e a relação desses com a engenharia de software e de sistema.
- (Faz sem orientação). Explicar a construção de software e a relação dessa área de conhecimento com as demais da engenharia de software.
- (Faz com orientação). Usar e configurar ferramentas para edição, compilação, depuração (debugging), build, teste, controle de versão, coleta de medidas (desempenho, consumo de memória, análise estática e cobertura), integração contínua, e aplicativos de linha de comandos para busca e encadear comandos.
- (Faz com orientação). Criar código orientado a objetos em conformidade com o projeto (design) detalhado seguindo estratégias recomendadas.
- (Faz com orientação). Usar processos para a construção de software (criação de código, controle de versão, inspeção e integração).
- (Faz com orientação). Detalhar projeto (design) em conformidade com requisitos de software.
- (Faz com orientação). Detalhar projeto de interação (design) em conformidade com requisitos de software.

- (Faz com orientação). Localizar e reutilizar código (bibliotecas e frameworks).
- (Faz com orientação). Colaborar com a construção de código em equipe.
- (Faz com orientação). Explicar a construção de software, segurança e a relação desses com a engenharia de software e de sistema.
- (Faz com orientação). Auxiliar na criação de modelo de ameaça.
- (Faz com orientação). Criar código que faz uso de recursos de segurança (criptografia de dados, assinatura e verificação de assinatura digital).

Bibliografia básica

1. MCGRAW, G. Software Security: Build Security In. Addison-Wesley, 2006.
2. GREENGARD, S. The Internet of Things. The MIT Press, 2015.
3. WILMSHURST, T. An introduction to the design of small-scale embedded systems. Palgrave, 2001.

Bibliografia complementar

1. HOWARD, M. 19 deadly sins of software security programming: flaws and how to fix them. McGraw-Hill/Osborne, 2005.
2. KERN, C.; KESAVAN, A.; DASWANI, N. Foundations of security: what every programmer needs to know. Apress, 2007.
3. MANICO, J.; DETLEFESEN, A. Iron-Clad Java: Building Secure Web Applications. McGraw-Hill, 2014.
4. MARK, D. Dominando o desenvolvimento no iPhone: explorando o SDK do iPhone. Alta Books, 2009.
5. VAHID, F. Embedded system design: a unified hardware/software introduction. John Wiley & Sons, 2002.

Teste de Software

Ementa

Conceitos de testes de software (8h): relação entre testes e construção, garantia da qualidade e depuração de software; definições de falha e defeito de software; critérios de seleção e de adequação de testes; teste caixa branca/estrutural, caixa preta/funcional e o problema do oráculo (decisão sobre a corretude do comportamento do software); casos de teste e testabilidade; objetos de teste (unidade, componente, sistema). Objetivos de teste de software (8h): critério de sucesso de teste (encontrar defeitos ou demonstrar conformidade); testes de qualificação, aceitação, instalação, alfa e beta; testes operacionais de confiabilidade; testes de regressão; testes de desempenho, de segurança e de estresse; testes comparativos (*back-to-back*); testes de recuperação, de interface, de configuração e de usabilidade. Técnicas de teste de software baseadas em (16h): intuição e experiência do engenheiro de software (*ad hoc,* exploratório); domínio das entradas (particionamento de equivalência, análise de valor limite, randomização de entrada); fluxo de controle de código (grafo de fluxo, fluxo de dados); tipos de defeitos (histórico de defeitos, mutação); uso (perfil operacional, observação de usuários); modelos (tabelas de decisão, máquinas de estado, especificação formal, *workflow*); natureza da aplicação (web, aplicativo, embarcado). Processo de teste de software (16h): planejamento; geração de casos de teste; preparação do ambiente; execução; documentação e análise de resultados; relato de problemas Prática de teste de software (16h): uso de ferramentas de teste (drivers, stubs, geradores; captura e reexecução, comparadores e oráculos, análise de cobertura e de confiabilidade, rastreadores de caminho); automação de testes com ferramentas de integração contínua.

Competências mínimas:

- (Faz sem orientação). Explicar o teste de software, a relação dessa área de conhecimento com as demais da engenharia de software e a relação com o conceito de sistema.
- (Faz com orientação). Usar um processo de teste de software.
- (Faz com orientação). Desenvolver planos de teste para o teste de unidade.

- (Faz com orientação). Propor casos de teste segundo algum critério da técnica de teste funcional para o teste de unidade.
- (Faz com orientação). Propor casos de teste segundo algum critério da técnica de teste estrutural para o teste de unidade.
- (Faz com orientação). Construir código para automação do teste de software.
- (Faz com orientação). Aplicar teste exploratório no teste de sistema.
- (Faz sem orientação). Executar casos de teste.
- (Faz sem orientação). Citar e explicar métricas de teste de software.
- (Faz com orientação). Realizar medições pertinentes a teste de software.
- (Faz com orientação). Documentar atividades do teste de software.
- (Faz com orientação). Identificar e utilizar ferramentas de teste de software.

Bibliografia Básica

1. COPELAND, L. A Practitioner's Guide to Software Test Design. Artech House, 2003.
2. MYERS, G. J. The Art of Software Testing. Wiley. 2011.
3. IEEE/ISO/IEC 29119-1-2021 – International Standard - Software and systems engineering – Software testing – Part 1:General concepts.

Bibliografia Complementar

1. GROSS, H. Component-based software testing with UML. Springer, 2005.
2. MESZAROS, G. xUnit test patterns: refactoring test code. Addison-Wesley, 2007.
3. SWEBOK - Guide to the Software Engineering Body of Knowledge, V.4, Chapter 5: Software Testing, IEEE Computer Society, 2023. Disponível em https://swebok.org.
4. WU, M.; LI, K. Effective software test automation: developing an automated software testing tool. SYBEX, 2004.

6.4.3 Disciplinas optativas do núcleo específico

A seguir são apresentadas as disciplinas optativas para o curso. Conforme o fluxo sugerido, essas disciplinas deveriam ser cursadas no 7º e no 8º períodos do curso.

Compiladores

Ementa

A estrutura de um compilador. Análises léxica e sintática. Definições dirigidas por sintaxe e análise semântica. Organização da tabela de símbolos. Representação intermediária do programa-fonte. Geração do código-objeto da compilação. Noções de otimização de código-objeto. Implementação de um compilador.

Bibliografia básica

1. AHO, A. V; LAM, M. S.; SETHI, R.; ULLMAN, J. D. Compiladores: Princípios, técnicas e ferramentas. 2.ª ed. Pearson-Addison-Wesley, 2008.
2. APPEL, A. W. Modern Compiler Implementation in C - Basic Techiniques. Cambridge University Press, 1997.
3. LOUDEN, K. C. Compiladores - Princípios e Práticas. Editora Thompson 2004.

Bibliografia complementar

1. APPEL, A. W. Modern Compiler Implementation in Java. 2nd edition. New York: Cambridge University Press, 2002.
2. HANSON, D. R.; FRASER, C. W. A Retargetable C Compiler: Design and Implementation. Benjamin Cummings Pub., 1995.
3. HOLMES, J. Modern Compiler Implementation in Java. Prentice Hall, 1995.
4. LEBLANC, R. J.; FISCHER, C. N. Crafting a Compiler with C. Benjamin/Cummings, 1991.
5. MUCHNICK, S. S. Advanced Compiler Design and Implementation. Morgan Kaufmann, 1997.

Desenvolvimento Full Stack

Ementa

Abordagens, técnicas, tecnologias e ferramentas para desenvolvimento de sistemas de informação para múltiplas plataformas e multimodais. Plataformas para front-end e back-end. Prática em desenvolvimento de aplicações móveis. Computação móvel e ubíqua: conceitos, principais características, internet das coisas, smart spaces, sensibilidade a contexto, tecnologias de comunicação e desafios. Projeto de aplicações móveis: macro e micro arquitetura, padrões de interface com usuário, persistência de dados, segurança, privacidade e comunicação.

Bibliografia básica

1. GREBE, S. Hands-On Full-Stack Web Development with GraphQL and React: Build scalable full-stack applications while learning to solve complex problems with GraphQL. Birmingham, Inglaterra: Pack Publishing, 2019.
2. KRUMM, J. Ubiquitous Computing Fundamentals. 1. ed., Chapman & Hall/CRC, 2009.
3. NORTHWOOD, Chris. The Full Stack Developer: Your Essential Guide to the Everyday Skills Expected of a Modern Full Stack Web Developer. Manchester, Inglaterra: Apress, 2018.

Bibliografia complementar

1. APPLE. Apple Developer (https://developer.apple.com), acessado em Agosto de 2022.
2. GARDNER, D.; GRIGSBY, J. Head First Mobile Web. O'Reilly. 2011.
3. GOOGLE. Android Developers (https://developer.android.com/), acessado em Agosto de 2022.
4. GUBBI, J.; BUYYA, R.; MARUSIC, S.; PALANISWAMI, M. 2013. Internet of Things (IoT): A vision, architectural elements, and future directions. Future Gener. Comput. Syst. 29, 7 (September 2013), 1645-1660.
5. NEIL, T. Padrões de Design para Aplicativos Móveis. O'Reilly, 2012.

Empreendedorismo Digital

Ementa

O perfil e as habilidades do empreendedor. Estudo de mecanismos e procedimentos para ideação, validação e lançamento de uma empresa digital no mercado. Montagem e gerenciamento de equipe. Constituição, tributação e legalização de empresas. Inovação tecnológica na geração de novos produtos e negócios. Marketing e competitividade. Escala e avaliação de desempenho. Financiamento de empresas digitais. Análise de Casos. Estratégias e modelos de negócio eletrônicos.

Bibliografia básica

1. BARON, R. A.; SHANE, S. A. Empreendedorismo - Uma Visão do Processo. 1. ed. São Paulo: Editora Thomson Learning, 2011.
2. FARAH, O.; CAVALCANTI, M.; MARCONDES, L. Empreendedorismo Estratégico – Criação e Gestão de Pequenas Empresas. São Paulo: Editora Thomson Learning, 2008.
3. SALIM, C.; HOCHMAN, N.; RAMAL, A.; RAMAL, S. Construindo planos de negócios: todos os passos necessários para planejar e desenvolver negócios de sucesso. 3. ed. Rio de Janeiro: Editora Elsevier, 2005.

Bibliografia complementar

1. CAVALCANTI, M. Gestão estratégica de negócios: evolução, cenários, diagnóstico e ação. 2. ed. São Paulo: Pioneira Thomson Learning, 2007.
2. CHIAVENATO, I. Empreendedorismo: Dando asas ao espírito empreendedor. 2.ed. São Paulo: Saraiva, 2008.
3. DOLABELA, F. O segredo de Luísa: uma ideia e uma paixão e plano de negócios. GMT Editores, 2008.
4. FERRARI, R. Empreendedorismo para computação: criando negócios de tecnologia. 1. ed. Rio de Janeiro: Elsevier, 2009.
5. IBPQ. GEM – Global Entrepreneurship Monitor. Empreendedorismo no Brasil: Relatório Executivo. Paraná: IBPQ, 2014. Disponível em: http://www.ibpq.org.br.

Engenharia de Sistemas de Informação 1

Ementa

Sistemas de Informação em perspectiva. Teorias de Sistemas de Informação. Fundamentos e abordagens de Engenharia de Sistemas. Teoria e Design de Informação. Linguagem e domínios linguísticos. Pensamentos processual, contextual e relacional. Introdução a Métodos de Estruturação de Problemas e Fundamentos de pesquisa operacional soft. Complexidade sistêmica. Distinção do sistema definido em torno da situação-problema. SoS - Sistema de interesse (sistema relevante). Pressupostos da visão organicista. Cibernética de terceira ordem. Metodologia SSM – estágios de 1 a 4. Mapeamento cognitivo. Epistemologia, teoria e prática em Sistemas de Informação. Pesquisa etnográfica e Pesquisa-ação. Modelagem conceitual.

Bibliografia básica

1. ROSINI, A. M. Administração de sistemas de informação e a gestão do conhecimento. 2. ed. rev., ampl. São Paulo: Cengage Learning, 2012.
2. SENGE, P. M. A Quinta Disciplina - Arte e Prática da Organização que Aprende. 29. ed., Rio de Janeiro: BestSeller, 2013.
3. VASCONCELLOS, M. J. Pensamento Sistêmico: O Novo Paradigma da Ciência. 10. ed. rev. e atual, Campinas: Papirus, 2013.

Bibliografia complementar

1. ACKOFF, R. L. Planejamento empresarial. Rio de Janeiro: Livros Técnicos e científicos, 1975. 114 p.
2. BEER, S. Cibernética e administração industrial. Rio de Janeiro: Zahar, 1969.
3. BUCKLEY, W F. A sociologia e a moderna teoria dos sistemas. Tradução de Octavio Mendes Cajado. 2. ed. São Paulo: Cultrix, 1976.
4. CHANDOR, A; GRAHAM, J; WILLIAMSON, R. Análise de sistemas: teoria e prática. Tradução de Antônio Garcia de Miranda Netto. Rio de Janeiro: LTCE, 1977.
5. CHURCHMAN, C. W. Introdução à teoria dos sistemas. Tradução de Francisco M. Guimarães. 2. ed., Petrópolis: Vozes, 1972.

Gestão de Sistemas e de TI

Ementa

Fundamentos de gestão. Entendendo a organização de TI. Processos de gestão de tecnologia da informação. Modelos e tecnologias de gestão. Gestão estratégica de tecnologia da informação. Gestão de sistemas de produção e operações de TI. Análise de negócio. Modelos e padrões de referência em gerenciamento de serviços de TI.Método de implementação do gerenciamento de serviços de TI. Gerenciamento do ciclo de vida de serviços de TI. Gerenciamento de projetos no contexto de TI. Projeto e desenvolvimento de serviços de TI.

Bibliografia básica

1. FERNANDES, A. A. Implantando a governança de TI: da estratégia à gestão dos processos e serviços. 3. ed., Rio de janeiro: Brasport, 2012. 615p.
2. FOINA, P. R. Tecnologia de informação: planejamento e gestão, 2. ed., São Paulo: Atlas, 2006. 339p.
3. RAINER, R. K.; POTTER, R. E.; TURBAN, E. Administração de tecnologia da informação: teoria e prática. Rio de Janeiro: Elsevier, 2005, 618p.

Bibliografia complementar

1. FITZSIMMONS, J. A.; FITZSIMMONS, M. J.; BORDOLOI, S. K. Service management: operations, strategy, information technology. 8. ed., McGraw-Hill, 2014.
2. FREITAS, M. A. S. Fundamentos do gerenciamento de serviços de TI. 2. ed. São Paulo: Brasport, 2013.
3. LAZZERI, J. C. Arquitetura orientada a serviços: fundamentos a estratégias. Rio de Janeiro: Ciência Moderna, 2009. 229 p.
4. MINTZBERG, H. Safári de estratégia: um roteiro pela selva do planejamento estratégico. 2. ed., Porto Alegre-RS: Bookman, 2010, 392p.
5. MOLINARO, L. F. R.; RAMOS, K. H. C. Gestão de Tecnologia da Informação: Governança de TI: Arquitetura e Alinhamento entre Sistemas de Informação e o Negócio. São Paulo: LTC, 2011.

6. REZENDE, D. A.; ABREU, A. F. Tecnologia da Informação aplicada a sistemas de informação empresariais: o papel estratégico da informação e dos sistemas de informação nas empresas. 8. ed, São Paulo: Atlas, 2011.
7. SHIMIZU, T. Decisão nas organizações: introdução aos problemas de decisão encontrados nas organizações e nos sistemas de apoio à decisão, 3. ed., São Paulo: Atlas, 2010. 317p.
8. SOULA, J. M. F. ISO/IEC 20000: Gerenciamento de serviços de tecnologia da informação. Rio de Janeiro: Brasport, 2013.

Introdução aos Sistemas de Informação

Ementa

Princípios de Sistemas de Informação (SI). Teoria geral de sistemas. Tipificação de SI. SI no contexto organizacional. Infraestrutura de Tecnologia da Informação. SI e suas aplicações. Visão geral de SI. Estudos de casos.

Bibliografia básica

1. LAUDON, K. C.; LAUDON, J. P. Sistemas de informação gerenciais: administrando a empresa digital. 9. ed., São Paulo: Pearson Prentice-Hall, 2010.
2. O'BRIEN, J., MARAKAS, G.; Administração de Sistemas de Informação. 15. ed., Porto Alegre, McGraw Hill Education, 2013.
3. ROSINI, A. M. Administração de sistemas de informação e a gestão do conhecimento. 2.ed. rev., ampl. São Paulo: Cengage Learning, 2012. xv, 212 p.

Bibliografia complementar

1. BATISTA, E. O. Sistemas de Informação: o uso consciente da tecnologia para o gerenciamento. 1.ed., São Paulo, Saraiva, 2006.
2. BRASIL ORGANIZAÇÃO PAN-AMERICANA DE SAÚDE; FUNDAÇÃO OSWALDO CRUZ. A Experiência brasileira em sistemas de informação em saúde. Brasília, D.F.: Ed. MS, 2009.
3. MATTOS, A. C. M. Sistemas de informação: uma visão executiva. 2. ed., São Paulo: Saraiva, 2010.

4. REZENDE, D..A. Tecnologia da informação aplicada a sistemas de informação empresariais: o papel estratégico da informação e dos sistemas de informação nas empresas. 8. ed., 2011.
5. VASCONCELLOS, M. J. Pensamento sistêmico: o novo paradigma da ciência. 10. ed. rev. e atual, Campinas: Papirus, 2013.

Introdução à Língua Brasileira de Sinais

Ementa

Introdução às práticas de compreensão e produção em LIBRAS por meio do uso de estruturas e funções comunicativas elementares. Concepções sobre a língua de sinais. O surdo e a sociedade.

Bibliografia Básica

1. BRASIL. Ministério da Educação. Instituto Nacional de Educação de Surdos. Curso básico de LIBRAS. Manaus: CD+, 2007. 1 DVD, color. (Educação de surdos, n. 6).
2. GESSER, A. LIBRAS? Que língua é essa? Crenças e preconceitos em torno da língua de sinais e da realidade surda. São Paulo: Parábola Editorial, 2009.
3. SKLIAR, Carlos (Org.). A surdez: um olhar sobre as diferenças. 6. ed. Porto Alegre: Mediação, 2012.

Bibliografia Complementar

1. CAPOVILLA, F. C.; RAPHAEL, W. D. (Ed.). Enciclopédia da Língua de Sinais Brasileira. v. 1 e 2. São Paulo: EDUSP, 2004
2. FELIPE, T.; MONTEIRO, M. S. LIBRAS em contexto. Curso Básico. Brasília: Ministério da Educação e do Desporto/Secretaria de Educação Especial, 2001.
3. PIMENTA, N.; QUADROS, R. M. Curso de LIBRAS 1 Iniciante. 3 ed. rev. e atualizada. Porto Alegre: Editora Pallotti, 2008.
4. SACKS, Oliver. Vendo Vozes: uma viagem ao mundo dos surdos. Tradução Laura Motta. São Paulo: Editora Cia das Letras, 1999.
5. THOMA, Adriana da Silva; LOPES, Maura Corcini (Coautor). A invenção da surdez: cultura, alteridade, identidade e diferenca no

campo da educação. Santa Cruz do Sul, RS: EDUNISC, 2005. 232 p. Inclui bibliografia. ISBN 8575780794 (Broch.).

Jogos Digitais

Ementa

Conceitos fundamentais de jogos, Storyboard, Mecânica de Jogos, game engines, ambientação e animação em jogos. Aplicação de técnicas de apresentação dos jogos. Deploy de jogos em plataformas. Noções de Mercado e tendências tecnológicas para jogos.

Bibliografia básica

1. MARTINHO, C.; SANTOS, P.; PRADA, R. Design e Desenvolvimento de Jogos. 1.ed. Lisboa: FCA - Editora de Informática, 2014.
2. SCHELL, J. The Art of Game Design: A book of lenses. Nova Iorque: CRC Press, 2008.
3. SCHUYTEMA. P. Design de Games - Uma Abordagem Prática. 1. ed. Cengage CTP, 2008.

Bibliografia complementar

1. LIMA, A. Design de personagens para games Next-Gen. Rio de Janeiro: Ed. Ciencia Moderna, 2011.
2. FEIJÓ, B.; CLUA, E.; SILVA, F. Introdução à Ciencia da Computação com Jogos. Rio de Ja- neiro: Ed. Campus, 2009.
3. HUIZINGA, J. Homo ludens: O jogo como elemento da cultura. São Paulo: Perspectiva, 1996.
4. PERUCIA, A. S.; BERTHEM, A.; BERTSCHINGER, G.; CASTRO, R. R. Desenvolvimento de Jogos Eletronicos – Teoria e Prática. 2. ed. São Paulo: Ed. Novatec, 2007.
5. VASCONCELOS, J. B.; RIBEIRO, N. M. Tecnologias de Programação de Jogos . 1. ed. Lisboa: FCA - Editora de Informática, 2013.

Linguagens Formais e Autômatos

Ementa

Conceitos básicos de linguagens. Mecanismos geradores (gramáticas) e reconhecedores (determinísticos e não determinísticos) de linguagens regulares, livres de contexto e sensíveis ao contexto; relação entre estas classes de linguagens e suas principais propriedades. Hierarquia de Chomsky.

Bibliografia básica

1. HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. Introdução à teoria de autômatos: linguagens e computação. Campus, 2002.
2. SIPSER, M. Introduction to the theory of computation. ITP, 1997.
3. SUDKAMP, T. A. Languages and machines. 3.ª edição. Addison Wesley, 2006.

Bibliografia complementar

1. CARROLL, J.; LONG, D. Theory of finite automata. New Jersey: Prentice-Hall International Editions, 1989.
2. LEWIS, H. R.; PAPADIMITRIOU, C. H. Elementos de teoria da computação. 2.ª edição. Porto Alegre: Bookman, 2000.
3. MENEZES, P. F. B. Linguagens formais e autômatos. 5.ª edição. Porto Alegre: Sagra Luzzatto, 2005.
4. RAMOS, M. V. M.; NETO, J. J.; VEGA, I. S. Linguagens formais: teoria, modelagem e implementação. Bookman, 2009.
5. ROSEN, K. H. Matemática discreta e suas aplicações. 6.ª ed. Rio de Janeiro: McGraw Hill, 2009.

Matemática Discreta

Ementa

Conjuntos: finito, infinito, enumerável, cardinalidade. Técnica de diagonalização. Tópicos em teoria dos números. Estruturas algébricas e criptografia.

Bibliografia Básica

1. COUTINHO, S. C. Números Inteiros e Criptografia RSA . 5. ed. IMPA, 2003. (Coleção Matemática e Aplicações).

2. ROSEN, K. H. Matemática Discreta e Suas Aplicações, 6.ed. Boston: McGraw-Hill, 2009.
3. SANTOS, J. P. D. O. Introdução à Teoria dos Números. IMPA, 2006. (Coleção Matemática Universitária).

Bibliografia Complementar

1. GONÇALVES, A. Introdução à Álgebra, 5. ed.Rio de Janeiro: CNPQ-IMPA, 2013.
2. GRIMALDI, R. P. Discrete and Combinatorial Mathematics: An Applied Introduction. 5. Boston: Pearson Addison-Wesley, 2003.
3. HERSTEIN, I. Abstract Algebra. 2 ed. Wiley, 1996.
4. LEMOS, M. J. M. S. Criptografia, Números Primos e Algoritmos. IMPA, 2007.
5. ROSEN, K. H. Elementary Number Theory and Its Application, 7.ed., McGraw-Hill Education, 2011.

Sistemas Distribuídos

Ementa

Princípios de Sistemas Distribuídos. Invocação Remota. Objetos Distribuídos. Arquiteturas Orientadas a Serviços e utilização de serviços Web. Computação Móvel e Ubíqua. Estudo de Casos de Tópicos Emergentes em Sistemas Distribuídos.

Bibliografia básica

1. ANDREWS, G. R. Foundations of multithreaded, parallel, and distributed programming. Addison-Wesley, 2000.
2. COULOURIS, G. F. et al. Distributed Systems: Concepts and Design, 5th edition, Addison-Wesley, 2012.
3. KUROSE, J. F.; ROSS, K. Redes de Computadores e a Internet, 6.ª edição. Pearson Education, 2013.

Bibliografia complementar

1. BIRMAN, K. P. Reliable distributed systems: technologies, web services, and applications, New York: Springer, 2005.
2. CLARK, M. et al. Web services business strategies and architectures. Expert Press, 2002.

3. JOSUTTIS, N. M. SOA in practice. O'Reilly, 2007.
4. LYNCH, N. A Distributed algorithms. M. Kaufmann, 1997.
5. TANENBAUM, A.S.; STEEN, M. van. Distributed Systems: Principles and Paradigms. Prentice Hall, 2nd Edition, 2006.

Sistemas Inteligentes de Apoio à Decisão

Ementa

Tomada de decisão e processo decisório. Métodos de análise de decisão multicritério. Fundamentos, técnicas e métodos de Sistemas de Apoio à Decisão. Abordagens centradas em dados, informação e conhecimento. Conceitos básicos de Inteligência Artificial, representação do conhecimento e raciocínio. Técnicas de Inteligência Artificial para desenvolvimento de Sistemas Inteligentes de Apoio à Decisão. Fundamentos de ciência de dados. Fundamentos de Inteligência de negócios. Data mining e visualização de dados. Data warehousing. Análise Inteligente de Dados.

Bibliografia básica

1. CHOO, C. W. A organização do conhecimento: como as organizações usam a informação para criar significado, construir conhecimento e tomar decisões. 2. ed. São Paulo: Ed. SENAC, 2006.
2. JOHNSON, J. D. Gestão de redes de conhecimento. São Paulo: Ed. SENAC, 2009.
3. VERCELLIS, C. Business intelligence: data mining and optimization for decision making. Chichester, U.K.: Wiley, 2009, 417p.

Bibliografia complementar

1. BIGUS, J. P. Data mining with neural networks: solving business problems from application development to decision support. New York: McGraw-Hill, 1996, 220 p.
2. CARVALHO, L. A. V. Datamining: a mineração de dados no marketing, medicina, economia, engenharia e administração. São Paulo: Ciência Moderna, 2005.
3. CHAKRABARTI, S. Data mining: know it all. Burlington, MA: Elsevier/Morgan Kaufmann Publishers, 2009, 460 p.

4. DAVENPORT, T. H.; PRUSAK, L. Conhecimento empresarial: como as organizações gerenciam o seu capital intelectual. Rio de Janeiro: Campus, 1998.
5. DE SORDI, J. O. Administração da informação: fundamentos e práticas para uma nova gestão do conhecimento. São Paulo: Saraiva, 2008.
6. FISCHER, L. 2010 BPM and Workflow handbook: methods, concepts, case studies and standards in business process management and workflow: spotlight on business intelligence. Future Strategies Inc., 2010, 281p.
7. GOLFARELLI, M. Data warehouse design: modern principles and methodologies. New York: McGrawHill, 2009, 458 p.

Software Básico

Ementa

Visão geral de um sistema computacional. Linguagem de montagem. Mapeamento de estruturas de linguagens de alto nível para a linguagem Assembly. Pilha de execução e modelo de chamadas de função. Interrupções e seus tipos. "Link-edição" e bibliotecas.

Bibliografia básica

1. BRYANT, R.; O'HALLARON, D. Computer Systems: A programmer's Perspective. 2nd ed. Prentice Hall, 2011.
2. DUNTEMANN, J. Assembly Language Step-by-Step: Programming wiht Linux. 3rd. ed. Willey, 1999.
3. WARFORD, J. S. Computer Systems.4th ed. Jones and Bartlet Publishers, 2010.

Bibliografia complementar

1. CARPINELLI, J. D. Computer systems organization & architecture. Addison-Wesley, 2001.
2. CARTER., P. A. PC Assembly Language, 2006. (disponível em: http://www.drpaulcarter.com/pcasm/).

3. HENNESSY, J. L.; PATTERSON, D. A. Computer organization and design: the hardware / software interface, 2nd ed. Morgan Kaufmann Publishers, 1998.
4. KERRISK, M. The Linux Programming Interface: A Linux and Unix System Programming Handbook. No Starch Press, 2010.
5. LOVE, R. Linux system programming, O'Reilly, 2007.

Tópicos em Engenharia de Software

Ementa

Tópicos emergentes em Engenharia de Software, que inclui, mas não se limita às áreas de conhecimento de: requisitos de software, arquitetura de software, *design* de software, construção de software, teste de software, operações de Engenharia de Software, manutenção de software, gerenciamento de configuração de software, gerenciamento de projetos de software, modelos e métodos de Engenharia de Software, processos de Engenharia de Software, qualidade de software, segurança de software, prática profissional em Engenharia de Software, Economia em Engenharia de Software e fundamentos de Computação, Matemática e Engenharia.

Bibliografia Básica

1. PRESSMAN, R.; MAXIM, B. Software Engineering: A Practitioner's Approach, 8th edition, McGraw-Hill, 2014.
2. SWEBOK - Guide to the Software Engineering Body of Knowledge, V.4, IEEE Computer Society, 2023. Disponível em https://www.computer.org/web/swebok.

Bibliografia Complementar

1. ISO/IEC/IEEE 24765. SEVOCAB: Software and Systems Engineering Vocabulary, 2017. Disponível em https://pascal.computer.org/sev_display/index.action.
2. ISO/IEC/IEEE 12207:2017. Standard for Systems and Software Engineering-Software Life Cycle Processes.
3. WAZLAWICK, R. S. Engenharia de software: teoria e prática. Campus, 2013.

6.5. Equivalência de disciplinas e transição entre currículos

É possível a transição entre currículos definidos na matriz anterior do BES (PPC de 2017) e aquele definido no presente PPC (PPC de 2024). O objetivo é maximizar os aproveitamentos de disciplinas já cursadas, reduzindo o esforço necessário para a integralização do currículo pelos estudantes sem, contudo, prejudicar a formação desejada.

O Quadro 4 mostra as disciplinas do PPC de 2024 que não têm equivalência com disciplinas do PPC de 2017. A transição do PPC de 2017 para o PPC de 2024 exige que estas disciplinas sejam cursadas com base nas definições do PPC de 2024.

Quadro 4. Disciplinas obrigatórias do PPC de 2024 sem equivalência no PPC de 2017.

DISCIPLINAS DO PPC DE 2024	CH	NATUREZA
Banco de Dados Avançado (NOVA)	64	NC – OBR
Gerência de Configuração de Software (NOVA)	64	NE – OBR
Gestão de Segurança da Informação (INF0405)	64	NC – OBR
Manutenção de Software (NOVA)	64	NE – OBR
Metodologia Científica (INF0307)	64	NC – OBR
Software para Dispositivos Móveis (NOVA)	64	NE – OBR

A equivalência entre as disciplinas do PPC de 2017 e 2024 é apresentada nos Quadros 5 e 6. No Quadro 5 é apresentada a equivalência entre disciplinas do PPC de 2017 para o PPC de 2024 que serve como referência de aproveitamento para os alunos do PPC de 2017 que quiserem migrar para o novo PPC. Já o Quadro 6 apresenta a equivalência entre disciplinas do PPC de 2024 para o PPC de 2017 e servirá como referência para a oferta das disciplinas enquanto os dois PPCs tiverem alunos ativos. É importante destacar que as disciplinas do Quadro 5 marcadas como "Sem equivalência" podem ser aproveitadas como uma disciplina de Núcleo Livre.

Quadro 5. Equivalência entre disciplinas do PPC de 2017 para o PPC de 2024.

DISCIPLINA DO PPC DE 2017	CH	NATUREZA	DISCIPLINA EQUIVALENTE NO PPC DE 2024	CH	NATUREZA
Álgebra Linear (IME0351)	64	NC – OBR	Sem equivalência		
Algoritmos e Estruturas de Dados 1 (INF0286)	64	NC – OBR	Algoritmos e Estruturas de Dados 1 (INF0286)	64	NC – OBR
Algoritmos e Estruturas de Dados 2 (INF0287)	64	NC – OBR	Algoritmos e Estruturas de Dados 2 (INF0287)	64	NC – OBR
Análise e Projeto de Algoritmos (INF0013)	64	NC – OBR	Análise e Projeto de Algoritmos (INF0013)	64	NC – OBR
Arquitetura de Computadores (INF0015)	64	NC – OBR	Arquitetura de Computadores (INF0016)	64	NC – OBR
Arquitetura de Software (INF0018)	64	NC – OBR	Arquitetura de Software (NOVA)	64	NC – OBR
Banco de Dados (INF0022)	64	NC – OBR	Banco de Dados (INF0022)	64	NC – OBR
Cálculo 1A (IME0350)	96	NC – OBR	Sem equivalência		
Compiladores (INF0337)	64	OPT	Compiladores (INF0337)	64	OPT
Computação e Sociedade (INF0283) Empreendedorismo Digital (INF0327)	32 64	NC – OBR OPT	Software, Empreendedorismo e Sociedade (NOVA)	64	NE – OBR
Construção de Software (INF0291)	128	NE – OBR	Implementação e Integração de Software (NOVA) e Implantação e Entrega de Software (NOVA)	64 64	NE – OBR NE – OBR
Design de Software (INF0295)	128	NE – OBR	Projeto de Software (INF0290) E Modelagem de Software (INF0292)	64 64	NC – OBR NE – OBR
Domínios de Software (INF0296)	64	NE – OBR	Domínios de Software (INF0296)	64	NE – OBR
Engenharia de Sistemas (INF0294)	64	NE – OBR	Engenharia de Sistemas de Informação 1 (INF0308)	64	OPT
Engenharia de Software (INF0056)	64	NC – OBR	Introdução à Engenharia de Software (NOVA)	64	NE – OBR
Experiência do Usuário de Software (INF0299)	64	NE – OBR	Interação Humano-Computador (NOVA)	64	NC – OBR

DISCIPLINA DO PPC DE 2017	CH	NATUREZA	DISCIPLINA EQUIVALENTE NO PPC DE 2024	CH	NATUREZA
Fundamentos de Matemática para Computação (INF0285) e Lógica Matemática (INF0305)	64 64	NC – OBR NC – OBR	Introdução à Matemática para Computação (NOVA)	64	NC – OBR
Gerência de Projeto de Software (INF0073)	64	NC – OBR	Gerência de Projeto de Software (INF0073)	64	NC – OBR
Governança e Gestão de Serviços de Software (INF0300)	64	NE – OBR	Governança e Gestão de Serviços de Software (INF0300)	64	NE – OBR
Interação Humano-Computador (INF0289)	64	NC – OBR	Interação Humano-Computador (NOVA)	64	NC – OBR
Introdução à Língua Brasileira de Sinais (FAL0214)	64	OPT	Introdução à Língua Brasileira de Sinais (FAL0214)	64	OPT
Introdução à Programação (INF0284)	128	NC – OBR	Introdução à Programação (INF0284)	128	NC – OBR
Linguagens e Paradigmas de Programação (INF0288)	64	NC – OBR	Linguagens e Paradigmas de Programação (INF0288)	64	NC – OBR
Linguagens Formais e Autômatos (INF0333)	64	OPT	Linguagens Formais e Autômatos (INF0333)	64	OPT
Mercado e Economia de Software (INF0381)	64	OPT	Mercado e Economia de Software (INF0381)	64	NE – OBR
Metodologia e Experimentação em Eng. de Software (INF0368)	64	OPT	Engenharia de Software Experimental (NOVA)	64	NE – OBR
Modelagem de Software (INF0292)	64	NE – OBR	Modelagem de Software (INF0292)	64	NE – OBR
Padrões de Arquitetura de Software (INF0302)	64	NE – OBR	Sem equivalência		
Pesquisa Operacional (INF0313)	64	OPT	Sem equivalência		
Prática em Engenharia de Software (INF0304)	320	NE – OBR	Prática em Engenharia de Software (INF0304)	320	NE – OBR
Probabilidade e Estatística A (IME0378)	64	NC – OBR	Probabilidade e Estatística (IME0231)	64	NC – OBR
Processos de Qualidade de Software (INF0297)	64	NE – OBR	Processos de Qualidade de Software (INF0297)	64	NE – OBR
Processos de Software (INF0293)	64	NE – OBR	Processos de Engenharia de Software (NOVA)	64	NE – OBR
Programação Orientada a Objetos (INF0164)	64	NC – OBR	Programação Orientada a Objetos (INF0164)	64	NC – OBR
Projeto de Software (INF0290)	64	NC – OBR	Projeto de Software (INF0290)	64	NC – OBR

DISCIPLINA DO PPC DE 2017	CH	NATUREZA	DISCIPLINA EQUIVALENTE NO PPC DE 2024	CH	NATUREZA
Redes de Computadores (INF0314) e Sistemas Operacionais (INF0310)	64 64	OPT OPT	Sistemas Operacionais e Redes (NOVA)	64	NC - OBR
Requisitos de Software (INF0188)	64	NE – OBR	Requisitos de Software (INF0188)	64	NE – OBR
Sistemas Distribuídos (INF0317)	64	OPT	Sistemas Distribuídos (INF0317)	64	OPT
Software Concorrente e Distribuído (INF0298)	64	NC – OBR	Software Concorrente e Distribuído (INF0298)	64	NC – OBR
Software para Persistência de Dados (INF0301)	64	NE – OBR	Software para Persistência de Dados (INF0301)	64	NE – OBR
Software para Computação Ubíqua (INF0306)	64	NE – OBR	Software para Sistemas Ubíquos (NOVA)	64	NE – OBR
Teste de Software (INF0303)	64	NE – OBR	Teste de Software (INF0303)	64	NE – OBR

Quadro 6. Equivalência entre disciplinas do PPC de 2024 para o PPC de 2017.

DISCIPLINA DO PPC DE 2024	CH	NATUREZA	DISCIPLINA EQUIVALENTE NO PPC DE 2017	CH	NATUREZA
Algoritmos e Estruturas de Dados 1 (INF0286)	64	NC – OBR	Algoritmos e Estruturas de Dados 1 (INF0286)	64	NC – OBR
Algoritmos e Estruturas de Dados 2 (INF0287)	64	NC – OBR	Algoritmos e Estruturas de Dados 2 (INF0287)	64	NC – OBR
Análise e Projeto de Algoritmos (INF0013)	64	NC – OBR	Análise e Projeto de Algoritmos (INF0013)	64	NC – OBR
Arquitetura de Computadores (INF0016)	64	NC – OBR	Arquitetura de Computadores (INF0015)	64	NC – OBR
Arquitetura de Software (NOVA)	64	NC – OBR	Arquitetura de Software (INF0018)	64	NC – OBR
Banco de Dados (INF0022)	64	NC – OBR	Banco de Dados (INF0022)	64	NC – OBR
Compiladores (INF0337)	64	OPT	Compiladores (INF0337)	64	OPT
Software, Empreendedorismo e Sociedade (NOVA)	64	NE – OBR	Computação e Sociedade (INF0283)	32	NC – OBR
Implementação e Integração de Software (NOVA) e Implantação e Entrega de Software (NOVA)	64 64	NE – OBR NE – OBR	Construção de Software (INF0291)	128	NE – OBR
Projeto de Software (INF0290) E Modelagem de Software (INF0292)	64 64	NC – OBR NE – OBR	Design de Software (INF0295)	128	NE – OBR
Domínios de Software (INF0296)	64	NE – OBR	Domínios de Software (INF0296)	64	NE – OBR
Engenharia de Sistemas de Informação 1 (INF0308)	64	OPT	Engenharia de Sistemas (INF0294)	64	NE – OBR
Introdução à Engenharia de Software (NOVA)	64	NE – OBR	Engenharia de Software (INF0056)	64	NC – OBR
Interação Humano-Computador (NOVA)	64	NC – OBR	Experiência do Usuário de Software (INF0299)	64	NE – OBR
Gerência de Projeto de Software (INF0073)	64	NC – OBR	Gerência de Projeto de Software (INF0073)	64	NC – OBR
Governança e Gestão de Serviços de Software (INF0300)	64	NE – OBR	Governança e Gestão de Serviços de Software (INF0300)	64	NE – OBR
Interação Humano-Computador (NOVA)	64	NC – OBR	Interação Humano-Computador (INF0289)	64	NC – OBR

DISCIPLINA DO PPC DE 2024	CH	NATUREZA	DISCIPLINA EQUIVALENTE NO PPC DE 2017	CH	NATUREZA
Introdução à Língua Brasileira de Sinais (FAL0214)	64	OPT	Introdução à Língua Brasileira de Sinais (FAL0214)	64	OPT
Introdução à Programação (INF0284)	128	NC – OBR	Introdução à Programação (INF0284)	128	NC – OBR
Linguagens e Paradigmas de Programação (INF0288)	64	NC – OBR	Linguagens e Paradigmas de Programação (INF0288)	64	NC – OBR
Linguagens Formais e Autômatos (INF0333)	64	OPT	Linguagens Formais e Autômatos (INF0333)	64	OPT
Mercado e Economia de Software (INF0381)	64	NE – OBR	Mercado e Economia de Software (INF0381)	64	OPT
Engenharia de Software Experimental (NOVA)	64	NE – OBR	Metodologia e Experimentação em Eng. de Software (INF0368)	64	OPT
Modelagem de Software (INF0292)	64	NE – OBR	Modelagem de Software (INF0292)	64	NE – OBR
Prática em Engenharia de Software (INF0304)	320	NE – OBR	Prática em Engenharia de Software (INF0304)	320	NE – OBR
Probabilidade e Estatística (IME0231)	64	NC – OBR	Probabilidade e Estatística A (IME0378)	64	NC – OBR
Processos de Qualidade de Software (INF0297)	64	NE – OBR	Processos de Qualidade de Software (INF0297)	64	NE – OBR
Processos de Engenharia de Software (NOVA)	64	NE – OBR	Processos de Software (INF0293)	64	NE – OBR
Programação Orientada a Objetos (INF0164)	64	NC – OBR	Programação Orientada a Objetos (INF0164)	64	NC – OBR
Projeto de Software (INF0290)	64	NC – OBR	Projeto de Software (INF0290)	64	NC – OBR
Requisitos de Software (INF0188)	64	NE – OBR	Requisitos de Software (INF0188)	64	NE – OBR
Sistemas Distribuídos (INF0317)	64	OPT	Sistemas Distribuídos (INF0317)	64	OPT
Software Concorrente e Distribuído (INF0298)	64	NC – OBR	Software Concorrente e Distribuído (INF0298)	64	NC – OBR
Software para Persistência de Dados (INF0301)	64	NE – OBR	Software para Persistência de Dados (INF0301)	64	NE – OBR
Software para Sistemas Ubíquos (NOVA)	64	NE – OBR	Software para Computação Ubíqua (INF0306)	64	NE – OBR
Teste de Software (INF0303)	64	NE – OBR	Teste de Software (INF0303)	64	NE – OBR

7. Políticas e procedimentos de gestão do curso

7.1. Política e gestão de estágio curricular

O estágio curricular segue as normas contidas na Instrução Normativa nº 01/2022 [UFG 2022] publicada pela PROGRAD/UFG, que dispõe sobre as orientações para elaboração de projetos pedagógicos de curso (PPC), que está em consonância com a legislação pertinente (Resoluções CEPEC/UFG nº 1557/2017, n°1538/2017, n° 1539/2017 e à Lei 11.788 de 2008).

O estágio curricular constitui um mecanismo de complementação e aperfeiçoamento de competências do discente, além de oportunidade de vivenciar experiências de trabalho em um ambiente real. O convívio com pessoas, ambientes, obrigações, hierarquias e processos onde o estágio se desenvolve, resultam em oportunidade valiosa para a formação profissional do egresso. O estágio curricular do presente curso é não obrigatório, ficando a critério do estudante realizá-lo ou não, e está definido em regulamento próprio do Instituto de Informática [INF 2023a].

Convém observar que o local de estágio está restrito a organizações devidamente conveniadas com a UFG, ou que se utilizam de agentes de integração conveniados. Adicionalmente, os seguintes papéis são obrigatórios no estágio: (a) coordenador de estágio do curso, (b) orientador (professor do curso no INF) e (c) supervisor (no concedente do estágio). As competências e atribuições de cada papel encontram-se no "Capítulo IV - Da Competência e das Atribuições dos Participantes do Estágio" [INF 2023a].

Para a formalização do estágio, o estudante deverá apresentar o Plano de Atividades, Termo de Compromisso (ou Termo Aditivo), Relatórios Semestrais e, por fim, o Relatório Final. As atividades desenvolvidas pelo estudante no estágio devem estar relacionadas ao objetivo geral do curso. Essas atividades deverão ser descritas em um Plano de Atividades, que deverá ser aprovado e acompanhado pelo coordenador de estágio, supervisor na empresa concedente e professor orientador do Instituto de Informática.

7.2. Política de inserção de ações curriculares de extensão - ACEx

No curso de Engenharia de Software, as Atividades Curriculares de Extensão (ACEx) seguem a Resolução CONSUNI/UFG nº 03/2008, que regulamenta as Ações de Extensão e Cultura na UFG, e a Resolução CEPEC/UFG Nº 1699/2021, que regulamenta as Atividades Curriculares de Extensão (ACEx) nos Projetos Pedagógicos dos Cursos de Graduação da UFG [ACEX 2021]. As ACEx são compreendidas como um processo interdisciplinar, educativo, cultural, científico, tecnológico e político que visa a fortalecer a integração entre ensino, pesquisa e extensão, de forma a assegurar a dimensão acadêmica da extensão na formação de estudantes e promover a articulação da comunidade acadêmica com a comunidade externa à UFG, por meio do diálogo, da troca de conhecimentos, da participação e da vivência com a realidade social.

Em consonância com os propósitos formativos do curso Engenharia de Software, cujo objetivo é formar cidadãos com capacitação sólida em engenharia de software, habilitados para a produção de modelos abstratos de software e a construção de código de qualidade, as ACEx serão realizadas integralmente no componente curricular "*Prática em Engenharia de Software*".

Esse componente curricular, que deve ser cumprido por todos os estudantes do curso, tem como objetivo a participação do estudante em um ou mais projetos integradores que aplicam competências adquiridas no decorrer do curso. Envolve a participação integral do aluno em projetos reais de Engenharia de Software realizados em ambiente típico da indústria de software. É importante contrapor esse cenário de "projeto real", onde todas as competências são exigidas, com o cenário das atividades práticas realizadas dentro do contexto de uma disciplina, normalmente limitado pelo conteúdo da própria disciplina. A Engenharia de Software compreende o uso de processos claramente explicitados no projeto pedagógico do curso e podem ser classificados em dois grandes tipos: Processos Técnicos de Engenharia de Software e Processos de Gestão de Engenharia de Software. Além disso, o domínio de Tecnologias de Engenharia de Software, aplicadas tanto a processos técnicos quanto a processos de gestão de Engenharia de Software, é uma competência obrigatória para o profissional que trabalha com software.

Assim, pode-se considerar que o Engenheiro de Software deve ser capaz de realizar atividades que envolvem três tipos de competências: técnicas, gerenciais e tecnológicas.

A disciplina “Prática em Engenharia de Software” fornece o ambiente necessário para o amadurecimento das competências dos discentes, pela aquisição de conhecimentos e desenvolvimento de suas capacidades e atitudes éticas e profissionais. Esta disciplina tem carga horária de 320 horas, o que corresponde a 10% da carga horária total do curso, acontece na fase final do currículo (sugestão que seja cursada no 9o período), e reforça as competências adquiridas ao longo das disciplinas teórico-práticas do curso, em um ambiente que representa, de forma realista, os cenários que serão experimentados no mundo do trabalho da Engenharia de Software.

O INF possui uma Fábrica de Software que desenvolve projetos com o objetivo de atender as necessidades de usuários e patrocinadores reais, ou seja, visam a geração de produtos e/ou a prestação de serviços em Engenharia de Software para a sociedade. Dessa forma, os projetos precisam atender, por exemplo, requisitos de qualidade, de escopo e restrições de custo e prazo definidos pelas partes interessadas do projeto em questão. Para isso, a Fábrica de Software trabalha com processos definidos com base nas normas técnicas aplicáveis da Engenharia de Software.

Todas as atividades da "Prática em Engenharia de Software" previstas no BES são realizadas na Fábrica de Software, sob a supervisão direta de docentes do BES. Portanto, a cada semestre letivo, a Fábrica de Software recebe docentes e estudantes que atuam como colaboradores em seus projetos. Cabe ressaltar que, embora projetos e atividades da Fábrica de Software não estejam limitados pelo calendário acadêmico da UFG, o processo de Gestão do Portfólio da Fábrica de Software tem o compromisso de selecionar projetos e atividades que apresentem um contexto adequado ao exercício das práticas de Engenharia de Software pelos discentes, como descritas neste PPC.

Desta forma, o estudante tem a oportunidade de integrar teoria e prática, envolvendo-se em situações-problema geradas pela experiência de campo e que exigem capacidades cognitivas, técnicas e comportamentais, bem como adoção de novas condutas e atitudes. É também uma oportunidade para que o discente seja avaliado quanto à sua atitude ética e postura profissional,

quanto ao respeito às normas institucionais definidas pela Fábrica de Software e quanto à sua relação com os demais envolvidos nos projetos, incluindo usuários e patrocinadores.

A “Prática em Engenharia de Software” é realizada pelo estudante preferencialmente após ter obtido aprovação nas demais disciplinas do BES, é ofertada a cada semestre letivo e os docentes atuam como preceptores dos discentes. Neste sentido, o docente é o responsável pela integração teoria-prática ao longo do projeto, ensinando, supervisionando, orientando e conduzindo o aluno na prática efetiva de sua futura profissão. Os estudantes são divididos em três grupos para melhor acompanhamento das atividades. Cada grupo, em um dado instante, realiza um tipo de atividade típica da profissão: processos técnicos, processos de gestão ou domínio de tecnologias. Ao final da disciplina, é garantido que todos os grupos vivenciaram atividades dos três tipos. A avaliação do estudante é formativa, enfatizando o seu acompanhamento durante todo o período letivo, com o intuito de não apenas verificar se o estudante está alcançando os objetivos propostos, mas também informando seus erros e acertos, além de promover o estímulo necessário para continuar os estudos e o seu desenvolvimento.

Concluindo, a disciplina “Prática em Engenharia de Software” favorece um ensino baseado na prática, com foco centrado na ética e na postura do profissional de Engenharia de Software perante os desafios que ocorrem quando se trabalha com software, que é um dos artefatos mais complexos que a humanidade produz. Quando o discente atua em atividades práticas relevantes para o seu mundo do trabalho, enfrentando os desafios reais de sua profissão ainda durante sua formação, sua compreensão se torna cada vez mais crítica e comprometida com a sociedade na qual se insere. Portanto, essa disciplina estimula e valoriza as dimensões ética e humanística na formação do Engenheiro de Software, desenvolvendo atitudes e valores orientados para a cidadania e para o desenvolvimento da sociedade.

A execução das ACEx no componente curricular “Prática em Engenharia de Software” estará prevista no conteúdo programático do plano de ensino [ACEX 2021], nos termos do RGCG e contempla ações caracterizadas como programas, projetos ou prestação de serviço e que estejam cadastradas e aprovados na Pró-Reitoria de Extensão e Cultura (PROEC).

Outros detalhes sobre ACEx, em conformidade com § 4o, Art. 13, IN CEPEC n. 01/2022, são definidos pelo *Regulamento de Ações Curriculares de Extensão (RACEx) do Curso do Bacharelado em Engenharia de Software*.

Sem embargo, os discentes também são estimulados a desenvolver, ao longo do curso, atividades não curriculares de extensão, por meio da participação em ações extensionistas (tais como programa; projeto; curso; evento; e prestação de serviços) devidamente coordenadas e registradas pelos órgãos competentes da UFG. A participação em ações não curriculares de extensão pode ser registrada como Atividade Complementar, o que contribui para que os discentes possam cumprir os requisitos de carga horária relativa a este tipo de atividade. Além disso, a livre escolha de atividades extensionistas permite que os discentes possam atuar em diferentes tipos de ações de extensão universitária, de acordo com seus interesses específicos, e também possibilita um enfoque adicional do estudante na sociedade, permitindo assim que ele contribua associando os conhecimentos teóricos com vivências práticas.

7.3. Políticas de ensino, pesquisa e extensão

As políticas de ensino, pesquisa e extensão do curso de Bacharelado em Engenharia de Software estão em consonância com o Plano de Desenvolvimento Institucional (PDI) da UFG [PDI 2023]. A integração do ensino com a pesquisa e a extensão ocorre por meio de atividades complementares, do estágio não obrigatório, de disciplinas do curso e da postura didática dos docentes.

As atividades complementares serão identificadas em resolução própria do INF vigente e perfazem pelo menos 192 horas e explicitamente incluem ações de extensão e de P&D. Exemplos dessas ações incluem divulgação de trabalhos em eventos científicos e participação em projetos de extensão e cultura e de P&D.

O estágio não obrigatório é um instrumento de integração. As atividades a serem realizadas pelo estudante podem incluir ações de P&D.

As disciplinas "Metodologia Científica" e "Engenharia de Software Experimental" oferecem a visão exigida para a realização de pesquisas na área. Isso significa não apenas orientar o estudante acerca da elaboração de

uma revisão sistemática da literatura, mas também da necessidade da expansão do conhecimento da área e dos mecanismos metodológicos atualmente empregados para realizar essa expansão.

Além disso, a disciplina "Prática em Engenharia de Software" representa as ACEx previstas no curso, com carga horária de 320 horas. Essa disciplina envolve em sua concepção o ensino, a extensão e a pesquisa ao se concentrar em projetos reais executados no ambiente de uma Fábrica de Software. Qualquer que seja o projeto, esse envolverá extensão e/ou P&D, naturalmente, ainda em um cenário de ensino e aprendizagem.

A integração, contudo, não ocorre apenas por meio dos elementos acima, que seriam pontos de integração localizados e em períodos específicos. A integração deve ocorrer oportunamente em cada ação do curso. Por exemplo, uma postura didática recomendável exige a contextualização de cada aula acerca do que será visto, da repercussão ou relação dela com a indústria de software (sociedade) e do estado da arte. De fato, essa postura não é apenas recomendável, mas exigida explicitamente por meio das “competências mínimas” definidas para as disciplinas específicas de Engenharia de Software. Ou seja, a integração do ensino com a pesquisa e a extensão ocorre ao longo de todo o curso.

7.4. Trabalho de Conclusão de Curso

O Trabalho de Conclusão de Curso (TCC) no BES é um componente curricular obrigatório regulado pelas Normas e Procedimentos de Trabalho de Conclusão de Curso do Bacharelado em Engenharia de Software [INF 2023b]. O TCC é desenvolvido na última etapa da graduação, durante a disciplina "Prática em Engenharia de Software", sob a orientação de um professor, e compreende um relatório e uma apresentação.

O relatório é individual e tem como objetivo a expressão do estudante na forma escrita, a capacidade de analisar, caracterizar, investigar, discutir, implantar, pesquisar, realizar, sintetizar e avaliar, entre outras. Neste sentido, o relatório descreve, de forma crítica, as atividades teórico-práticas e de formação profissional relacionadas ao desenvolvimento do estudante como

profissional, em especial, ao longo da disciplina "Prática em Engenharia de Software".

A apresentação do relatório, também individual, tem como objetivo a expressão do estudante, agora na forma oral, das atividades descritas no relatório, em sessão pública e para uma banca formada por professores do INF. A banca de professores tem o direito a um período de arguição e é responsável pela avaliação tanto do relatório quanto da apresentação.

7.5. NDE, monitoração e avaliação de ensino-aprendizagem e apoio a discentes

As atribuições do NDE (Núcleo Docente Estruturante) do BSE estão preconizadas institucionalmente na Resolução CEPEC Nº 1801 [UFG, 2023], em conformidade com a resolução da Comissão Nacional de Avaliação da Educação Superior (CONAES), 01/2010 de 17 de junho de 2010, a qual normatiza o núcleo docente estruturante, e pelo parecer CONAES 04/2010 de 17 de junho de 2010 [CONAES 2010].

O NDE não se apresenta como órgão ou mecanismo deliberativo, mas consultivo, reunindo docentes que continuamente refletem sobre questões pertinentes ao curso e, em consequência, definem ações que assistem, apoiam e fomentam a qualidade do curso, bem como contribuem com a execução dessas ações. Embora a atuação do NDE seja realizada em estreita interação com a coordenação do curso, ela não é, e nem pode ser vista, como meio para auxiliar a realização de fluxos administrativos, que representa uma atribuição da coordenação do curso.

O acompanhamento pelo NDE das ações realizadas para garantir que o estudante adquira o perfil esperado é subsidiado por informações obtidas por fontes externas e internas ao curso. A avaliação do curso pelo MEC e os resultados obtidos pelos estudantes no Exame Nacional de Desempenho de Estudantes (ENADE) são exemplos de insumos externos para a avaliação do processo de ensino. Internamente, são fontes valiosas de informação: a avaliação do docente pelo discente (instrumento formal e institucionalizado na UFG), questionários aplicados periodicamente aos estudantes, o

desempenho dos estudantes do curso nas disciplinas, os planos de ensino das disciplinas e consultas aos docentes.

O NDE do curso é o principal consumidor dessas informações, com o propósito de detectar oportunidades de melhoria nos processos de ensino-aprendizagem. Neste contexto, o NDE possui papel relevante na identificação de dificuldades e na busca de alternativas, em comunhão com os docentes do curso. Por exemplo, a Semana de Planejamento Pedagógico é um evento realizado na semana que precede o início de cada semestre letivo e propicia um ambiente adequado para o monitoramento e aperfeiçoamento da execução do PPC. Nesse evento, formalmente institucionalizado na UFG, são realizadas atividades visando a preparação para o semestre letivo, com destaque para a avaliação do plano de curso de cada disciplina pelo NDE.

É importante ressaltar que, embora o docente tenha autonomia metodológica ao planejar e executar o plano de ensino de uma disciplina, cabe ao NDE analisar o plano e, quando considerar oportuno, recomendar ajustes. Ao final dessa avaliação, os planos de curso devem ser ainda aprovados formalmente pelo Conselho Diretor da unidade antes do início de sua execução no período letivo. Outra ação relevante do NDE realizada na Semana de Planejamento Pedagógico é a reunião com os docentes das disciplinas do curso. Nesta oportunidade é realizada a apresentação e discussão sobre lições aprendidas no período letivo anterior. Esta ação provê visibilidade de processos de ensino exitosos, valoriza o docente responsável e estimula a adoção desse processo pelos demais docentes. Outras ações são realizadas de apoio docente, tais como: (i) a seleção semestral de monitores via edital para as disciplinas nas quais o docente identifica essa necessidade; (ii) a tutoria educacional aos alunos do curso, para dirimir dúvidas que possam surgir no ingresso à UFG, ao INF e ao curso em si; (iii) o incentivo a estágios não curriculares, na forma de divulgação de vagas nos principais órgãos de apoio a estágio; (iv) apoio psicopedagógico, com a existência de uma assessoria educacional dentro do Instituto; e (v) o apoio à mobilidade nacional e internacional, na forma de editais específicos.

Cabe destacar que o acompanhamento da execução do PPC pelo NDE não está restrito à Semana de Planejamento Pedagógico, mas é realizada de maneira contínua ao longo dos semestres letivos. O objetivo é viabilizar a

introdução de uma eventual mudança o mais próximo possível do instante no qual a necessidade é detectada. Neste contexto, o NDE pode realizar reuniões com os docentes ao longo do semestre letivo com intuito de identificar necessidades de ajustes e garantir que estes possam ser aplicados nos planos de ensino do semestre. Outro evento importante é a Semana de Integração do INF, realizado no início de cada semestre letivo, com foco no apoio aos discentes, em particular os ingressantes. São realizadas várias atividades visando apresentar as normas e procedimentos da UFG, como prazos e metas de integralização curricular, detalhes relevantes sobre o PPC do curso, a infraestrutura física e as diversas oportunidades oferecidas em ações de ensino, pesquisa e extensão.

Sobre as ações específicas para avaliação da aprendizagem, estas podem ser materializadas de várias formas. São comuns avaliações (provas) escritas e individuais, a avaliação por meio de relatórios ou trabalhos escritos e apresentações orais, dentre outras. A avaliação contínua por meio de exercícios realizados ao longo de toda uma disciplina também é uma forma. Vale notar que um método adequado de avaliação em um cenário não o é em outro. De maneira geral, cada avaliação deve atender, no seu planejamento e na forma contínua de sua execução, o estipulado pelo RGCG da UFG [RGCG 2022], notadamente em seu Capítulo IV, Seção 1 - “Da Avaliação”.

A avaliação dos processos de ensino-aprendizagem está estreitamente conectada à avaliação do próprio curso. O sistema de avaliação do BES tem como objetivo primeiro de sua aplicação, por meio de sua estrutura curricular, utilizando instrumentos pedagógicos constantes nos princípios norteadores, permitir que cada estudante adquira o perfil desejado. A avaliação do estudante deve levar em consideração não apenas o atendimento de requisitos técnicos de produtos e processos das diversas disciplinas da Engenharia de Software, mas também demonstrar postura ética e profissional ao desenvolver ações do curso.

Convém observar que as disciplinas específicas de Engenharia de Software (Seção 6.4.2) são acompanhadas de definição de competências mínimas, que identificam elementos obrigatórios a serem satisfeitos pelo estudante para ser considerado aprovado na disciplina. A intenção é estabelecer um contrato claro do compromisso do estudante e do docente com a disciplina em

questão. A avaliação de cada uma dessas disciplinas deve observar as competências mínimas estabelecidas. De fato, a condução de toda a disciplina deve ser orientada por tais competências.

Dada a especificidade da disciplina “Prática em Engenharia de Software”, o sistema de avaliação também é específico. Nesse caso, as avaliações devem envolver o domínio de processos e a geração de produtos de Engenharia de Software. Em particular, atividades avaliativas podem ser realizadas mesclando teoria e prática em um ambiente real de desenvolvimento de software, no qual a qualidade estabelecida para os entregáveis do projeto em questão é verificada.

7.6. Gestão do curso e avaliação interna e externa

O presente PPC deverá ser revisado formalmente a cada cinco anos, em ação proativa, ou em intervalo de tempo menor, na ocorrência de evento que justifique tal decisão. O NDE é responsável pelas revisões do PPC e, em particular, pela contínua vigilância da adequação do PPC ao cenário corrente. Esse esforço é realizado de duas formas: (a) zelando para que as orientações do PPC sejam observadas no cotidiano do curso; e (b) analisando possíveis melhorias para reagir adequadamente às mudanças do ambiente.

Ao acompanhar o curso, o NDE continuamente coleta, detecta e propõe alterações. As demandas podem se originar dos estudantes e dos docentes. Além disso, o curso considera as avaliações internas, realizadas pela Comissão Própria de Avaliação (CPA), e externas, realizadas pelo MEC/INEP (ENADE, Conceito Preliminar de Curso e relatórios de avaliação in loco) [UFG 2022].

7.7. Política de qualificação do corpo docente e técnico-administrativo

Desde a sua criação o INF implementa uma política vigorosa de capacitação do seu corpo docente, a qual inclui a meta de que todos os docentes tenham a titulação de doutor. De fato, a liberação de docentes para prosseguirem os seus estudos de pós-graduação é incentivada e, em consequência, nenhum pedido para afastamento com o propósito de realizar doutorado foi negado

até o momento. Essa política é implementada por regras institucionalizadas para concessão de licença para capacitação ou concessão de afastamento para doutorado ou pós-doutorado.

A qualificação de docentes do INF também pode ser obtida pela sua participação em congressos, simpósios, dentre outros eventos, quer seja nos papéis de autor de artigo, organizador de eventos, membro de comitê de programa ou avaliador de artigos. O Instituto de Informática ainda financia ou co-financia viagens e inscrições de seus docentes em congressos e simpósios importantes, principalmente quando há publicação de artigo.

Em harmonia com a política de capacitação docente, o INF estimula o aprimoramento e capacitação de seu corpo Técnico-Administrativo em Educação (TAE). Normalmente, um TAE faz solicitação ao diretor, encarregado de registrar as intenções de afastamento no plano anual de capacitação e conduzir a apreciação das intenções pelo Conselho Diretor do INF, haja vista que é do interesse da unidade fomentar a especialização e capacitação dos seus colaboradores. É frequente a liberação das atividades de TAE para que possam participar de treinamentos, tanto em cursos esporádicos quanto em programas de pós-graduação. Em tempo, as ações de extensão do INF reservam vagas exclusivas para participação de TAEs, sem necessidade de pagamento (quando é o caso).

Do ponto de vista legal, o INF se apoia na Resolução CEPEC 1286/2014 de 2014 [CEPEC 2014], que regulamenta o afastamento de docentes para cursar Mestrado, Doutorado e estágios Pós-Doutorais, e na Resolução CONSUNI 02/2014 [CONSUNI 2014], que regulamenta o Programa de Capacitação e o Plano Anual de Capacitação dos TAEs. Em consonância com a Resolução CEPEC 1286/2014, o INF instrui o afastamento de docentes com a Resolução CD/INF n. 01 de 2014, que dá suporte ao planejamento administrativo e incentiva a participação de seus docentes, em cursos de doutorado, pós-doutorado e capacitação, no país e no exterior, de acordo com a sua política de pessoal para o ensino, a pesquisa, a extensão e a administração.

Por fim, cabe ao NDE monitorar o PPC e, consequentemente, fomentar ações que promovam a qualidade dos serviços oferecidos tanto por docentes quanto por TAEs.

7.8. Requisitos legais e normativos do curso

O BES contempla requisitos legais específicos, dentre outros, o ensino de Libras, a Política de Educação Ambiental e a Educação das Relações Étnico-raciais e da História e Cultura Afro-brasileira e Indígena, conforme detalhado nas seções seguintes. A atenção a tais requisitos se verifica por meio de disciplinas e de um Programa de Extensão, devidamente institucionalizado na unidade executora do curso.

7.8.1 Carga horária mínima, atividades supervisionadas e Libras

A carga horária total do BES é de 3200 horas, o que atende a Resolução N°2, de 18 de julho de 2007, que dispõe sobre a carga horária mínima e procedimentos relativos à integralização e duração dos cursos de graduação, bacharelados, na modalidade presencial [CNE/CES 2007].

De acordo com a Resolução CNE/CES 03/2007 de 2 de julho de 2007, cabe às Instituições de Educação Superior, respeitando o mínimo dos duzentos dias letivos de trabalho acadêmico efetivo, a definição da duração da atividade acadêmica ou do trabalho discente efetivo, o que compreende: (a) preleções e aulas expositivas; e (b) atividades práticas supervisionadas, tais como laboratórios, atividades em biblioteca, iniciação científica, trabalhos individuais e em grupo, práticas de ensino e outras atividades, no caso das licenciaturas.

O BES divide cada hora de atividade acadêmica em 45 minutos de preleções e aulas expositivas e 15 minutos de atividades supervisionadas. O planejamento de cada hora deve estar devidamente registrado no plano de cada disciplina. Em particular, este plano deve incluir, de forma clara, as atividades supervisionadas.

O Decreto 5.626/2005 regulamenta a Lei 10.436, de 24 de abril de 2002, e o artigo 18 da Lei 10.098, de 19 de dezembro de 2000 [LIB 2005], determinando que a disciplina curricular Língua Brasileira de Sinais (Libras) seja obrigatória para os cursos de licenciatura e para os cursos de Fonoaudiologia. O Decreto também estabelece, no Capítulo II, § 2°, que, nos demais cursos de educação superior, a disciplina curricular Libras seja

optativa. Em atenção a tal requisito legal, o BES inclui a disciplina Libras como uma das suas disciplinas optativas.

7.8.2. Política de educação ambiental e programa institucional de extensão

A Lei 9.795, de 27 de abril de 1999, institui a Política Nacional de Educação Ambiental, que é regulamentada pelo Decreto 4.281, de 25 de junho de 2002 [AMB 1999]. Conforme essa Lei, Seção II, Art. 10, "a educação ambiental será desenvolvida como uma prática educativa integrada, contínua e permanente em todos os níveis e modalidades do ensino formal". Adicionalmente, lê-se na Seção II, Art. 10, § 1º: "a educação ambiental **não** deve ser implantada como disciplina específica no currículo de ensino". O destaque (negrito) é do presente documento.

O BES trata a Educação Ambiental por duas linhas contínuas de atuação: uma baseada no planejamento das disciplinas e outra em programa institucional do Instituto de Informática. O planejamento da oferta de disciplinas deve se inspirar em possíveis estratégias de inserção de questões ambientais. Por exemplo, adoção de material em formato digital em vez de formato impresso. A apresentação do conteúdo de cada disciplina também pode se beneficiar do volume significativo de dados sobre o meio ambiente. Por exemplo, área desmatada ao longo do tempo; áreas de preservação; consumo de energia; emissão de CO_2; qualidade do ar; consumo de materiais poluentes e geração de lixo eletrônico. Tais dados podem ser empregados para ilustrar o funcionamento de algoritmos e visualização de informações, dentre outras possibilidades. Convém destacar que esses exemplos devem ser vistos como elementos de inspiração, a serem renovados continuamente.

Além dessas ações transversais, o tema "educação ambiental" também é abordado nos conteúdos programáticos de alguns componentes curriculares. Questões ambientais, como o descarte de lixo eletrônico, são apresentadas e discutidas na disciplina "Software, Empreendedorismo e Sociedade". Por sua vez, na disciplina "Interação Humano-Computador" destaca-se que sistemas interativos causam grande impacto no mundo, de modo que o seu design deve abordar a interação a partir da perspectiva do que é sustentável e este

design deve reconhecer a diversidade e contribuir para o aprimoramento dos valores humanos e do ambiente em que vivemos.

A UFG executa um Plano de Logística Sustentável (PLS), que estabelece práticas de sustentabilidade e racionalização de gastos e processos na administração pública [PLS 2014]. Esse plano é internalizado no Instituto de Informática (INF) por um Programa Institucional de Extensão (PIE). O PLS une o cotidiano da prática acadêmica com atitudes "sustentáveis" por meio de recomendações simples, como a impressão em ambos os lados de uma folha e a redução do uso de copos descartáveis, dentre muitas outras. O PIE do INF reúne ações que contemplam as orientações do PLS. O objetivo é colocar em prática essas orientações. Por exemplo, enquanto o PLS sugere a coleta seletiva, o PIE cria um repositório para coleta de pilhas e baterias já utilizadas, além de assegurar que aquele material coletado será descartado de forma correta.

O PIE possui objetivos e ações que incluem a educação ambiental, com uma constante avaliação dos resultados de ações como:

- Cursos de extensão cuja inscrição seja efetuada mediante entrega de lixo eletrônico;
- Elaboração de material de conscientização sobre consumo parcimonioso de água e energia;
- Palestras e cursos sobre TI Verde (*green computing*);
- Pesquisas sobre consumo e economia de energia por *data centers;*
- Campanhas de divulgação de informações sobre o meio ambiente;
- Monitoramento e divulgação de informações ambientais sobre Goiás;
- Divulgação de softwares que promovem o meio ambiente, por exemplo, evitando deslocamentos e reduzindo o consumo de energia.

O PIE é realizado por meio de ações que, tipicamente, enfatisam a socialização de conhecimento sobre questões pertinentes à formação dos brasileiros, o que invariavelmente inclui negros e indígenas. Esse conhecimento é indispensável para a promoção da diversidade, do respeito

às diferenças e da igualdade, independente de crença, gênero, idade, cor ou situação social.

O INF, por meio do PIE, atua nas relações étnico-raciais e no meio ambiente e promove a reflexão contínua sobre esses tópicos durante toda a permanência dos seus estudantes nessa unidade, sejam de graduação ou pós-graduação. Especificamente sobre questões de cunho étnico-racial, sem a intenção de ser uma apresentação exaustiva ou restritiva, são identificadas algumas ações que fazem parte do escopo do PIE do INF:

- Ações afirmativas por meio de cursos de extensão.
- Ações para promoção da diversidade.
- Palestras sobre a construção de uma sociedade justa.
- Palestras sobre a diversidade da formação dos brasileiros.
- Palestras sobre a história afro-brasileira e sobre a história africana.
- Palestras sobre os povos indígenas.
- Apresentações artísticas que valorizam a cultura africana e indígena.

7.8.3. Relações étnico-raciais e cultura afro-brasileira

A Resolução Nº 1, de 17 de junho de 2004, trata das Diretrizes Curriculares Nacionais para a Educação das Relações Étnico-Raciais e para o Ensino de História e Cultura Afro-Brasileira e Africana [CNE/CP 2004]. O objetivo é claro: "combater o racismo e as discriminações que atingem particularmente os negros. Nessa perspectiva, propõe a divulgação e produção de conhecimentos, a formação de atitudes, posturas e valores que eduquem cidadãos orgulhosos de seu pertencimento étnico-racial, descendentes de africanos, povos indígenas, descendentes de europeus, de asiáticos. "

Tal objetivo pressupõe a "adoção de políticas educacionais e de estratégias pedagógicas de valorização da diversidade", conforme consta na Resolução, assim como os princípios a serem observados para atendê-la: (a) consciência política e histórica da diversidade; (b) fortalecimento de identidades e de direitos e (c) ações educativas de combate ao racismo e a discriminações. O

Art. 7.º da Resolução ainda destaca: “as instituições de ensino superior, respeitada a autonomia que lhe é devida, incluirão nos conteúdos de disciplinas e atividades curriculares dos diferentes cursos que ministram, a Educação das Relações Étnico-Raciais, bem como o tratamento de questões e temáticas que dizem respeito aos afrodescendentes. ”

Tendo em vista o requisito legal estabelecido pela Resolução, duas linhas de atuação são adotadas pelo Bacharelado em Engenharia de Software: inserção de conteúdo pertinente em duas disciplinas curriculares obrigatórias e o Programa Institucional de extensão do Instituto de Informática, descrito na Seção 7.7.2. As disciplinas “Software, Empreendedorismo e Sociedade” e “Interação Humano-Computador” incluem, em seus ementários, tópicos pertinentes às exigências da Resolução. Em particular, elas remetem para o conhecimento de questões pertinentes ao continente africano e para o conhecimento e respeito à diversidade.

7.8.4. Educação em direitos humanos e transtorno do espectro autista

Em atendimento às Diretrizes Nacionais para a Educação em Direitos Humanos, conforme disposto no Parecer CNE/CP n° 8, de 06/03/2012, que originou a Resolução CNE/CP n° 1, de 30/05/2012, a disciplina “Software, Empreendedorismo e Sociedade” trata questões relevantes acerca de direitos humanos [CNE/CP 2012]: aspectos profissionais; participação de mulheres na computação e evolução de aplicações com acessibilidade, dentre outras, nas quais os direitos humanos são objeto de discussões mais aprofundadas. Na disciplina “Interação Humano-Computador”, as características humanísticas, culturais, de direitos humanos e de aspectos políticos também são abordados em maior profundidade.

A proteção dos direitos da pessoa com transtorno do espectro autista é fundamentada na Lei N.º 12.764 de 27 de dezembro de 2012, que institui a política nacional de proteção dos direitos da pessoa com transtorno do espectro autista, e altera o § 3.º do Art. 98 da Lei N.º 8.112, de 11 de dezembro de 1990 [PTEA 2012]. Esse requisito legal é atendido por meio da disciplina obrigatória “Interação Humano-Computador”, que inclui em seu ementário tópicos pertinentes ao tratamento de características humanísticas e biológicas

na construção de interfaces de usuário, o que possibilita a socialização de informações relevantes para que portadores do Transtorno do Espectro Autista possam ser adequadamente considerados por meio de suas necessidades especiais.

7.8.5. Acessibilidade para pessoas com deficiência ou mobilidade reduzida

A UFG, em consonância com Constituição Federal (arts. 205, 206 e 208); NBR 9050/2004, da ABNT; Lei n° 10.098,de 19/12/2000; Decretos n° 5.296, de 02/12/2004, n° 6.949, de 25/08/2009, n° 7.611, de 17/11/2011; e Portaria MEC n° 3.284/2003, Lei nº 13.146, de 6 de julho de 2015, criou em 2022 a Secretaria de Inclusão (SIN), uma unidade administrativa superior que atua no desenvolvimento, fortalecimento e concretização de políticas e ações da UFG voltadas para (a) reconhecimento da diferença e da diversidade; (b) inclusão de segmentos societários historicamente discriminados; e (c) promoção da igualdade de oportunidades e de tratamento.

A SIN busca assegurar a inclusão na educação superior, que compreende o ingresso e a permanência de pessoas com deficiência, surdas, negras, indígenas, quilombolas, de comunidades tradicionais, povos do campo, mulheres, mães, LGBTQIA+, migrantes, refugiadas e pertencentes a outros grupos e segmentos socialmente discriminados.

A Diretoria de Acessibilidade (DAC) da SIN tem como foco o respeito às diferenças, buscando a formação e a sensibilização da comunidade acadêmica, bem como a aquisição de recursos e tecnologias assistivas para o acesso a todos os espaços, ambientes, ações e processos educativos desenvolvidos na instituição. As diversas ações da DAC têm como objetivos:

- Conscientizar a comunidade universitária do direito das pessoas com deficiência e do processo de inclusão em um ensino superior público de qualidade, minimizando as barreiras atitudinais.
- Oferecer apoios diversos por meio de soluções para a eliminação de barreiras atitudinais, arquitetônicas, pedagógicas e de comunicação, buscando seu ingresso, acesso e permanência, favorecendo a aprendizagem, no ambiente universitário.

- Orientar as coordenações e professores dos cursos da Universidade Federal de Goiás na adequação curricular para atender às especificidades do estudante com necessidade educacional especial;
- Implantar e implementar a Política de Acessibilidade da UFG, juntamente com os órgãos e pró-reitorias desta instituição, visando institucionalizar as ações já existentes e deslumbrando ações futuras nos vários níveis da instituição.

Os docentes do INF trabalham em parceria com a Secretaria de Inclusão (SIN) da UFG, particularmente com a sua Diretoria de Acessibilidade (DAC), para proporcionar aos alunos com deficiência ou mobilidade reduzida um ambiente inclusivo e para fomento à educação superior inclusiva aos estudantes com deficiência física, visual, auditiva e intelectual, altas habilidades e superdotação por meio da eliminação ou minimização de barreiras atitudinais, arquitetônicas, pedagógicas, informacionais e comunicacionais.

Referências

- [ACEX 2021] Orientações Gerais para inserção das Atividades Curriculares de Extensão (ACEx) nos Projetos Políticos Pedagógicos (PPCs) da Universidade Federal de Goiás (UFG). 2021.

- [ACM/IEEE 1999] Software Engineering Code of Ethics and Professional Practice (Version 5.2) as recommended by the ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices and jointly approved by the ACM and the IEEE-CS as the standard for teaching and practicing software engineering. Disponível on-line em: https://ethics.acm.org/code-of-ethics/software-engineering-code/

- [ACM/IEEE 2020] Computing Curricula 2020: Paradigms for Global Computing Education. ACM, IEEE 2020. ISBN: 978-1-4503-9059-0.

- [AMB 1999] Lei 9.795, de 27 de abril de 1999. Dispõe sobre a educação ambiental, institui a Política Nacional de Educação Ambiental e dá outras providências, 1999.

- [APOEMA 2022] Apoema: Tecnologia e Inovação. Órgão complementar de pesquisa e desenvolvimento do Instituto de Informática. Disponível on-line em: http://apoema.inf.ufg.br.

- [CEIA 2022] Unidade EMBRAPII em Inteligência Artificial - CEIA/UFG – Centro de Excelência em Inteligência Artificial. Disponível on-line em: https://embrapii.org.br/unidades/unidade-embrapii-em-inteligencia-artificial-ceia-ufg-centro-de-excelencia-em-inteligencia-artificial/

- [CEPEC 2014]. Resolução CEPEC 1286/2014, de 6 de junho de 2014. Disciplina o afastamento de docentes da UFG para a realização de cursos de Pós-Graduação Stricto Sensu e Estágios de Pós-Doutorado. Universidade Federal de Goiás (UFG), 2014.

- [CEPEC 2017] Resolução CEPEC n.º 1431. Aprova o Projeto Pedagógico do Curso de Graduação em Engenharia de Software, grau acadêmico Bacharelado, modalidade presencial, do Instituto de Informática, para os alunos ingressos a partir de 2017.

- [CEPEC 2021]. Resolução CEPEC/UFG n.º 1699, 22 de Outubro de 2021. Dispõe sobre a regulamentação das Atividades Curriculares de Extensão (ACEx) nos Projetos Pedagógicos dos Cursos de Graduação da Universidade Federal de Goiás. Universidade Federal de Goiás (UFG), 2021.

- [CNE/CES 2007] Resolução CNE/CES n.º 2, de 18 de junho de 2007. Dispõe sobre carga horária mínima e procedimentos relativos à integralização e duração dos cursos de graduação, bacharelados, na modalidade presencial, 2007.

- [CNE/CES 2018] Resolução CNE/CES n.º 7, de 18 de dezembro de 2018. Diretrizes para a Extensão na Educação Superior Brasileira, 2018.

- [CNE/CP 2004] Resolução CNE/CP n.º 1, de 17 de junho de 2004. Institui Diretrizes Curriculares Nacionais para a Educação das

Relações Étnico-Raciais e para o Ensino de História e Cultura Afro-Brasileira e Africana, 2004.

- [CNE/CP 2012] Resolução CNE/CP n.º 1, de 30 de maio de 2012. Estabelece Diretrizes Nacionais para a Educação em Direitos Humanos, 2004.

- [CONAES 2010] Parecer CONAES n.° 4, de 17 de junho de 2010. Sobre Núcleo Docente Estruturante, 2010.

- [CONSUNI 2008] CONSUNI. RESOLUÇÃO CONSUNI 10/2008, 27/06/2008. Cria o curso de graduação em Engenharia de Software, modalidade Bacharelado, Instituto de Informática da UFG. Universidade Federal de Goiás (UFG), 2008.

- [CONSUNI 2014] RESOLUÇÃO CONSUNI 02/2014, 24 de janeiro de 2014. Regulamenta as Normas para o Programa de Capacitação, de que trata a Resolução ECU n.º 07/96, e para o Plano Anual de Capacitação dos servidores integrantes do Plano de Carreira dos Cargos Técnico-Administrativos em Educação – PCCTAE. Universidade Federal de Goiás (UFG), 2014.

- [FÁBRICA 2022] Fábrica de Software do Instituto de Informática da UFG. Disponível on-line em: https://fabrica.inf.ufg.br/.

- [INF 2023a] Resolução INF 06/2023 - Regulamento de Estágio no âmbito do curso de Bacharelado em Engenharia de Software.

- [INF 2023b] Resolução INF - Regulamento de Trabalho de Conclusão de Curso no âmbito do curso de Bacharelado em Engenharia de Software.

- [ISO/IEC/IEEE 2017] International Standard ISO/IEC/IEEE 12207:2017 – Systems and software engineering — Software life cycle processes, 2017.

- [LIB 2005] Decreto n.º 5.626, de 22 de dezembro de 2005. Regulamenta a Lei n.º 10.436, de 24 de abril de 2002, que dispõe sobre

a Língua Brasileira de Sinais - Libras, e o art. 18 da Lei n.º 10.098, de 19 de dezembro de 2000, 2005.

- [LOV 2008] Lucena, F.N.D., Oliveira, J.L.D. and Vincenzi, A.M.R.. *Bacharelado em Engenharia de Software na Universidade Federal de Goiás,* Fórum de Educação em Engenharia de Software (Simpósio Brasileiro de Engenharia de Software), número 43/08, páginas 16-21, Campinas/SP, 17/10/2008, ISSN 0103-9741.

- [MEC 2016] Diretrizes Curriculares Nacionais para os cursos de graduação na área da Computação. Resolução CNE/CES n.º 5, de 16 de novembro de 2016 (Brasil. MEC. CNE. CES, 2016).

- [PDI 2023] Plano de Desenvolvimento Institucional – PDI da Universidade Federal de Goiás. Disponível on-line em https://drive.google.com/file/d/1HaJIGWoQOSPIbe0q3g4060LnjHwRB4xd/view?usp=share_link.

- [PTEA 2012] Lei n.º 12.764, de 27 de dezembro de 2012. Institui a Política Nacional de Proteção dos Direitos da Pessoa com Transtorno do Espectro Autista, 2012.

- [PLS 2014]. Plano de Logística Sustentável da Universidade Federal de Goiás. Disponível on-line em https://sustentabilidade.prodirh.ufg.br/up/676/o/original_Site_PLS-UFG_2014-2015.pdf.

- [RGCG 2022]. Resolução CEPEC/UFG n.º 1791, DE 07 DE OUTUBRO DE 2022. Regulamento Geral dos Cursos de Graduação (RGCG). Universidade Federal de Goiás (UFG), 2022.

- [SWEBOK 2023] Bourque, P. e Fairley, R. E. (editores). *Guide to the Software Engineering Body of Knowledge*, V.4, IEEE Computer Society, 2023. Disponível on-line em: http://swebok.org.

- [SWECOM 2014] Software Engineering Competency Model (SWECOM), IEEE Computer Society, 2014. ISBN 978-0-7695-5373-3. Disponível on-line em: https://www.computer.org/web/peb/swecom.

- [UFG 2022] Instrução Normativa Nº 01/2022 - Diretrizes e procedimentos para elaboração de Projetos Pedagógicos de Curso (PPC) de graduação da Universidade Federal de Goiás. Disponível em https://files.cercomp.ufg.br/weby/up/90/o/Instrucao_Normativa_01-2022_PPC.pdf.

- [UFG 2023] Resolução CEPEC n.º 1801, de 13 de janeiro de 2023. Dispõe sobre o Núcleo Docente Estruturante (NDE) dos cursos de graduação da Universidade Federal de Goiás (UFG), 2023.

- [ZNM 2017] Zorzo, A. F.; Nunes, D.; Matos, E.; Steinmacher, I.; Leite, J.; Araujo, R. M.; Correia, R.; Martins, S. "Referenciais de Formação para os Cursos de Graduação em Computação". Sociedade Brasileira de Computação (SBC). 153p, 2017. ISBN 978-85-7669-424-3.